Data Structure Using C

- A Course File

Prepared by the Special Academic Group

Silicon
...beyond teaching

Silicon Institute of Technology, Bhubaneswar.

1. Introduction and Classification of Data Structure

Data Type:

A data tepe is a set of values and set of operations defined on those values.

Example: **int** is a data type

Let **int** occupies 2 bytes then it can assign a set of values -327678 to 32767.Which is a set of values. Set of operation like addition, multiplication, subtraction and division and modulus.

Abstract Data Type:

An abstract data type is the specification of datatype without detail impletion.
Example: Array
Set of values can be int, char, float, double data. Set of operation can creation be, initialize, traverse, findmin, findmax, sorting, searching, reverse, insertion, deletion.

Data Structure:

Data structure is the way of organizing data for performing different operation on it. A data structure is an abstract data type where we specifying a set of data and a set of operation.
Like stacks and queues are used in operating system and Trees are used for any hierarchical data like directory structure and Graphs for representing networks of internet, etc.

Classification of data structure:

There are basically two types of data structure, primitive and non primitive data structure. Where primitive data structure are the basic data type of a language and non primitive data structure are composed of one or more primitive data structure.

Non primitive data structures are of two type linear and non linear data structure. A data structure is said to be linear data structure if each element has only one predecessor and one successor. A data structure is said to be non-linear data structure if each element has more than one predecessor or more than one successor. Stacks and Queues are linear data structure where as trees and graphs are non linear data structure.

2. Asymptotic Notations

It is the way to describe the time complexity of an algorithm. So lets start with complexity calculation of an algorithm.

Algorithm:

An algorithm is a finite step of instruction which accomplish a particular task and it must satisfy the following criteria.

I. Input : It should take zero or more values as input .
II. Output: It must produce one or more output values
III. Finiteness: It should have finite no of steps.
IV. Definiteness: It means each instruction should be clear and unambiguous.
V. Effectiveness: It should produce correct output.

Complexity of an Algorithm:

It describes the efficiency of an algorithm. There are two types of complexity

- Time complexity
- Space complexity

Time complexity:
It is the amount of time the algorithm takes to execute. Here time is the number of memory access, number of comparison or number of any operation in the algorithm.
Space complexity:
It is the amount of space the algorithm takes at the time of execution.

Out of these two complexity space complexity is depends of language where as time complexity is language independent. So we emphasize on time complexity and learn how to compute it.

Time complexity is represented by function f(n) where n is the input f the algorithm and f(n) is the number of time the statements will execute.Function can be any one of the following

Constant function f(n) = 1
Linear fnction f(n)=n+1
Quadratic function $f(n)=n^2+n+1$
Cubic function $f(n)=n^3+1$
Logarithm function=log n
Exponentioal function $f(n)=e^n$

Power function $f(n)=2^n$

Time complexity Computation of Linear Search

```
int search(int *a,int n,int x)
{
        int i;                                  1
        for(i=0;i<n;i++)                        n+1
                if(a[i]==x)                     n
                        return i;               1
        return -1;                              1
}                                       Σ
                                        f(n)=2n+4
```

Here f(n) is the sum of the maximum number of time each statement can execute on input n. The time complexity function depends on input n. Based on the input there can be three different time complexity.

i. Best case
ii. Average case
iii. Worst case

In the above linear search function, if the element to search is the first element then it is a best case as the loop will execute only once. If the element to search is not present then it the worst case as the loop will execute n+ 1 time. If the loop will execute more than one and less than n+1, then it's the average case.

Asymptotic Notations:

It defines the order of growth of an algorithm which is useful to measure the performance of an algorithm. There are different types of notations

i. Big Oh (O) – Used to represent the upper bound value
ii. Big mega (Ω) - Used to represent the lower bound value
iii. Theta (Θ) – Used to represent the tightly value
iv. Lttle oh (o) -
v. Little Omega (ω) -

So best case of linear search is O(1) and average case and Worst case is O(n).

Big Oh (O) :

It describes the upper bound time complexity of an algorithm. Let's take two function f(n) and g(n) ,then we can say f(n) = O (g(n)) , iff f(n) < = c. g(n) where constant c > 0 and the initial value n_0 <= n .

Example:

f(n) = 2n+4 and g(n)= n

We can f(n) = O (n) , if we can get a constant C and an initial value n_0 where f(n) < = c. g(n) always.

2n+4 <= C. n

Let take n_0 = 1, then 2 * 1 + 4 <= C* 1

⇨ 6 <= C

So we can take minimum 6 as the constant C.

Hence f (n) < = c. g(n) for n_0 = 1 and C=6. (Proved)

Big Omega (Ω):

It describes the lower bound time complexity of an algorithm. Let's take two function f(n) and g(n) ,then we can say f(n) = Ω (g(n)) , iff f(n) > = c. g(n) where constant c > 0 and the initial value n_0 <= n .

Example:

f(n) = $2n^2$+4 and g(n) s= n^2

We can f(n) = Ω (n^2) , if we can get a constant C and an initial value n_0 where f(n) > = c. g(n) always.

$2n^2$+4>= C. n^2

Let take n_0 = 1, then 2 * 1 + 4 >= C* 1

⇨ 6 >= C

So we can take minimum 6 as the constant C

. So f(n) >= c. g(n) for n_0 = 1 and C=6. (Proved)

Theta (Θ):

It describes to represent the tightly time complexity of an algorithm. Let's take two function f(n) and g(n) ,then we can say f(n) = Θ (g(n)) , iff c_1 g(n) <= f(n) < = c_2 g(n) where constant c_1 and c_2 > 0 and n_0 <= n

Insertion Sort:

```
void insertion (int *n, int n)
{
int i,j,x;
for(i=1;i<n;i++)
{
x=a[i];
for(j=i-1;j>0 && x>a[i];j--);
        a[j+1]=a[j];
a[j+1]=x;
}
}
```

Best cast O (n) and worst case O (n^2)

Bubble Sort:

```
void bubble(int a[], int n)
{
        int i, j, t;
        for(i= 0;i< n; i++)
        {
                for(j=0; j<n-1;j++)
                {
                        if(a[j]< a[j+1])
                        {
                                t= a[j];
                                a[j]= a[j+1];
                                a[j]=t;
                        }
                }
        }
}
```

Best cast and worst case O (n^2)

Binary Search:

```
int binary_search(int a[],int n,int k)
{
        int l=0, mid, r=n-1;
        while(l<= r)
        {
```

```
            mid= (l+r)/2;
            if(k== a[mid])
                return mid;
            else if( k> a[mid])
                l= mid+1;
            else
                r= mid-1;
        }
}
```

Best cast O(n) and worst case O (n^2)

3. Storage Structure For Arrays

1D array:

Int a[4]; Let integer takes two bytes.

100	102	104	106
10	50	30	20
0	1	2	3

Address of a[i] = base address + (i x size of element).

In the above array base address is 100.
Address of a[2] = base address + (2 x size of element)
= 100 + (2 x 2) = 104

2D array:

In 2D array values can be assigned in two was

I. Row major order
II. Column major order

Row major order:

int a[2][3]; Let integer takes two bytes.

	0	**1**	**2**
0	**10**	**5**	**30**
	1000	1002	1004
1	**15**	**25**	**20**
	1006	1008	1010

In the above array base address is 1000
Address of a[i][j] = base address + { [(i x total no of column) + j] x size of element }

Address of a[1][1] = 1000 + { [(1 x 3) +1] x 2 } = 1000 + 8 = 1008

Column major order:

int a[2][3];

	0	**1**	**2**
0	**10**	**5**	**30**
address	1000	1004	1008
1	**15**	**25**	**20**
address	1002	1006	1010

Address of a[i][j] = base address + { [(j x total no of row) + i] x size of element }

Address of a[1][1] = 1000 + { [(1 x 2) +1] x 2 }
= 1000 + 6 = 1006

4. Sparse Matrix

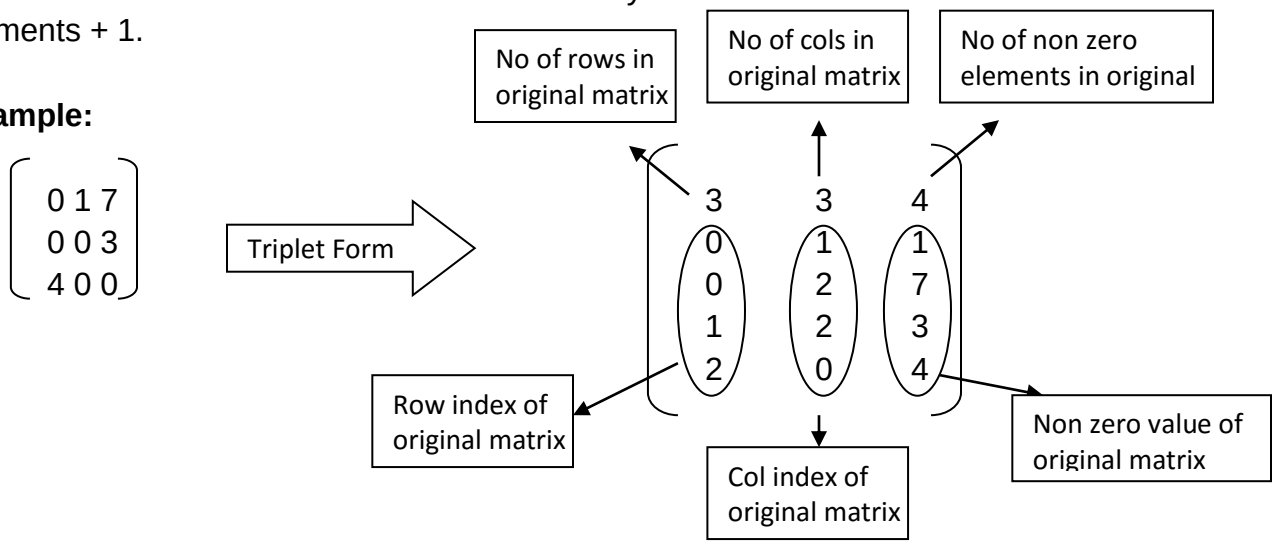
It is a matrix contains more number of than the non zero elements.

$$\begin{pmatrix} 0 & 0 & 7 \\ 0 & 0 & 0 \\ 0 & 4 & 0 \end{pmatrix}$$

Triplet representation of Sparse Matrix

It is a nx3 matrix. So there are 3 columns always. The no of rows n is the no of non zero elements + 1.

Example:



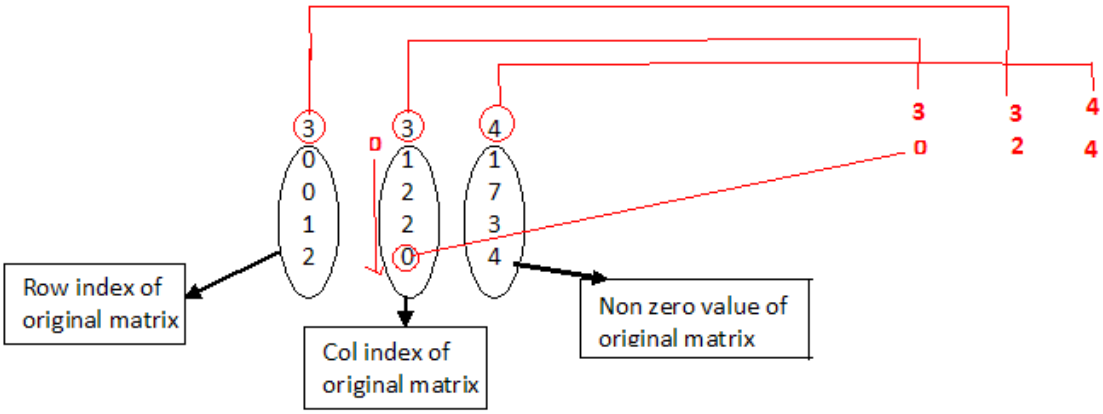

```
void triplet(int a[][SIZE], int spm[][3] ,int r,int c)
{
        int i,j;
        int k=1; //Row index of sparse matrix
        int m=0; //Counter that counts no. of non zero elements
        for(i=0;i<r;i++)
        {
                for(j=0;j<c;j++)
                {
                        if(a[i][j]!=0)
                        {
                                spm[k][0]=i;
                                spm[k][1]=j;
                                spm[k][2]=a[i][j];
                                k++;
```

```
                        m++;
                    }
                }
            }
            spm[0][0]=r;
            spm[0][1]=c;
            spm[0][2]=m;
            printf("The required triplet matrix is:\n");
            display(spm,(m+1),3);

    }
```

5. Finding transpose of a sparse matrix:

For finding transpose of a sparse matrix from its triplet form follow the following procedure. Find the column index from the 2nd column of the triplet matrix for 0th column and repeat the process up to the last column index.

Example:



Here we have searched for 0 index and there is only one 0 values. If there will be more number of zero then first 0 will copied first. In this example there are two number of 2's.See how these values are copied.

$$\begin{pmatrix} 3 & 3 & 4 \\ 0 & 2 & 4 \\ 1 & 0 & 1 \\ 2 & 0 & 7 \\ 2 & 1 & 3 \end{pmatrix}$$

transposed matrix of Triplet

```
void triplet_transpose(int spm[][3], int tspm[][3])
{
        int i,j;
        int k=1; //Row index of the matrix

        tspm[0][0]=spm[0][1];
        tspm[0][1]=spm[0][0];
        tspm[0][2]=spm[0][2];
        for(i=0;i<spm[0][1];i++)
```

```
        {
                for(j=1;j<=spm[0][2];j++)
                {
                        if(i==spm[j][1])
                        {
                                tspm[k][0]=spm[j][1];
                                tspm[k][1]=spm[j][0];
                                tspm[k][2]=spm[j][2];
                                k++;
                        }
                }
        }
        printf("The required transpose matrix is displayed below:\n");
        display(tspm,k,3);
}
```

6. Stacks: Representation, Basic Operations

Stack is an ordered list in which insertion and deletion is made at one end called TOP. The insertion operation is called PUSH and the deletion operation POP.

Stack can be represented in two way.

- Array
- Linked List (to be discussed in Linked List chapter)

Array Representation of Stack:

In this representation an array is considered as a STACK. The size of array is the size of STACK. TOP is initialized to -1 .TOP will take the value from 0 to size-1 which are the array indexes.

Basic Operations of Stack:

Let's consider a character STACK of SIZE = 3.

```
char STACK[SIZE];
```

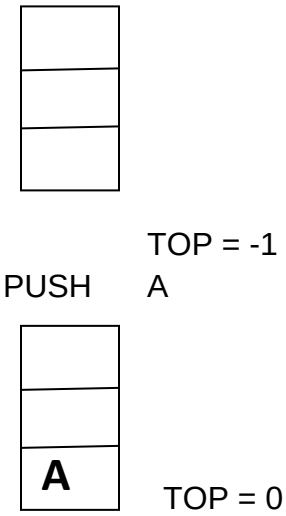
Initially the STACK is empty; it means there is no element in the STACK.

```
int TOP = - 1 ;
```

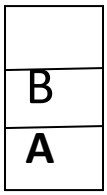
Push operation:

In a push operation the TOP value is increased by one and that top index of STACK, the new element is stored. But if the TOP value reaches at SIZE-1, then it can't increase. If the user wants to push any element then it's called an overflow condition or the STACK is full.

Example:

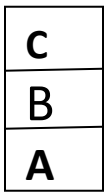


PUSH B



TOP = 1

PUSH C



TOP = 2 and the STACK is full.

As the STACK is full now, trying to PUSH any more elements will cause STACK overflow.

STACK full:

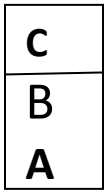
We say the stack is full if TOP reaches to SIZE-1.

```
if(top == size-1)
        printf("\n STACK is full");
```

Pop operation:

In a POP operation the element at TOP index of STACK is returned and the TOP value is decreased by one. But if the TOP value reaches at -1, then it can't decrease. If the user wants to POP any element then it's called an underflow condition or the STACk is empty.

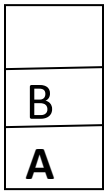
Example:



TOP = 2

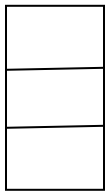
X = POP();

X will contain C and the STACK is as follows.



TOP = 1

Similarly after two POP (), the TOP value reaches to -1 and the STACK becomes empty.



TOP = -1

As the STACK is empty now, trying to POP any more elements will cause STACK underflow.

LIFO: Since the last element in the STACK is the first element removed / deleted .The stack is also known as Last In First Out (LIFO) data structure.

```
/*Program for implementing basic operation of stack */
//Demonstration of implementation of STACK of character
#include<stdlib.h>
#include<stdio.h>
#define SIZE 3
char stack[SIZE];
int top=-1;
void push(char);
char pop();
void traverse();

main()
{
        int ch;
        char x;
        printf("####*****  MENU  *****####\n");
        printf("1.PUSH\n2.POP\n3.TRAVERSE\n4.EXIT\n");
        while(1)
        {
                printf("Enter your choice\n");
                scanf("%d",&ch);
                switch(ch)
                {
                        case 1:
```

```
                        printf("Enter the element\n");
                        scanf(" %c",&x);
                        push(x);
                        break;
                case 2:
                        printf("The popped element is %c \n",pop());
                        break;
                case 3:
                        traverse();
                        break;
                case 4:
                        printf("Thankyou\n");
                        exit(0);
                default:
                        printf("Wrong Input\n");
            }
        }

}
void push(char c)
{
        if(top==SIZE-1)
                printf("STACK IS FULL\n");
        else
        {
                top=top+1;
                stack[top]=c;
        }
}
char pop()
{
        if(top==-1)
                printf("STACK IS EMPTY\n");
        else
                return(stack[top--]);
}
void traverse()
{
        int i=top;
        while(i>=0)
```

```
        {
                printf("%c\n",stack[i]);
                i--;
        }
}
```

7. Stack Application

There are several applications of stack, like function call in Operating System, Parsing of compiler, parenthesis matching, and String Matching, Conversion and Evaluation of Arithmetic expression.

Conversion and Evaluation of Arithmetic expression:

There are three types of expression infix, prefix and postfix.

Infix: In an infix expression operator is placed in between two operands i.e. a + b.
Prefix: In prefix expression operator is placed before two operands i.e. + a b.
Postfix: In postfix expression operator is placed after two operands i.e. a b +.

Infix to Postfix conversion:

All operators are shifted to the right after there operand as per there precedence and associativity.
Example:

	3 + 4 * 5	
Step 1	**3 + 4 5 ***	(as * has higher precedence over +)
Step 2	**3 4 5 * +**	(as + has two operands 3 and 4 5 *)

Compiler performs the above procedure by using a stack as per the following rules.

Infix to postfix conversion using Stack:

Requirements:
Take two character arrays Infix and Postfix and a character Stack and define a precedence function which returns the precedence of each operator.

Procedure:

I. Add closing parenthesis at the end of the infix expression
II. Push opening parenthesis to the stack
III. Scan each character form the infix expression from left to right.
    a. If the character is opening parenthesis ,push it to stack
    b. If the character is an operand then take it to the postfix expression.

c. If the character is an operator, then pop all the operators from stack which has higher or equal precedence over it and add them to postfix expression and push it to the stack.
d. If the character is closing parenthesis then pop all the operators from stack until it encounters opening parenthesis and add them to postfix expression and pop opening parenthesis from the stack

Example:

Find the postfix expression of the Infix expression **3 + 4 * 5.**

Infix	Stack	postfix
3	**(**	3
+	(+	3
4	(+	3 4
*****	(+ *	3 4
5	(+ *	3 4 5
)		3 4 5 * +

Example:

Find the postfix expression of the following Infix expression.

(A + B) * C / D

Infix	Stack	postfix
(	**((**	
A	**((**	**A**
+	**((+**	**A**
B	**((+**	**A B**
)	**(**	**A B +**
*****	**(***	**A B +**
C	**(***	**A B + C**
/	**(/**	**A B + C ***
D	**(/**	**A B + C * D**
)		**A B + C * D/**

```
/*Function to define precedence*/
int precidence(char ch)
{
        switch(ch)
```

```
	{
		case '(':
			return 0;
		case '+':
		case '-':
			return 1;
		case '*':
		case '/':
		case '%':
			return 2;
		case '^':
			return 3;
	}
}

// FUNCTION FOR INFIX TO POSTFIX CONVERSION

void conversion(char infix[], char postfix[])
{
	char ch;

	int i=0; //index variable for infix character array
	int j=0; //index variable for postfix character array
	int len; //variable storing the length of the infix expression
	len = strlen(infix);
	infix[len]=')';
	infix[len+1]='\0';
	push('(');


	while(infix[i]!='\0')
	{
		ch= infix[i];

		switch(ch)
		{
			case '(':
				push('(');
				break;
			case ')':
```

```
                        while(stack[top]!='(')
                                postfix[j++]=pop();
                        pop();
                        break;
                case '+':
                case '-':
                case '*':
                case '/':
                case '%':
                case '^':
                        while(precidence(stack[top])>=precidence(ch))
                                postfix[j++]=pop();
                        push(ch);
                        break;
                default: //for operands
                        postfix[j++]=ch;
        }
        i++;
    }


    postfix[j]='\0';


}
```

8. Infix to Prefix conversion

The operator are taken before the operands as per there associativity and precedence.

Example:

	3 + 4 * 5	
Step 1	**3 + * 4 5**	(as * has higher precedence over +)
Step 2	**+ 3 * 4 5**	(as + has two operands 3 and 4 5 *)

Infix to prefix conversion using Stack:

Requirements:

Take two character arrays Infix and prefix and a character Stack and Define a precedence function which returns the precedence of each operator.

Procedure:

i. Add opening parenthesis at the beginning of the infix expression
ii. Push closing parenthesis to the stack
iii. Scan each character form the infix expression from right to left.
   a. If the character is closing parenthesis ,push it to stack
   b. If the character is an operand then take it to the prefix expression (add at beginning).
   c. If the character is an operator, then pop all the operators from stack which has higher precedence over it and add them to prefix expression (add at beginning) and push it to the stack.
   d. If the character is opening parenthesis then pop all the operators from stack until it encounters closing parenthesis and add them to prefix expression (add at beginning) and pop closing parenthesis from the stack

Example Find the prefix expression of the Infix expression **3 + 4 * 5.**

Infix	Stack	prefix
5	**)**	5
*****	**) ***	5
4	**) ***	4 5
+	**) +**	* 4 5
3	**) +**	* 4 5
(		+ 3 * 4 5

Example: Find the prefix expression of the Infix expression **(A + B) * C / D**

Infix	Stack	prefix
D	**)**	D
/	**)/**	D
C	**)/**	C D
*****	**)/***	C D
)	**)/*)**	C D
B	**)/*)**	B C D
+	**)/*)+**	B C D
A	**)/*)+**	A B C D
(	**)**	+ A B C D
(		/ * A B C D

9. Postfix and Prefix Expression Evaluation

Procedure to evaluate Postfix expression using stack

I. Add closing parenthesis at the end of postfix expression
II. Scan each character of postfix expression until the closing parenthesis encounter
   a. If the character is an operand ,push it to stack
   b. If the character is an operator then pop the top two element of the stack where the first popped element will be the 2nd operand and the next popped element will be the 1st operand .Then perform the operation on the scanned operator and these popped operands and push the result to the stack.

Example: Evaluate the postfix expression **3 4 5 * +**

Postfix	**Stack**	**Action**
3	**3**	
4	**3 4**	
5	**3 4 5**	
*****	**3 20**	**4*5 =20, push 20**
+	**23**	**3+20=23,push 23**
)		

The result of evaluated expression is 23.

Procedure to evaluate Prefix expression using stack

I. Add opening parenthesis at the beginning of prefix expression
II. Scan each character of prefix expression from right to left until the opening parenthesis encounter
   a. If the character is an operand ,push it to stack
   b. If the character is an operator then pop the top two element of the stack where the first popped element will be the 1st operand and the next popped element will be the 2nd operand .Then perform the operation on the scanned operator and these popped operands and push the result to the stack.

Example: Evaluate the prefix expression **+ 3 * 4 5**

Prefix	Stack	Action
5	5	
4	5 4	
*	20	4 * 5=20 ,push 20
3	20 3	
+	23	3 + 20=23 ,push 23
(		

The result of evaluated expression is 23.

```
void evaluate(char postfix[])
{
	int j=0,z;
	int x,y;
	char ch;
	while(postfix[j]!='\0')
	{
		ch= postfix[j++];
			switch(ch)
			{
				case '+':
			      case '-':
				case '*':
			case '/':
			case '%':
				case '^':
					y=pop()-48;
					x=pop()-48;
					push(result(x,y,ch)+48);
					break;
			default:
				push(ch);
			}
		}
	printf("The evaluated postfix expression is : %d\n", stack[top]-48);
}

int result(int x,int y,char ch)
{
	switch(ch)
```

```
        {
                case '+':
                        return(x+y);
                case '-':
                        return(x-y);
                case '*':
                        return(x*y);
                case '/':
                        return(x/y);
                case '%':
                        return(x%y);
                case '^':
                        return(x^y);



        }
}
```

10. Linear Queue: Basic Operations

A queue is an ordered list to which insertion and deletion takes place at two different ends. The end at which new elements are added is called REAR and the end at which elements are deleted is called FRONT.

Queue can be represented in two ways.

- Array
- Linked List (to be discussed in Linked List chapter)

Array Representation of Queue:

In this representation an array is considered as a QUEUE. The size of array is the size of QUEUE. REAR and FRONT are initialized to -1. REAR and FRONT will take the value from 0 to size-1 which are the array indexes.

Basic Operations of Queue:

Let's consider a character QUEUE of SIZE = 3.

```
char QUEUE [SIZE];
```

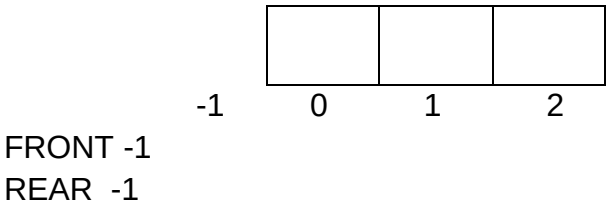
Initially the QUEUE is empty; it means there are no elements in the QUEUE.

```
int REAR = - 1 ;
int FRONT = - 1 ;
```

Insert an element to Queue:

In an insertion operation the REAR value is increased by one and at REAR index of QUEUE, the new element is stored. But if the REAR value reaches at SIZE-1, then it can't increase. If the user wants to insert any element then it's called an overflow condition or the QUEUE is full.

For the first element insertion FRONT and REAR are made 0.



FRONT -1
REAR -1

Initially when FRONT and REAR is -1 the QUEUE is empty.

Insert A

FRONT 0

REAR 0

A		
0	1	2

Insert B

FRONT 0

REAR 1

A	B	
0	1	2

Insert C

FRONT 0

REAR 2 and the QUEUE is full.

A	B	C
0	1	2

As the QUEUE is full now, trying to insert any more elements will cause QUEUE overflow.

QUEUE full: We say the QUEUE is full if REAR reaches to SIZE-1.

```
        if(REAR == size-1)
                printf("\n QUEUE is full");
/*Function for implementing insertion */
void insertion(char ch)
{
        if(rear==size-1)
                printf("Queue is full\n");
        else
        {
                if(rear==-1)
                        front=rear=0;
                else
```

```
                rear=rear+1;
            queue[rear]=ch;
        }
}
```

Delete an element from QUEUE:

In a delete operation the element at FRONT index of QUEUE is returned and the FRONT value is increased by one. But if the FRONT value reaches at SIZE, then the QUEUE becomes empty. If the user wants to delete any element then it's called an underflow condition or the QUEUE is empty.

Example:

A	B	C
0	1	2

FRONT 0
REAR 2 and the QUEUE is full.

X= delete();
X will contain A and the QUEUE is as follows.

	B	C
0	1	2

FRONT 1
REAR 2

Similarly after two delete (), the FRONT value reaches to SIZE and the QUEUE becomes empty.

0	1	2	3

FRONT 3
REAR 2

QUEUE empty: The QUEUE becomes empty in two cases one is the initial case where FRONT== -1 and the second is the case where FRONT == REAR +1**.**

```
If (FRONT == -1 || FRONT == REAR +1 )
        printf("\n QUEUE is empty");
```

FIFO: As the element inserted first is the element to delete first, Linear queue is also called First In First Out (FIFO) data structure.

```
/*Function for implementing deletion */

char deletion()
{
	if((front==rear+1)||(front==-1))
		printf("Queue is empty\n");
	else
		return(queue[front++]);
}

/*Function for implementing traverse */


void traverse()
{
	int i;
	if((front==-1)||(front==rear+1))
		printf("Queue is empty\n");
	else
	{
		for(i=front;i<=rear;i++)
			printf("%c ",queue[i]);
	}
}
```

11. Circular Queue: Basic Operations

Disadvantages of Linear Queue:

Once the REAR value reaches to SIZE-1 or Queue becomes full, we can't insert any element even if empty spaces are there.

	B	C
0	1	2

FRONT 1
REAR 2

Circular Queue:

A Circular queue is like a linear queue in which insertion takes place at REAR end and deletion takes place at FRONT end. The queue is made circular by increasing the FRONT and REAR value in a circular/modular way.

Circular Queue can be represented in two ways.

- Array
- Linked List

Array Representation of Circular Queue:

In this representation an array is considered as a Circular QUEUE. The size of array is the size of Circular QUEUE. REAR and FRONT are initialized to -1. REAR and FRONT will take the value from 0 to SIZE-1 which is the array indexes. After reaching at SIZE-1, REAR and FRONT are set to zero.

Basic Operations of Circular Queue:

Let's consider a character QUEUE of SIZE = 3.

```
char QUEUE [SIZE];
```

Initially the QUEUE is empty; it means there are no elements in the QUEUE.

```
int REAR = - 1 ;
int FRONT = - 1 ;
```

Insert an element to Circular Queue:

In an insertion operation the REAR value is increased by one and at REAR index of QUEUE, the new element is stored. But if the REAR value reaches at SIZE-1, then it can't increase. If the user wants to insert any element then it's called an overflow condition or the QUEUE is full.
For the initial element insertion FRONT and REAR are made 0.

Example:

-1	0	1	2

FRONT -1
REAR -1

Initially when FRONT and REAR is -1 the QUEUE is empty.

Insert A
FRONT 0
REAR 0

A		
0	1	2

Insert B
FRONT 0
REAR = (REAR +1)%SIZE = 1

A	B	
0	1	2

Insert C
FRONT 0
REAR 2 and the QUEUE is full.

A	B	C
0	1	2

As the QUEUE is full now, trying to insert any more elements will cause QUEUE overflow.

QUEUE full: We say the QUEUE is full if REAR reaches to SIZE-11where no element is deleted from FRONT.

```
        if(REAR == size-1 && FRONT==0)
                printf("\n QUEUE is full");

void insertion(char c)
{
        if((front==0 && rear==SIZE-1) || (rear==front-1))
                printf("Queue is empty\n");
        else
        {
                if(front==-1)
                        front=rear=0;
                else
                        rear=(rear+1)%SIZE;
                queue[rear]=c;
        }
}
```

Delete an element from Circular Queue:

In a delete operation the element at FRONT index of QUEUE is returned and the FRONT value is increased by one. But if the FRONT value reaches at SIZE, then the QUEUE becomes empty. If the user wants to delete any element then it's called an underflow condition or the QUEUE is empty.

Example:

A	B	C
0	1	2

FRONT 0
REAR 2 and the QUEUE is full.

X= delete();
X will contain A and the QUEUE is as follows.

	B	C
0	1	2

FRONT = (FRONT +1) %SIZE = 1

REAR 2

When there is one element in the Queue and we will delete it, then we have to make FRONT and REAR -1.It means the queue is made empty.

QUEUE empty:

```
if(FRONT == -1 && REAR == -1)
        printf("\n QUEUE is empty");
```

Insert and delete of an element to Circular Queue after it becomes full:

Insert D

FRONT 1

REAR = (REAR +1) %SIZE = (2+1)%3= 0

After insertion the QUEUE is as follows

D	B	C
0	1	2

X= delete();

X will contain B and the QUEUE is as follows.

FRONT = (FRONT +1) %SIZE = 2

REAR 0

D		C
0	1	2

//CODE FOR DELETION

```
char deletion()
{
        char c;
        if(front==-1)
                printf("Queue is empty\n");
        else
        {
                c=queue[front];
```

```
		if(front==rear)
			front=rear=-1;
		else
			front=(front+1)%SIZE;
		return c;
	}
}

//CODE FOR TRAVERSAL
void traverse()
{
	int i;
	if(front==-1)
		printf("Queue is empty");
	else
	{
		if(rear>=front)
		{
			for(i=front;i<=rear;i++)
				printf("%c\t",queue[i]);
			printf("\n");
		}
		else
		{
			for(i=front;i<SIZE;i++)
				printf("%c\t",queue[i]);
			for(i=0;i<=rear;i++)
				printf("%c\t",queue[i]);
			printf("\n");

		}
	}
}
```

12. Double Ended Queue , Priority Queue

Double Ended Queue (DeQueue):

It is a queue in which insertion and deletion are made at both LEFT and RIGHT end.

-1	0	1	2

LEFT -1
RIGHT -1
It means there is no element from the LEFT end and RIGHT end.

It has two variants

I. Input restricted DeQueue.
II. Output restricted DeQueue.

Input Restricted DeQueue:
Element can be inserted at one end (either left end or right end) but deletion can be done from both the end.
Output Restricted DeQueue:
Element can be inserted at both end but deletion can be done from one end (either left end or right end).
It is upto the user which end to close and which end to open
Insert an element to Double Ended Queue (DeQueue) from LEFT end:
.
Example:
Initially

-1	0	1	2

LEFT -1
RIGHT -1

Initially when LEFT and RIGHT is -1 the DEQUEUE is empty.

Insert A
LEFT 0
RIGHT 0

A		
0	1	2

Insert B

If (LEFT == 0) then LEFT= SIZE – 1=2

RIGHT 0

A		B
0	1	2

Insert C

LEFT= LEFT – 1=1

RIGHT 0 and the QUEUE is full as LEFT ==RIGHT+1

A	B	C
0	1	2

QUEUE full: We say the QUEUE is full if REAR reaches to SIZE-1.

```
if(LEFT ==RIGHT+1)
        printf("\n QUEUE is full");
```

Insert an element to Double Ended Queue (DeQueue) from RIGHT end:

It is similar to the operation of circular Queue insertion at rear end.

-1	0	1	2

LEFT -1

RIGHT -1

Initially when LEFT and RIGHT is -1 the QUEUE is empty.

Insert A

LEFT 0

RIGHT 0

A		
0	1	2

Insert B

LEFT 0

RIGHT = (RIGHT +1)%SIZE = 1

A	B	
0	1	2

Insert C

LEFT 0

RIGHT 2 and the QUEUE is full.

A	B	C
0	1	2

QUEUE full: We say the QUEUE is full if RIGHT reaches to SIZE-1where no element is deleted from LEFT.

```
if(RIGHT == size-1 && LEFT ==0)
        printf("\n QUEUE is full");
```

Delete an element from Double Ended Queue (DeQueue) from LEFT end:
It is similar to the operation of circular Queue deletion from FRONT.

Example:

A	B	C
0	1	2

LEFT 0

RIGHT 2 and the QUEUE is full.

X= delete();

X will contain A and the QUEUE is as follows.

	B	C
0	1	2

LEFT = (LEFT +1) %SIZE = 1

RIGHT 2

When there is one element in the Queue and we will delete it, then we have to make LEFT and RIGHT -1.It means the queue is made empty.

QUEUE empty:.

```
if(RIGHT == -1 && LEFT == -1)
        printf("\n QUEUE is empty");
```

Delete an element from Double Ended Queue (DeQueue) from RIGHT end:

A	B	C
0	1	2

LEFT 0

RIGHT 2 and the QUEUE is full.

X= delete();

X will contain A and the QUEUE is as follows.

A	B	
0	1	2

LEFT 0

RIGHT = RIGHT – 1 = 2 - 1 =1

A		
0	1	2

LEFT 0

RIGHT = RIGHT – 1 = 1 - 1 =0

When there is one element in the Queue and we will delete it, then we have to make LEFT and RIGHT -1.It means the queue is made empty. What if we will **insert D** at LEFT end?

LEFT 2
RIGHT 0

A		D
0	1	2

Now if we delete from RIGHT end.

LEFT 2
If (RIGHT == 0) then RIGHT = SIZE – 1=2

		D
0	1	2

Priority Queue:

It is a collection element in which each element has been assigned a priority and the highest priority element is deleted or operated first. It has many applications due to the priority properties.
If elements have same priority then they will be served as per FIFO basis.

Application Of Queue:

I. Task Scheduling of Operating System
II. BFS algorithm of Graph
III. Checking of palindrome number or string
IV. Etc.

13. Linked List: Introduction, Types, Single Linked List Traversal

Linked List:

It is a collection of nodes or data elements where the nodes are linked to one another .A node is a structure which contains the data fields and addresses fields.

Advantages:

I. It is a dynamic data structure which can grow or shrink during program execution.
II. The insertion and deletion operation are easier and efficient.

Disadvantages:

It occupies more memory for a single data set.

Types of Linked List:

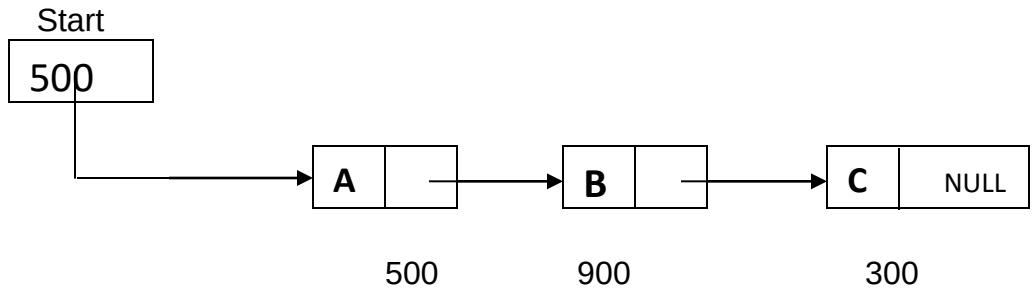
I. Single Linked List
II. Double Linked List
III. Circular Single/ Double Linked List

Operation of a Linked List:

I. Create
II. Traverse
III. Search
IV. Sort
V. Delete from (begin, end, middle)
VI. Insert at (begin, end, middle)
VII. Reverse

Single Linked List:

It contains a list of nodes where each node has one data field and one address field.



Start Node:

It is a pointer which contains the address of (points to) the starting node of the list. In the above figure ‘A’ is the starting node whose address is 500 and it is stored by the start pointer.

List’s Node:

Each node of the list has an address field which points to the next node of list and the data filed contains data.

Creation of Node structure:

```
typedef struct node
{
        char data;
        struct node *next;
} node;
```

Here data, node and next are identifiers. An alias ‘node’ is made for data type ‘struct node’.

Empty List:

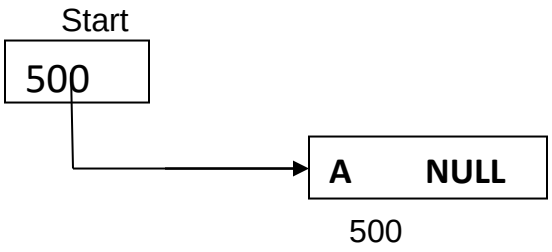
If there is no node in the list then the list is empty and start pointer should be assigned to NULL.

Last node:

If the address part of a node contains NULL then it is the last node.

Single Node List:

If the address part of the starting node contains NULL then it is a single node list.



Creation of empty Linked List:

node * start = NULL;

Start

NULL

Creation of ‘n’ node Single Linked List:

```
//CODE FOR CREATION FUNCTION
void create()
{
	int n,i;
	struct node *newnode,*temp;
	char ch;
	printf("Enter the no. of nodes : ");
	scanf("%d",&n);
	for(i=0;i<n;i++)
	{
		//Creation of node using malloc function
		newnode=(struct node *)malloc(sizeof(struct node));
		//Storing of data in node
		printf("Enter the data\n");
		scanf(" %c",&newnode->data);
		newnode->next=NULL;
		//linking newnode to list
		if(start==NULL)
			start=newnode; //assigning newnode address to start pointer
		else
		{
			temp=start; //assigning first node address to temp
			while(temp->next!=NULL)
				temp=temp->next;
			//Storing newnode address in last node address part
			temp->next=newnode;
		}
	}
}
```

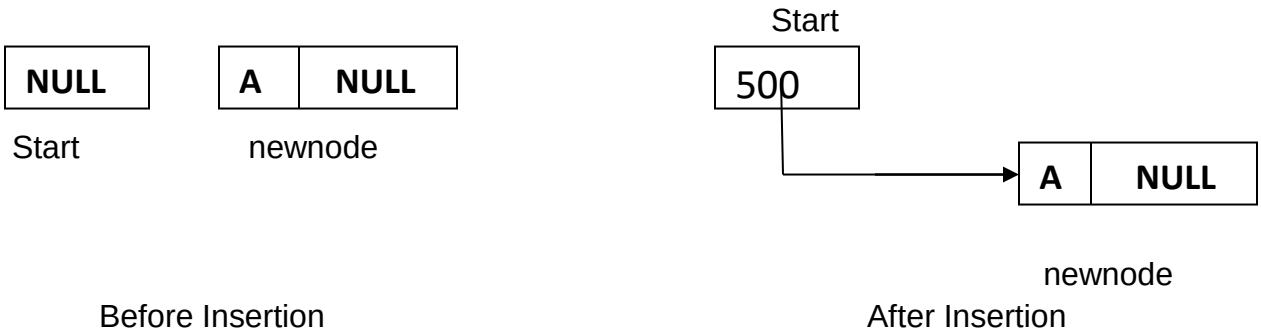
Traversal of Single Linked List:

```
//CODE FOR TRAVERSAL FUNCTION
void traverse()
{
	struct node *temp;
	if(start==NULL)
		printf("The list is empty.\n");
	else
	{
		temp=start;
		printf("The link list is displayed below : \n");
		while(temp!=NULL)
		{
			printf("\t %c",temp->data);
			temp=temp->next;
		}
		printf("\n");
	}
}
```

14. Single Linked Lists: Insertion

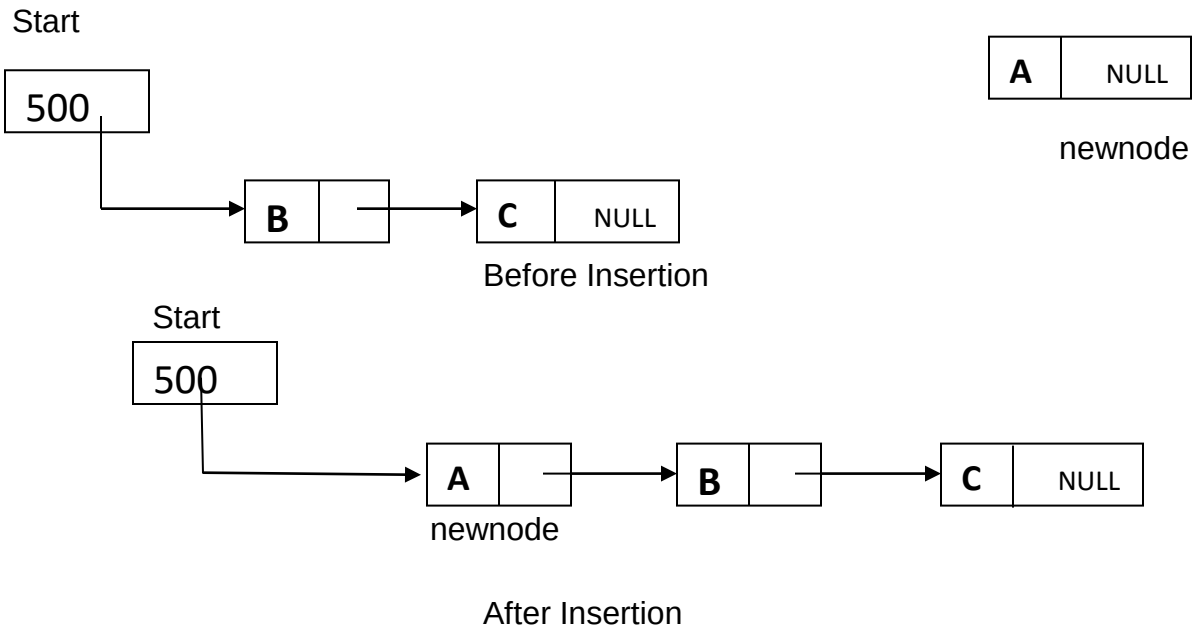
Insertion of a data can be done at any location of a linked list like beginning, middle and end. User will give a specific position of the data to insert and the algorithm will insert it.

At the time of insertion there might be different possibilities .Like one can try to insert at middle/begin/end in an empty list, and then the data will become a single node list which signifies all three possibilities are met. In such case the newnode becomes the starting node.




```
If(start==NULL)
        start=newnode;
```

While inserting a node at beginning, the newnode will become the start node and the existing start node will become the second node.



Here we are just assigning the existing start node to the address part of newnode and sart pointer is assigned to newnode.

newnode->next=start;
start=newnode;

If a node has to insert a the end of a list, then the newnode will insert next to the last node and the newnode will become last node. As the node is to add next to last node, we have to find the last node address and in the address part of it the newnode address will be assigned.

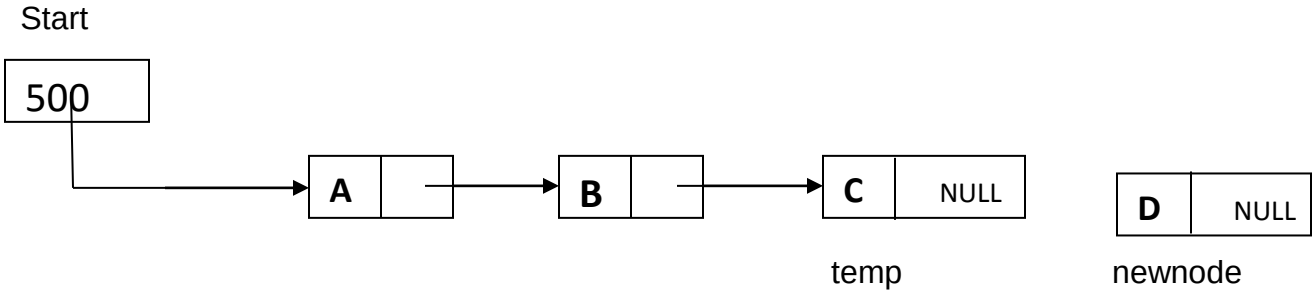
temp=start;

Here temp contains starting nodes address and temp will visit one bi one node up to the end of the list.

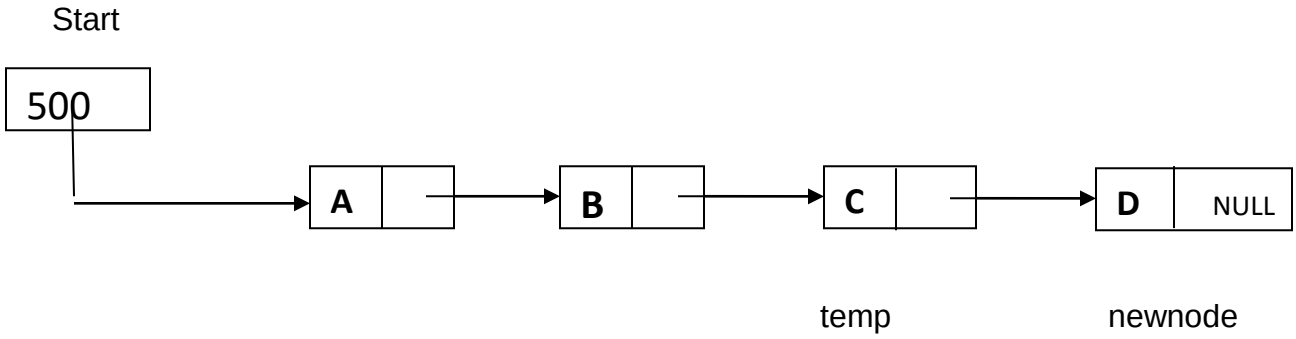
while(temp->next !=NULL)
temp=temp->next;

Now temp contains the last node address.

temp->next =newnode;



Before Insertion



Before Insertion

While inserting a node at middle of list user has to specify the position value n after which the newnode has to be inserted. So we have to find the address of nth node for assigning the newnde's address in its address part.

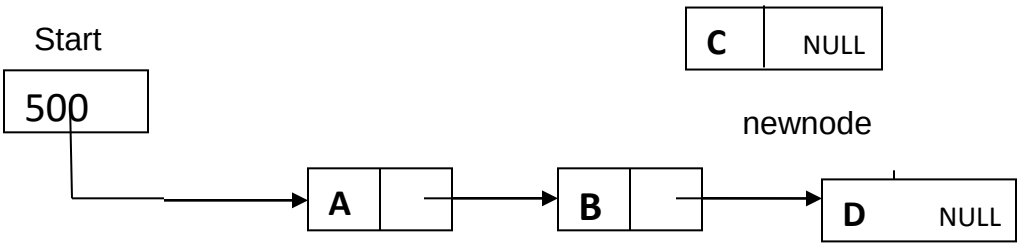
```
temp=start;
i=1;
```

Here temp contains starting nodes address and temp will visit one by one node up to n^{th} node of the list .

```
while (temp->next !=NULL && i <= n)
        {
        temp=temp->next;
        i++;
        }
```

Now temp contains the nth node address

```
temp->next =newnode;
```



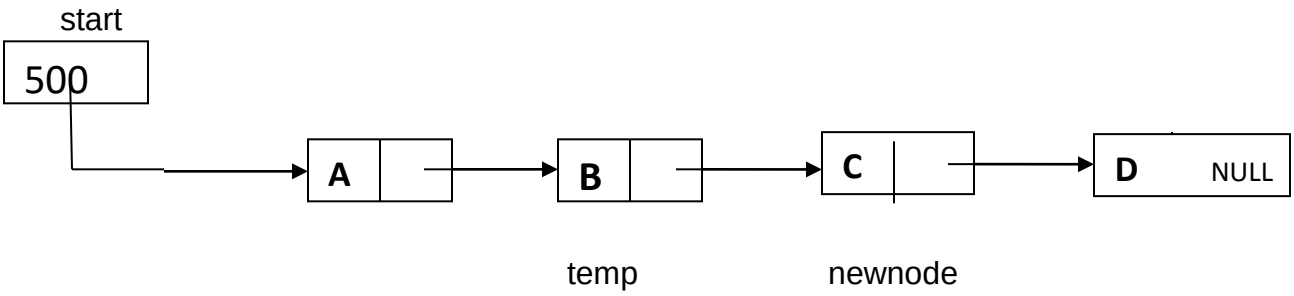
Let newnode will be inserted after position is 2 i.e. n=2.

```
while (temp !=NULL && i < 2)
        {
        temp=temp->next;
        i++;
        }
```

Here loop will run once. So temp temp will contain 2nd node(n=2) address.

```
newnode->next=temp->next;
temp->next =newnode;
```

Here temp->next which is the address of the next node of nth node is to be the next node of the newnode is assigned to the newnode address part. Then the newnode address is assigned to the temp nodes address part.



While finding the nth position if the temp value becomes null it has searched all the position and the position is an invalid one.

```
/*Function for insertion of Single Linked List*/
void insert(int x,int pos)
{
        struct node *newnode,*temp;
        int i;
        newnode=(struct node*)malloc(sizeof(struct node));
        newnode->data=x;
        newnode->next=NULL;
        if(start==NULL)
                start=newnode;
        else if(pos==1)
        {
                newnode->next=start;
                start=newnode;
        }
        else
        {
                temp=start;
                //any other function except position 1
                for(i=1;i<pos && temp->next!=NULL;i++)
                        temp=temp->next;
                if(temp==NULL)
                {
                        printf("Invalid Position\n");
                        return;
                }
                else
                {
                        newnode->next=temp->next;
```

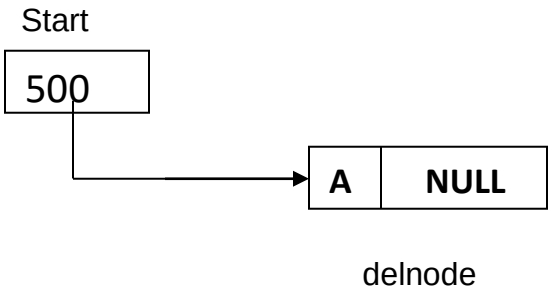
```
                    temp->next=newnode;
            }
        }
}
```

15. Single Linked Lists: Deletion

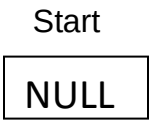
Deleting a node from a specific position has different possibilities also. If the list is empty then there is no node to delete where start pointer contains **NULL**.

If there is a single node in the list then after deleting that node the list becomes empty. So we have to make start NULL.

```
If (start->next==NULL)
        start = NULL;
```

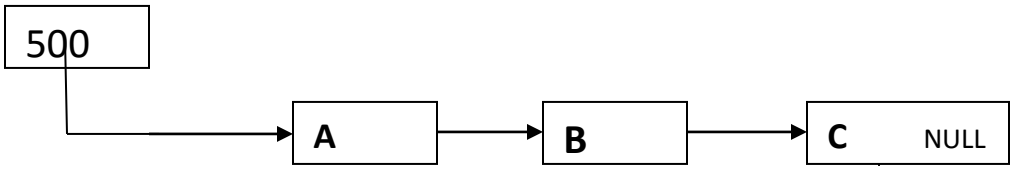


Before Deletion



After Deletion

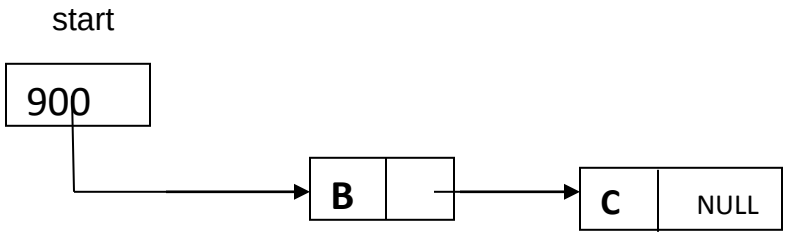
If the start node has to delete then after deletion start pointer will point to the 2nd node whose address is at the address part of start



Delnode
Before Deletion

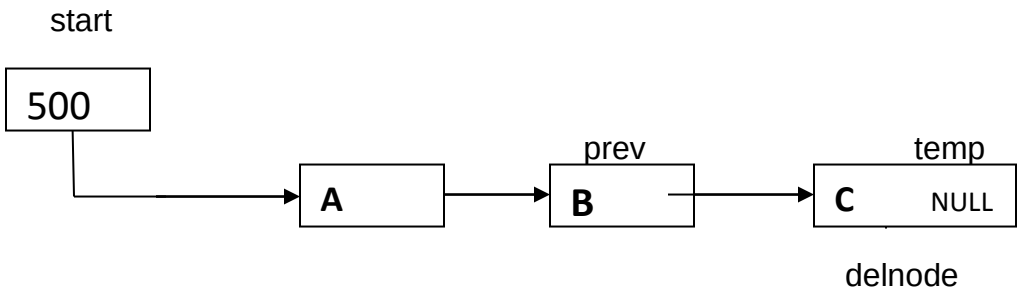
Here we simple do

```
If (pos == 1)
        start = start->next;
```



After Deletion

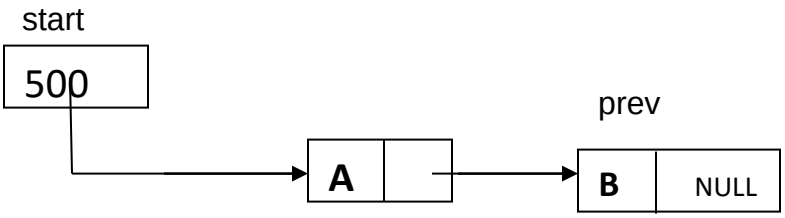
For deleting the last node from a list we need to find the previous node of the last node and in the address part of the previous node we have to store NULL as it will become the last node.



Before Deletion

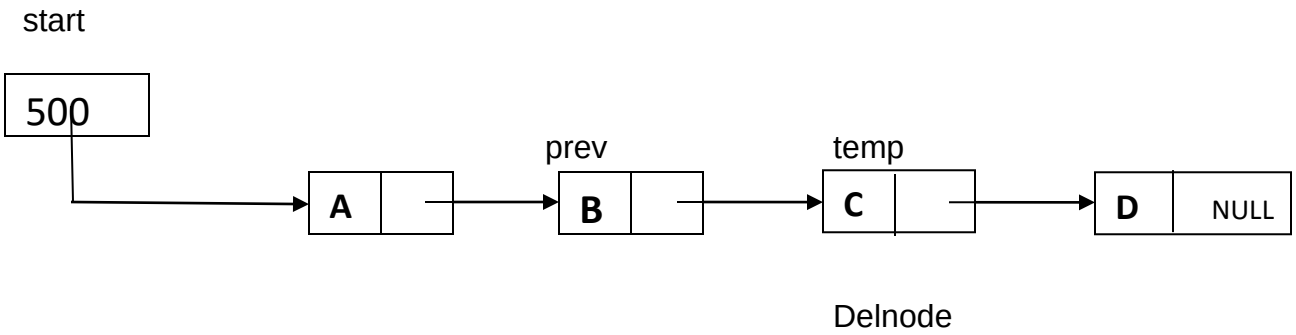
So we need another pointer which contains the previous nodes address of temporary pointer and temporary will takes up to the last node.

```
prev=temp=start;
while(temp->next != null)
{
        temp=temp->next;
        prev=temp;
}
prev->next=temp->next;
```



After Deletion

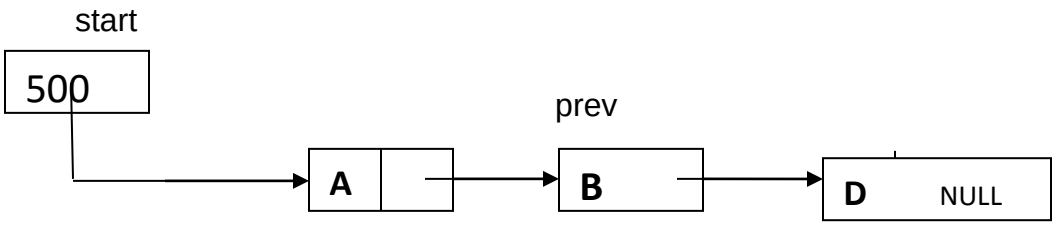
When user specify a specific position ‘n’ to delete we need to take the temporary pointer to n^{th} position and the previous pointer to its previous node n-1.



Before deletion

Let the deleting node position is 3.

```
prev = temp=start;
i=1;
while(temp->next != null && i < 3)
{
        temp=temp->next;
        prev=temp;
}
prev->next=temp->next
```



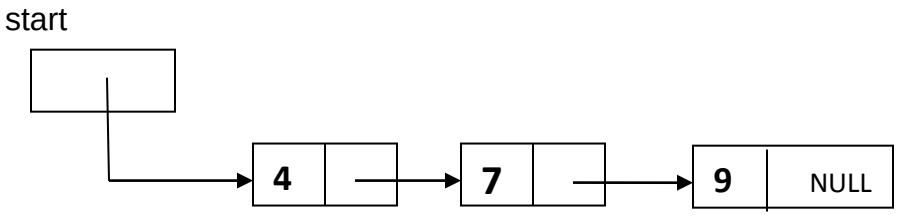
After Deletion

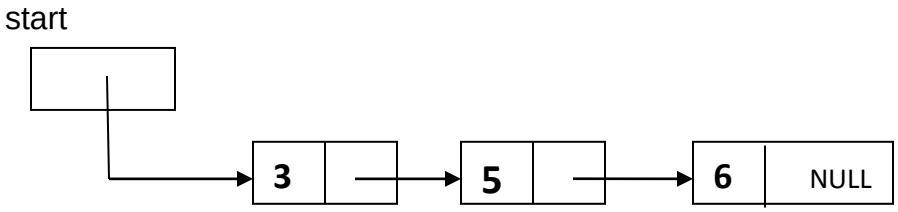
If the value of ‘n’ is greater than i then it’s an invalid position.After deleting a node we need to free its dynamically allocated memory using free pointer.

```
/*Function for deletion of a node in Single Linked List*/
void delete(int pos)
{
	struct node *prev,*temp,*delnode;
	int i;
	if(start==NULL)
		printf("The list is empty.\n");
	else if(pos==1 && start->next==NULL)
	{
		free(start);
		start=NULL;
	}
	else
	{
		prev=temp=start;
		for(i=1;i<pos && temp->next!=NULL;i++)
		{
			prev=temp;
			temp=temp->next;
		}
		if(temp==start)
		{
			delnode=start;
			start=start->next;
		}
		else
		{

			delnode=temp;
			prev->next=temp->next;
		}

		free(delnode);
	}
}
```

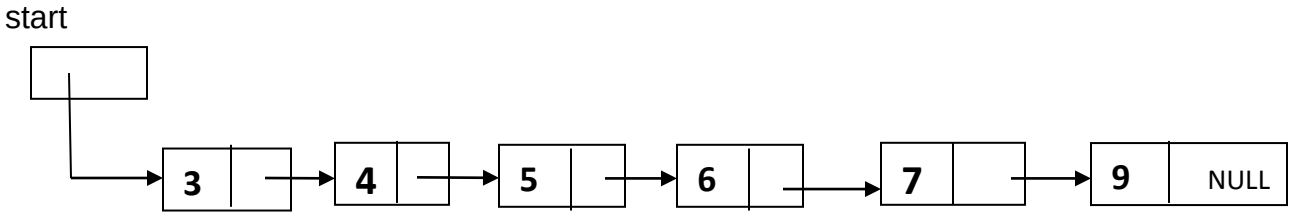
16. Single Linked Lists: Merging, Reversing, Sorting

Merging:





Before Merging

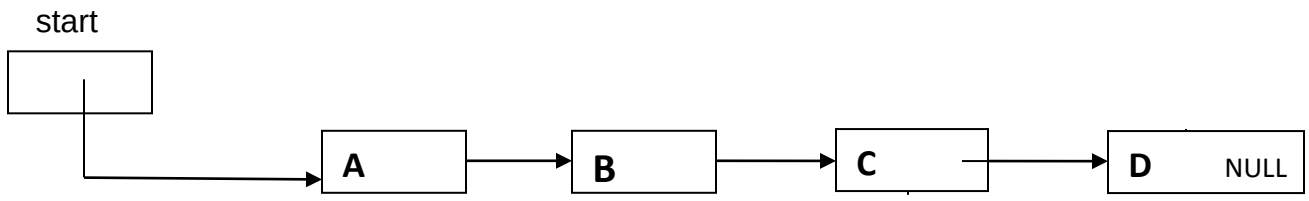


After Merging

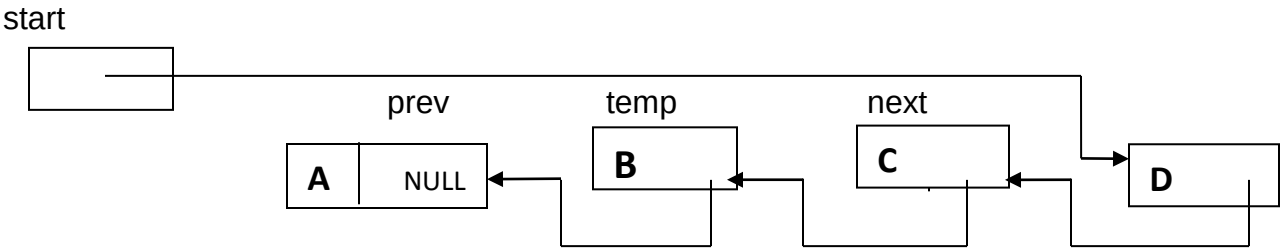
```
/*function for merging two sorted list*/
node merge(node start1,node start2)
{
if(start1 == null)
        return start2;
if(start2 == null)
        return start1;
if(start1.data < start2.data)
{
        start1.next=merge(start1.next,start2)
        return start1;
}
else
```

```
{
        start2.next=merge(start2.next,start1)
        return start2;
}
}
```

Reversing:



Before reverse



After reverse

```
/*function to reverse a linked list*/
void reverse()
{
        struct node *prev,*temp,*next;
        if(start->next!=NULL)
        {
                prev=start;
                temp=start->next;
                while(temp->next!=NULL)
                {
                        next=temp->next;
                        temp->next=prev;
                        prev=temp;
                        temp=next;
                }
```

```
                temp->next=prev;
                start->next=NULL;
                start=temp;

        }
}
```

Sorting:

Sorting of a linked list can be done using any type of sorting. Let's do the srting using bubble sort.

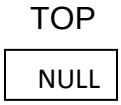
```
void bubble(node *start)
{
        int temp;
        node *temp1=start;
        node *temp2;
        while(temp1->next!=null)
        {
                temp2=start;
                while(temp2->next ->next!=null)

                {
                        if(temp2->data < temp2->next->data)
                        {
                                temp = temp2->data;
                                temp2->data= temp2->next->data;
                                temp2->next->data= temp;
                        }
                temp2=temp2->next;
                }
temp1=temp1->next;
}
}
```

17. Linked Representation of Stack and Queue

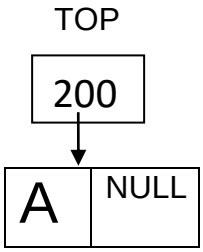
Linked Stack:

In a linked representation of stack, insertion and deletion of new node are at same end which is considered as top. Let consider TOP as start end. So instead of taking start pointer we will take TOP which pints to the starting node of the stack and insertion and deletion are to be done at beginning of the list.

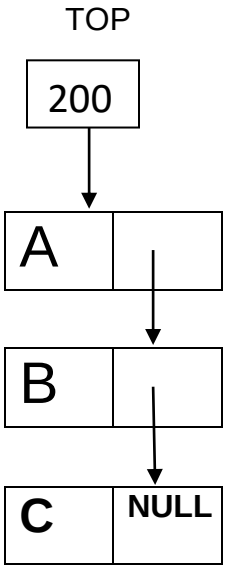
Initially stack list is empty.



PUSH A



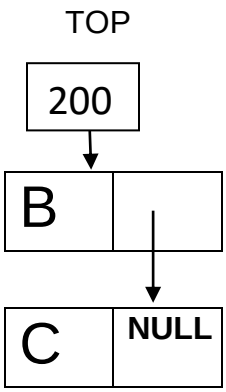
PUSH B, PUSH C




```
/*PUSH function*/
void push(char x)
{
```

```
node *newnode;
newnode=(node*)malloc(sizeof(node));
newnode->data=x;
newnode->next=null;
newnode->next=TOP;
TOP=newnode;
}
```

Similarly for POP operation means deleting the starting node of the list. After one POP operation the stack list as follows.



When TOP becomes NULL the stack is empty and there is no stack full situation as stack array.

```
/*PUSH function*/
char pop()
{
node *newnode,*delnode;
char x;
if(TOP==NULL)
        printf("list is empty");
else
{
x=TOP->data;
del_node=TOP;
TOP=TOP->next;
free(delnode);
return x;
}
}
```

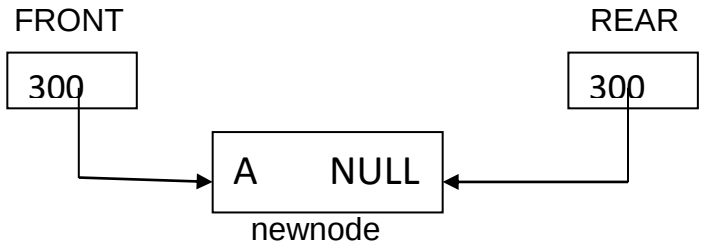
Linked Queue:

All types of queue can be represented in Linked List. Let's see a linear queue list where insertion and deletion are done at two different ends. Let insert a newnode at REAR end (end of list) and delete a node from FRONTend (beginning of list).

Insertion Operation:

Initially when FRONT and REAR is NULL and the QUEUE is empty.

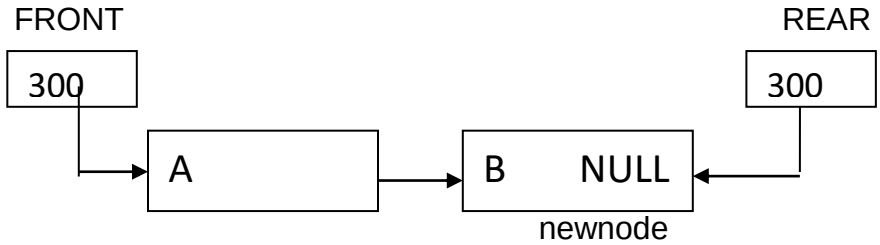
Insert A



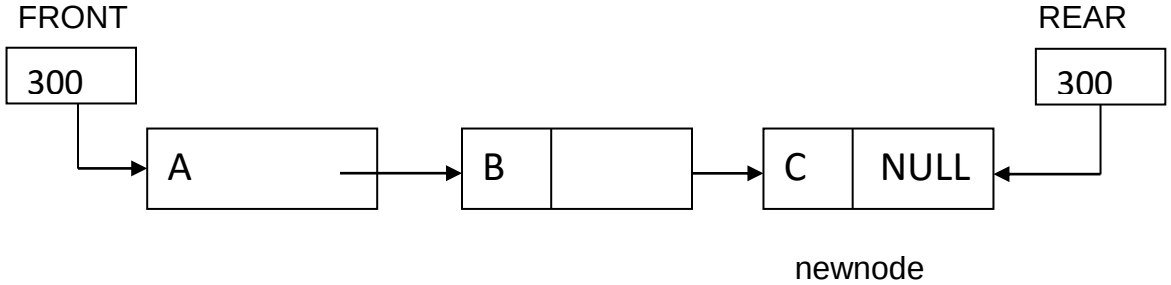
As it is a single node list FRONT and REAR points to the newnode.

FRONT=REAR=newnode;

For inserting B, the insertion is done at REAR end.



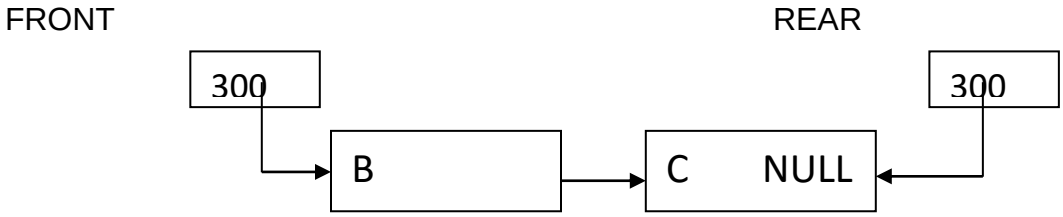
Insert C



```
/*Insert function*/
void insert(char x)
{
node *newnode;
newnode=(node*)malloc(sizeof(node));
newnode->data=x;
newnode->next=null;
if(FRONT==NULL)
        FRONT=REAR=newnode;
else
{
REAR->next=newnode;
REAR=newnode;
}
}
```

Deletion Operation:

Deletion of a node is done form the FRONT or beginning of list. When the FRONT value becomes NULL, the list becomes empty. After performing a delete operation the queue list is as fellows.



When there is a single node in the list and a delete operation is to be performed then FRONT and REAR are made NULL.

```
 /*Delete function*/
char delete()
{
node *delnode;
char x;
if(FRONT ==NULL)
        printf("queue is empty");
else if(FRONT== REAR)
        FRONT=REAR=NULL;
```

```
else
{
x=FRONT->data;
del_node=FRONT;
FRONT=FRNT->next;
free(del_node);
return x;
}
}
```

18. Doubly Linked Lists: Traversal, Insertion

Disadvantage of Single inked List:

I. We can traverse in both ways.
II. For insert and delete operation ,no need to keep track of the previous node.

It is a collection of nodes or data elements where the nodes are linked to their previous and next node .The node structure contains a data fields and two address fields.

Previous node address	Data	Next node address

```
typedef struct node
{
        struct node *prev;
        char data;
        struct node *next;
} node;
```

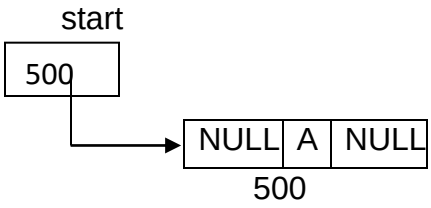
Here data, node, prev and next are identifiers. An alias ‘node’ is made for data type ‘struct node’.

Creation of empty Linked List:

```
node * start = NULL;
```

Single Node List:

If the address part of the starting node contains NULL then it is a single node list.



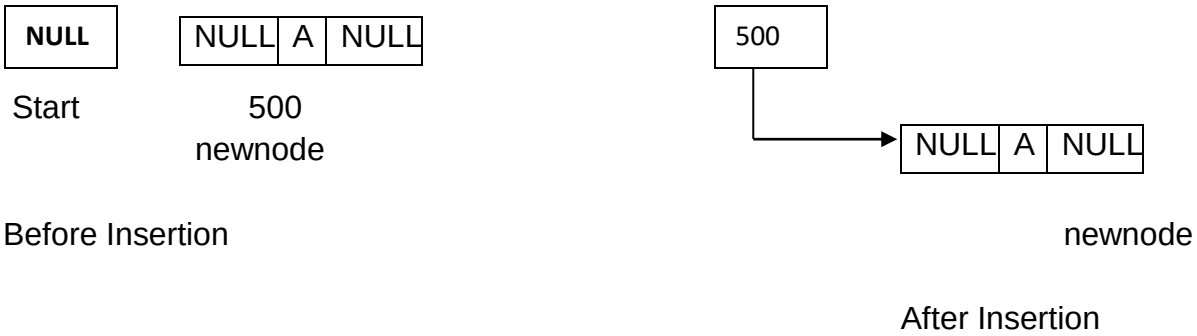
Creation of ‘n’ node Double Linked List:

In creation of the list we append newnode to the list .

Traversal of Single Double List:

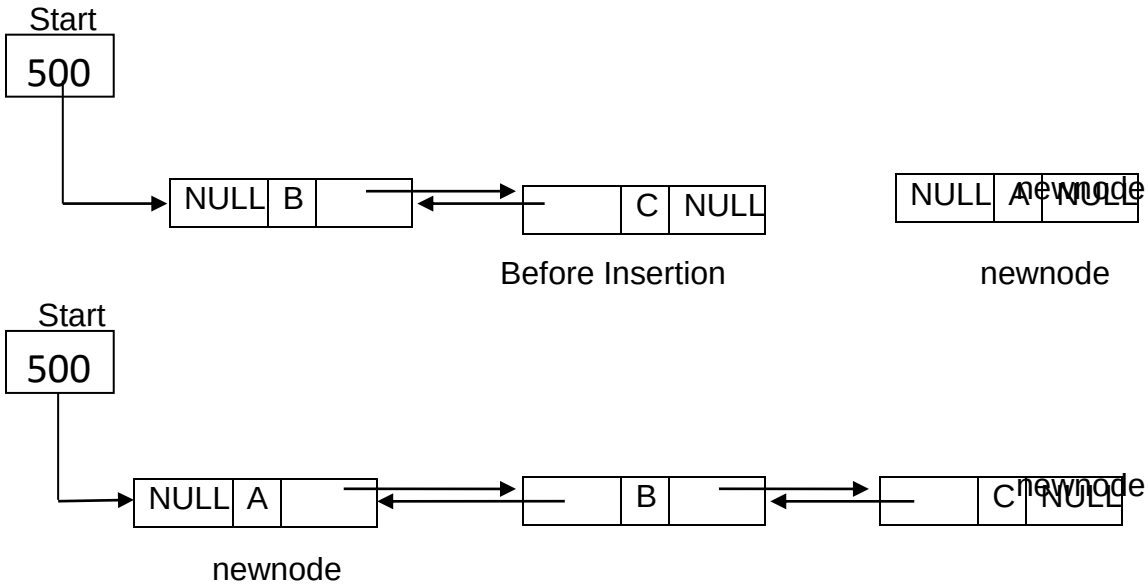
Insertion of a node into Double Linked List:

Like Single inked list there are different possibilities of insertion of a node beginning, middle and end . Insertion can be done in an empty list also.




```
If(start==NULL)
        start=newnode;
```

While inserting a node at beginning, the newnode will become the start node and the existing start node will become the second node.



After Insertion

Here we have to first assign the existing start node's to the address part of newnode and existing start node's previous part should point to the newnode. We are also assigning sart pointer to newnode.

```
newnode->next=start;
start->prev=newnode;
start=newnode;
```

If a node has to insert a the end of a list, then the newnode will insert next to the last node and the newnode will become last node. As the node is to add next to last node, we have to find the last node address and in the address part of it the newnode address will be assigned.

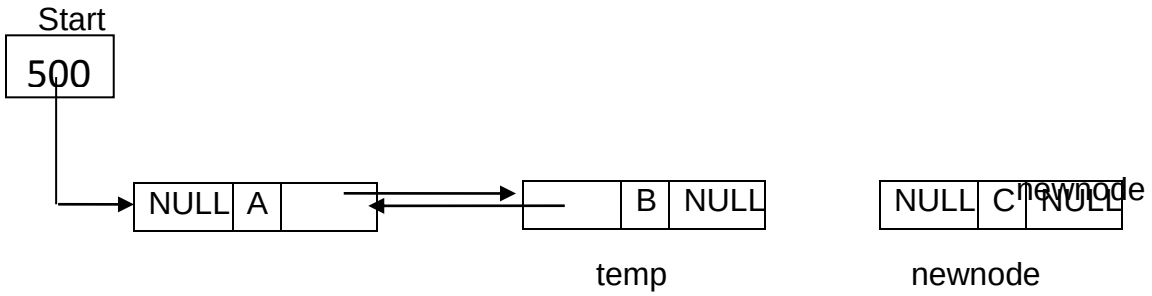
```
temp=start;
```

Here temp contains starting nodes address and temp will visit one bi one node up to the end of the list .

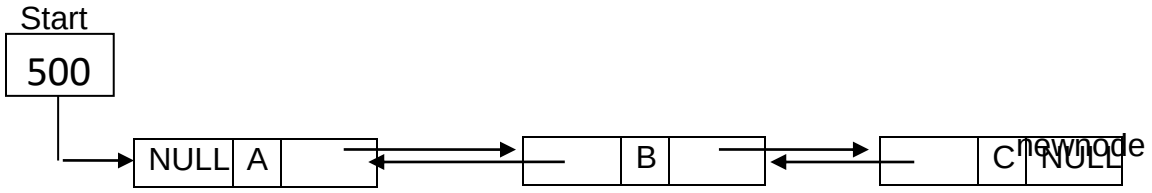
```
while(temp->next !=NULL)
        temp=temp->next;
```

Now temp pointer contains the last node address. So newnode previous is assigned to temp and temp's address part points to newnode.

```
temp->next =newnode;
newnode->prev=temp;
```



Before Insertion



newnode

After Insertion

While inserting a node at middle of list user has to specify the position value n after which the newnode has to be inserted. So we have to find the address of nth node for assigning the newnde's address in its address part.

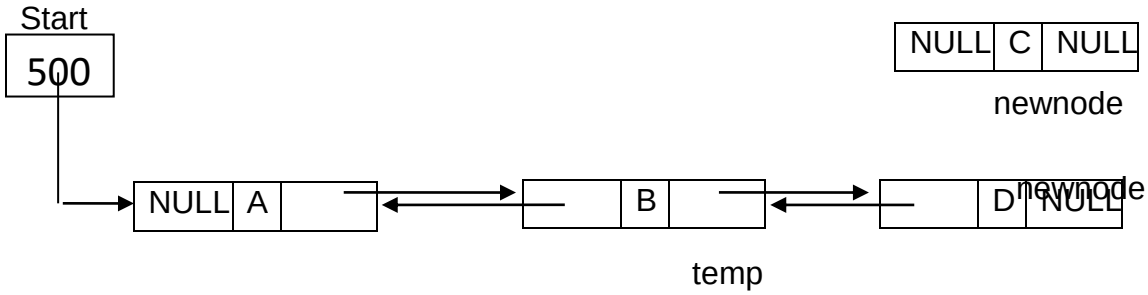
```
temp=start;
i=1;
```

Here temporary pointer contains starting nodes address and temp will visit one by one node up to n^{th} node of the list.

```
while (temp->next !=NULL && i <= n)
        {
        temp=temp->next;
        i++;
        }
```

/*Now temp contains the nth node address*/

```
temp->next =newnode;
```



Before Insertion

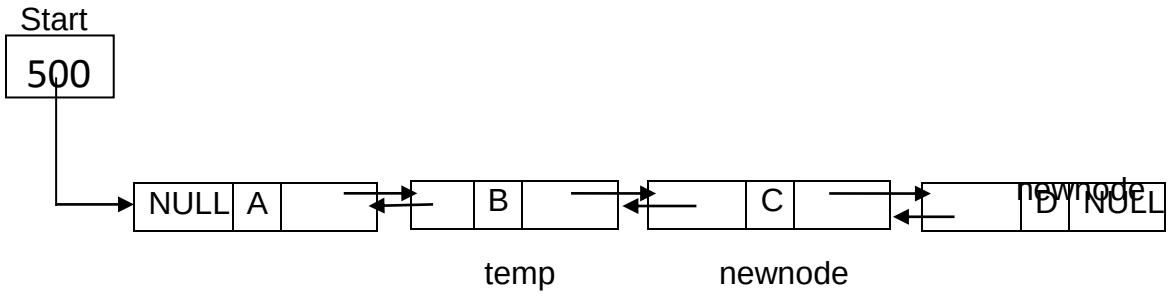
Let newnode will be inserted after position is 2 i.e. n=2.

```
while (temp !=NULL && i < 2)
        {
        temp=temp->next;
        i++;
        }
```

Here loop will run once and temp temp will contain 2nd node(n=2) address.

```
newnode->next=temp->next;
newnode->prev=temp;
temp->next->prev= newnode
```

temp->next =newnode;



After Insertion

While finding the nth position if the temp value becomes null it has searched all the position and the position is an invalid one.

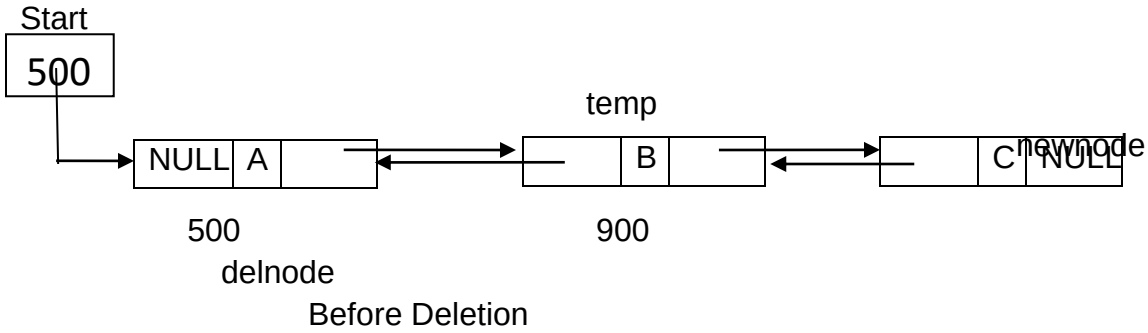
```
/*Function for insertion of Double Linked List*/
void insert(int pos)
{
        node *newnode,*temp;
        int i;
        newnode=(node*)malloc(sizeof(node));
        printf("Enter data\n");
        scanf(" %c",&newnode->data);
        newnode->next=newnode->prev=NULL;

        if(start==NULL)
                start=newnode;
        else if(pos==1)
        {
                start->prev=newnode;
                newnode->next=start;
                start=newnode;
        }
        else
        {
                temp=start;
                for(i=0;i<(pos-1) && temp->next!=NULL;i++)
                        temp=temp->next;
                if(temp->next==NULL)
                {
```

```
                temp->next=newnode;
                newnode->prev=temp;
            }
            else
            {
                newnode->prev=temp;
                newnode->next=temp->next;
                temp->next->prev=newnode;
                temp->next=newnode;
            }
        }
}
```

19. Double Linked list: Deletion

User will specify the position of node which the user will delete.If the list is empty then there is no node to delete where start pointer contains **NULL**.

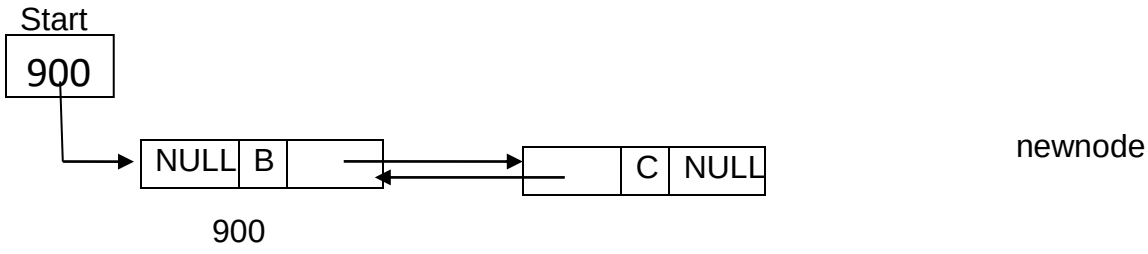
If the start node has to delete then after deletion start pointer will point to the 2nd node whose address is at the next address part of starting node.



Before Deletion

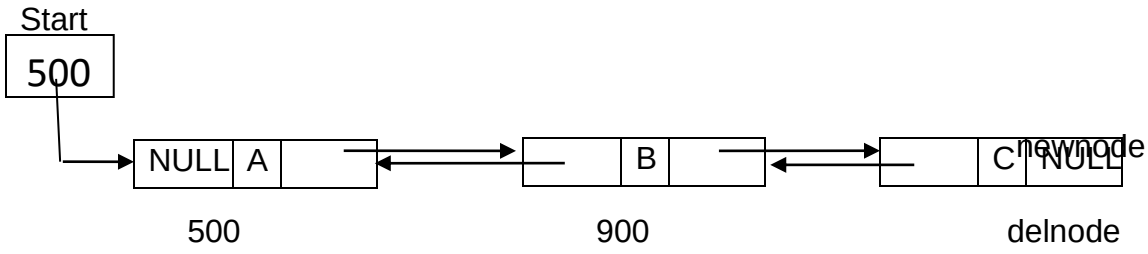
Here we simple assign start pointer to its next and the new starting node's previous address points to NULL.

```
if(pos == 1)
        {
        start=start->next;
        start->prev=NULL;
        }
```



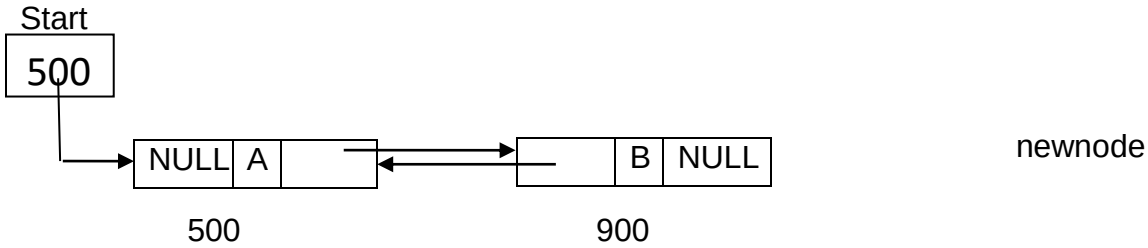
After Deletion

For deleting the last node from a list we need to find the previous node of the last node whose is in the previous address part of the last node where we have to store NULL as it will become the last node. So we take temporary pointer to last node.



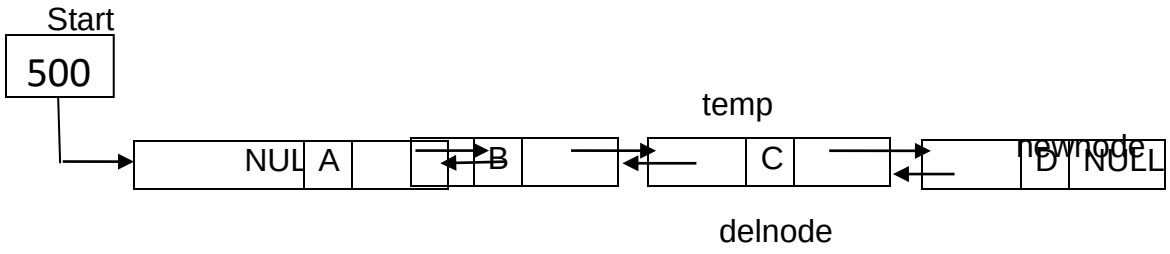
Before Deletion

```
temp=start;
while(temp->next != null)
        temp=temp->next;
temp->prev->next=temp->next;
```



After Deletion

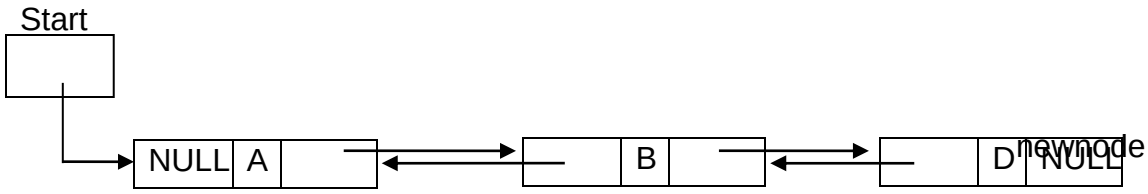
When user specifies a specific position ‘n’ to delete we need to take the temporary pointer to n^{th} position. Let the deleting node position is 3.



Before deletion

```
temp=start;
i=1;
```

```
        while(temp->next != null && i < 3)
                temp=temp->next;
        temp->prev->next=temp->next;
        temp->next->prev=temp->prev;
```



After Deletion

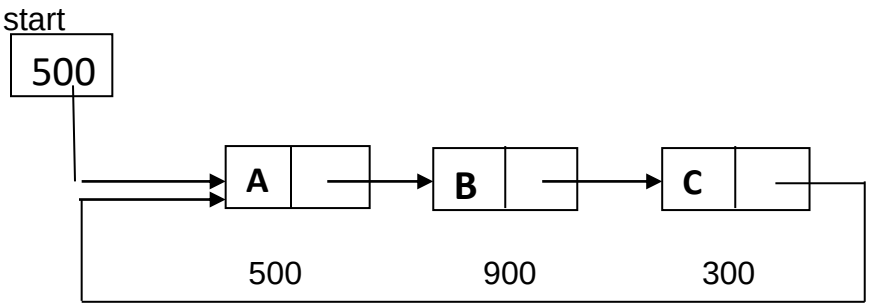
If the valueof ‘n’ is greater than i then it’s an invalid position. After deleting a node we need to free its dynamically allocated memory using free pointer.

```
/*Function for deletion of a node in double Linked List*/
void delete(int pos)
{
        node *delnode,*temp;
        int i;
        if(start==NULL)
                printf("The list is empty\n");
        else if(pos==1 && start->next==NULL)
        {
                free(start);
                start=NULL;
        }
        else if(pos==1)
        {
                start=start->next;
                start->prev=NULL;
        }
        else
        {
                temp=start;
                for(i=1;i<pos && temp->next!=NULL;i++)
                        temp=temp->next;
                if(pos-2>=i && temp->next==NULL)
                        printf("Invalid Position\n");
                else if(temp->next==NULL)
                {
```

```
                delnode=temp;
                temp->prev->next=NULL;
                free(delnode);
        }
        else
        {
                delnode=temp;
                temp->prev->next=temp->next;
                temp->next->prev=temp->prev;
                free(delnode);
        }
    }
}
```

20. Circular Linked List

In a circular inked list the last node's address part contains the starting node's address. Both single and double inked list can be made circular.

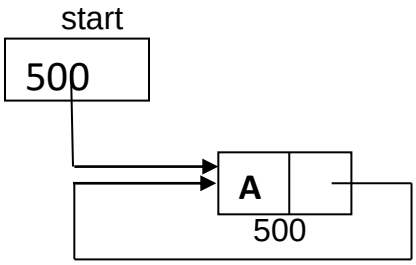


Single circular linked list

Traversal of Single circular linked list:

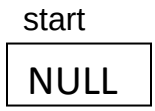
```
void traversal()
{
node * temp;
temp=start;
do
{
printf("%c", temp->data);
temp=temp->next;
} while(temp->next != start);
}
```

Single Node List:
If the address part of the starting node contains NULL then it is a single node list.

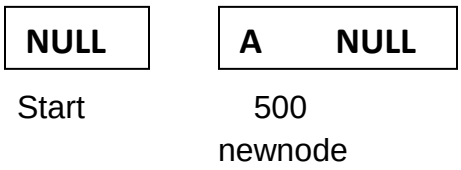


Creation of empty Linked List:

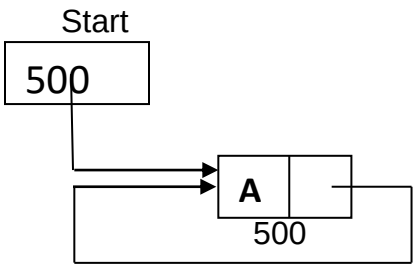
node * start = NULL;



Insertion of a data can be done at any location of a linked list like beginning, middle and end. User will give a specific position of the data to insert and the algorithm will insert it.
At the time of insertion there might be different possibilities .Like one can try to insert at middle/begin/end in an empty list, and then the data will become a single node list which signifies all three possibilities are met. In such case the newnode becomes the starting node.



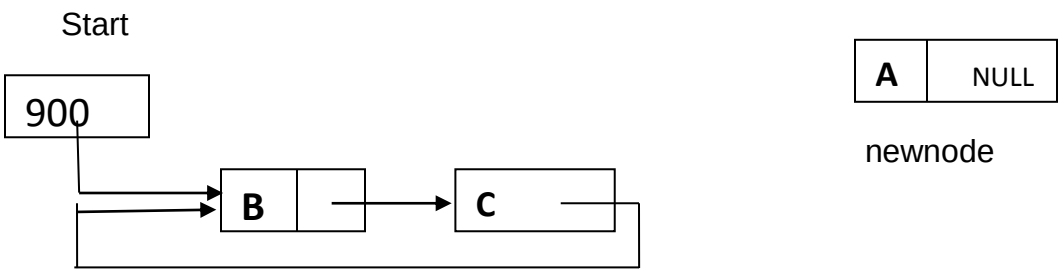
Before Insertion



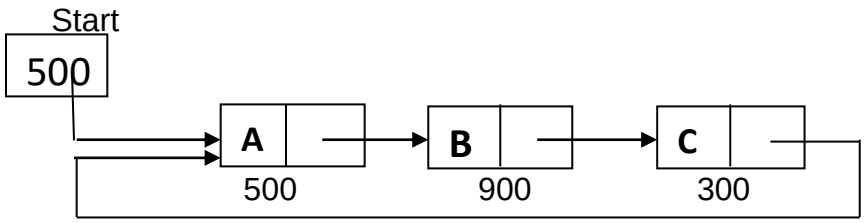
After Insertion

```
If(start==NULL)
        start=newnode;
        newnode->next=newnode;
```

While inserting a node at beginning, the newnode will become the start node and the existing start node will become the second node.



Before Insertion



After Insertion

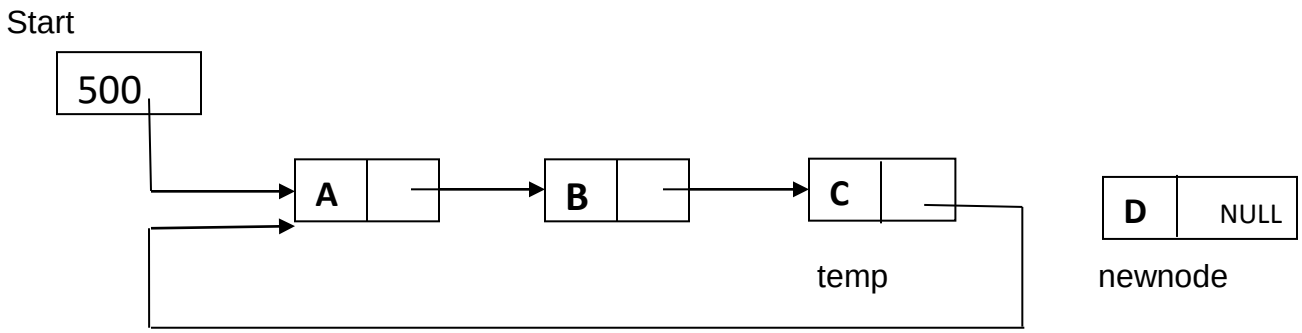
Here we are just assigning the existing start node's address which is at start pointer to the address part of newnode. The sart pointer and the last nodes address part are assigned to newnode.

```
temp=start;
do
{
temp=temp->next;
} while(temp->next != start);
newnode->next=start;
start=temp->next=newnode;
```

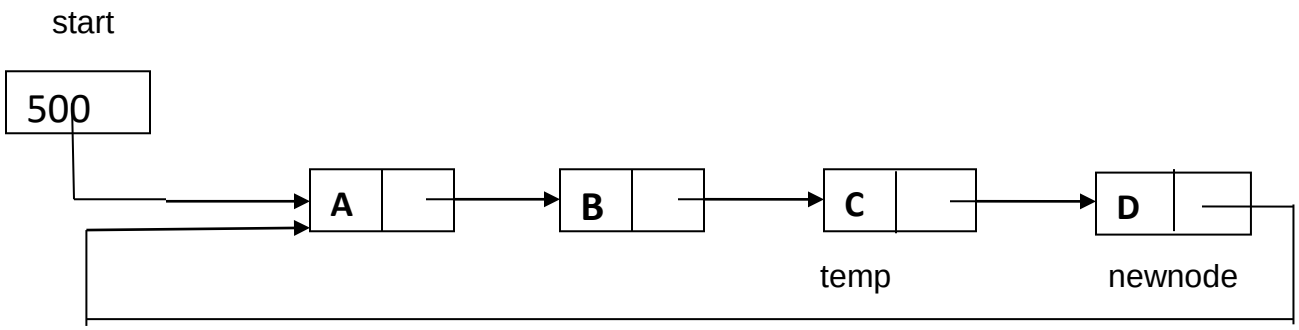
If a node has to insert at the end of a list, then the newnode will insert next to the last node and the newnode will become last node. As the node is to add next to last node, we have to find the last node address and in the address part of it the newnode address will be assigned and the newnode address part will point to the starting node as it the last node now.

```
temp=start;
do
{
temp=temp->next;
} while(temp->next != start);
temp->next =newnode;
```

newnode->next=start;



Before Insertion



After Insertion

While inserting a node at middle of list user has to specify the position value n after which the newnode has to be inserted. So we have to find the address of nth node for assigning the newnde's address in its address part.

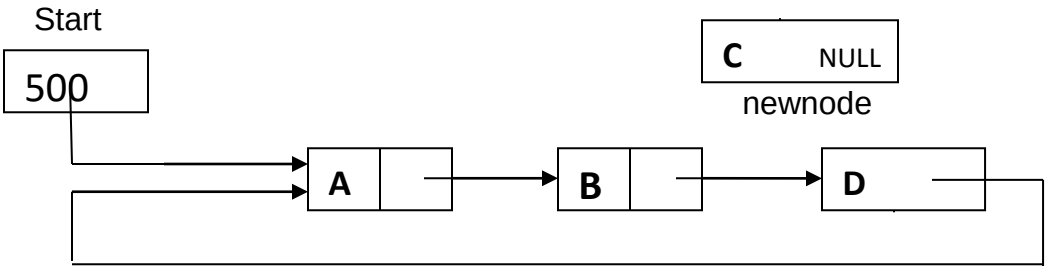
```
temp=start;
i=1;
```

Here temp contains starting nodes address and temp will visit one by one node up to n^{th} node of the list.

```
while (temp->next !=NULL && i <= n)
        {
        temp=temp->next;
        i++;
        }
```

Now temp contains the nth node address and we make.

```
temp->next =newnode;
```



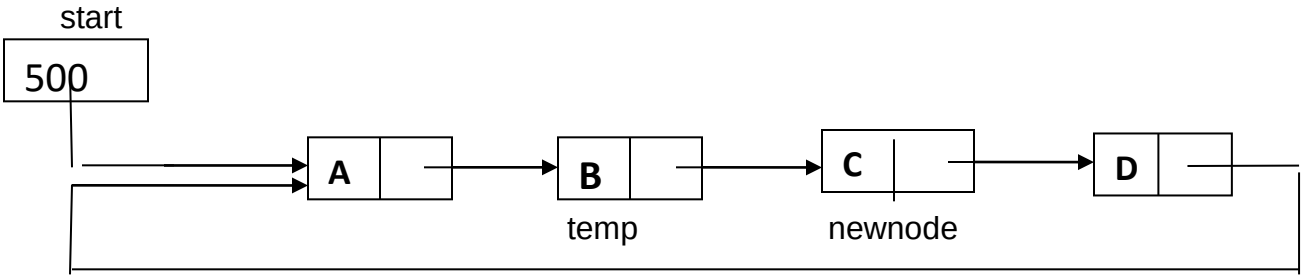
Let newnode will be inserted after position is 2 i.e. n=2.

```
while (temp !=NULL && i < 2)
        {
        temp=temp->next;
        i++;
        }
```

Here loop will run once. So temp temp will contain 2nd node(n=2) address.

```
newnode->next=temp->next;
temp->next =newnode;
```

Here temp->next which is the address of the next node of nth node is to be the next node of the newnode is assigned to the newnode address part. Then the newnode address is assigned to the temp nodes address part.

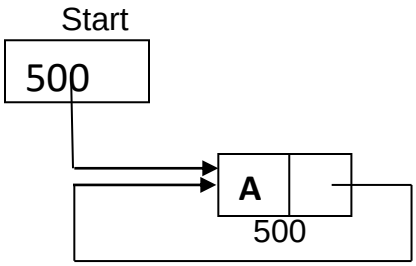


While finding the nth position if the temp value becomes null it has searched all the position and the position is an invalid one.

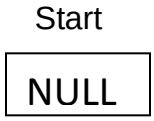
Deleting a node from a specific position has different possibilities also. If the list is empty then there is no node to delete where start pointer contains **NULL**.

If there is a single node in the list then after deleting that node the list becomes empty. So we have to make start NULL.

```
If (start->next==NULL)
        start = NULL;
```

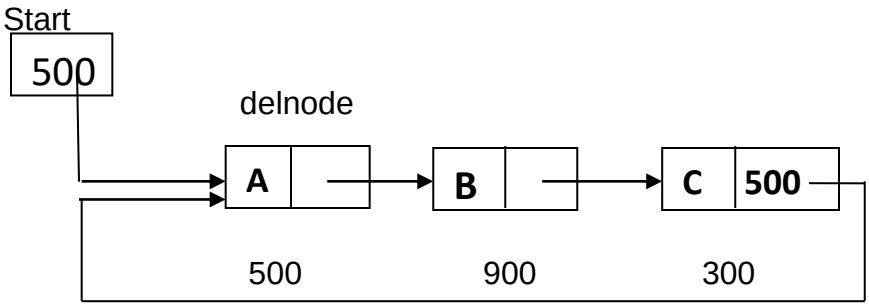


Before Deletion



After Deletion

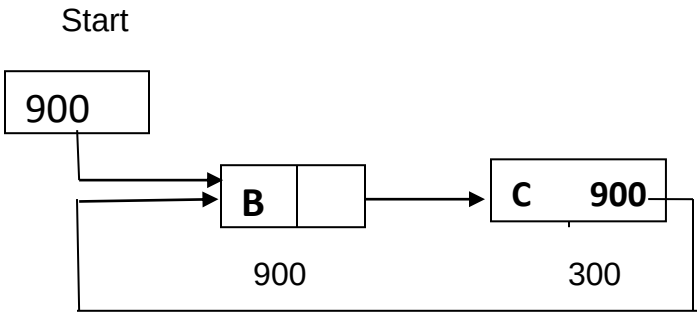
If the start node has to delete then after deletion start pointer will point to the 2nd node whose address is at the address part of starting node. As the starting node is changed ,the last node address part has to be changed.



Before Deletion

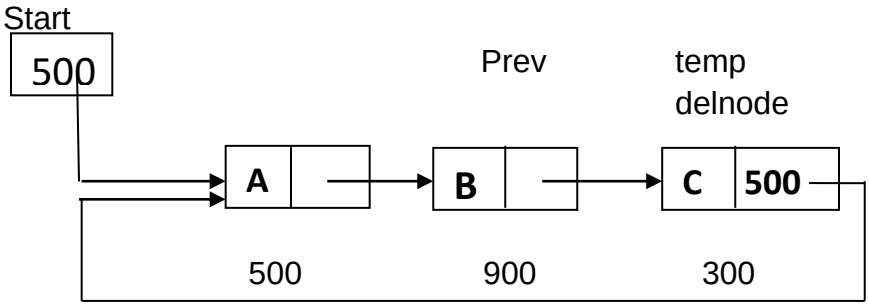
```
temp=start;
do
{
temp=temp->next;
} while(temp->next != start);
```

start=start->next;
temp->next=start;



After Deletion

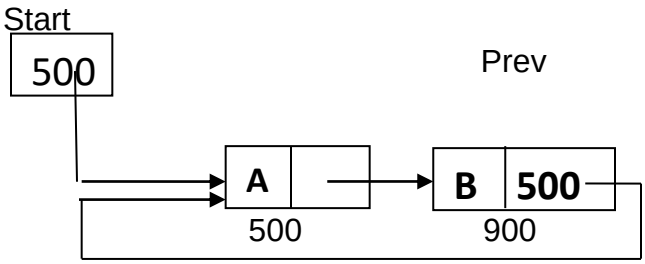
For deleting the last node from a list we need to find the previous node of the last node and in the address part of the previous node we have to store starting node's as it will become the last node.



Before Deletion

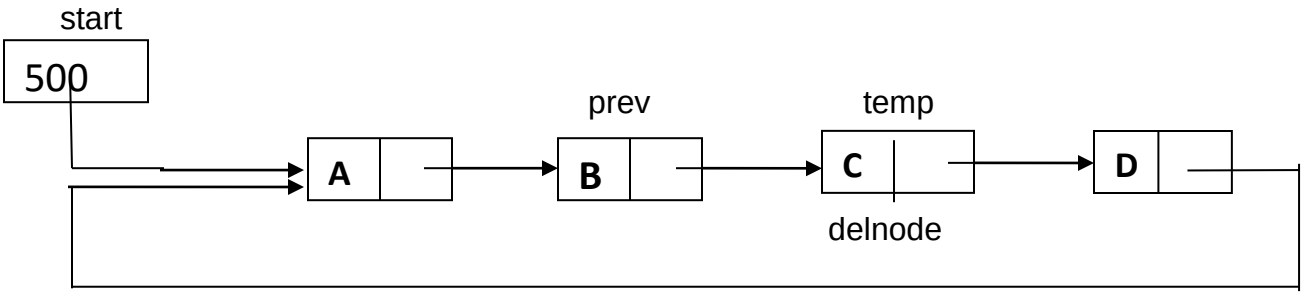
So we need another pointer which contains the previous nodes address of temporary pointer and temporary will takes up to the last node.

```
prev=temp=start;
do
{
        temp=temp->next;
        prev=temp;
} while(temp->next != start;
prev->next=temp->next;
```



After Deletion

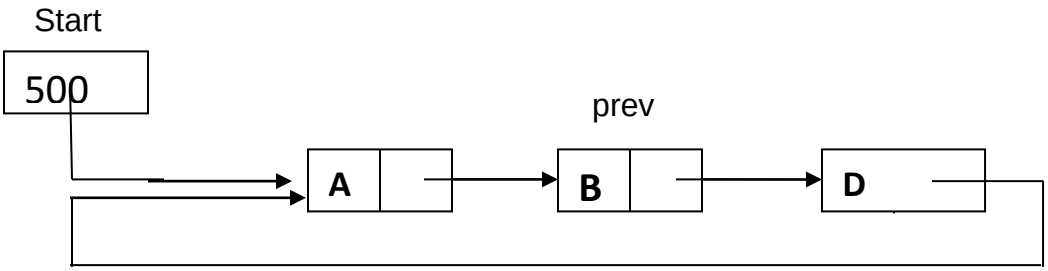
When user specify a specific position ‘n’ to delete we need to take the temporary pointer to n^{th} position and the previous pointer to its previous node n-1.



Before deletion

Let the deleting node position is 3.

```
prev = temp=start;
i=1;
while(temp->next != null && i < 3)
{
        temp=temp->next;
        prev=temp;
}
prev->next=temp->next
```



After Deletion

If the value of ‘n’ is greater than i then it’s an invalid position.After deleting a node we need to free its dynamically allocated memory using free pointer.

21. Application of Linked lists: Operation on Polynomials

Out of many applications, one application of linked list is used to add polynomials. Polynomials can be represented by an array but array representation cannot handle polynomials having large exponent. Array Representation of $8x^3 + x + 1$is as follows where index of array are represented as exponent and the coefficient values are stored in the indexes.

.

1	1		8
0	1	2	3

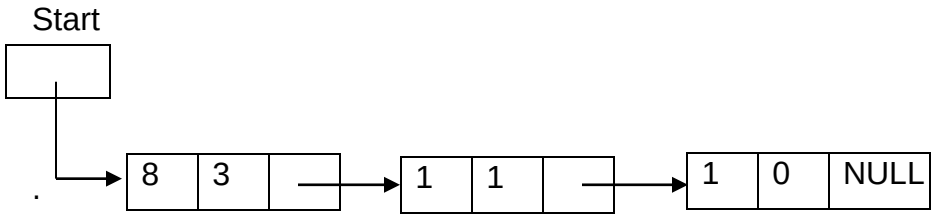
But there is wastage of memory space in an array representation for large polynomials like $5x^{16} + x^2 + x$ as follows.

0	1	1		5
0	1	2		16

For this problem the best representation of polynomial is linked representation. In a linked representation each term of the polynomial is represented by a node where each node has fields for coefficient, exponent and address filed for pointing to the next term. All the nodes of the list are ordered as per the exponent value.

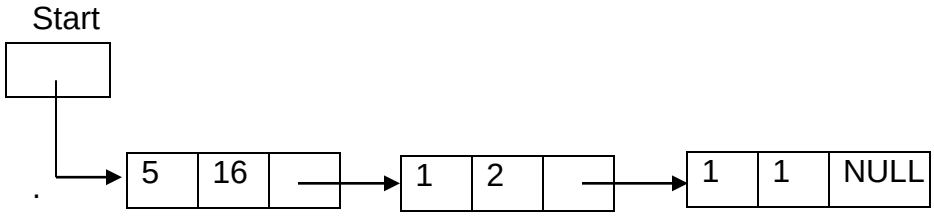
Example:

$8x^3 + x + 1$



.

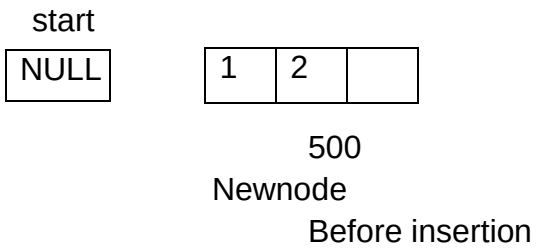
Similarly $5x^{16} + x^2 + x$ as follows



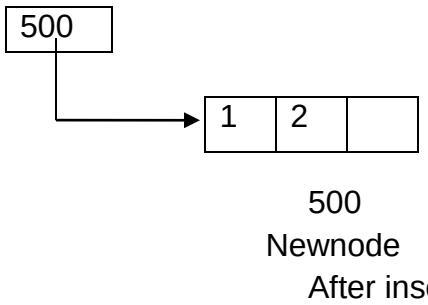
.

Creation of polynomial list:

The polynomial list is an ordered list. So at the time of creation we have search the right position of the node and insert the node at that position. Initially a node is inserted in an empty list.

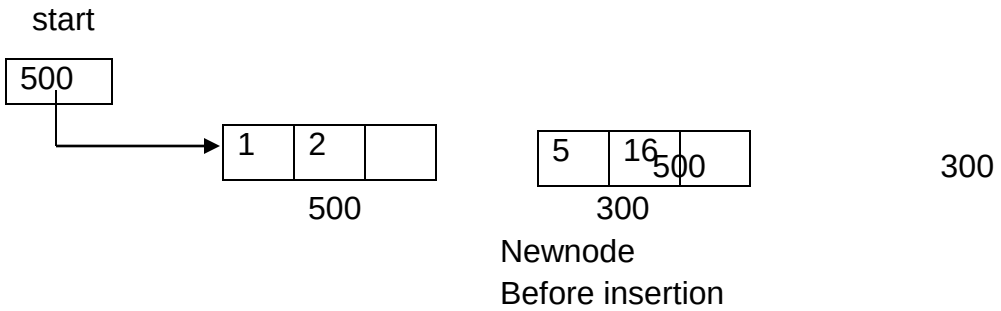


Before insertion

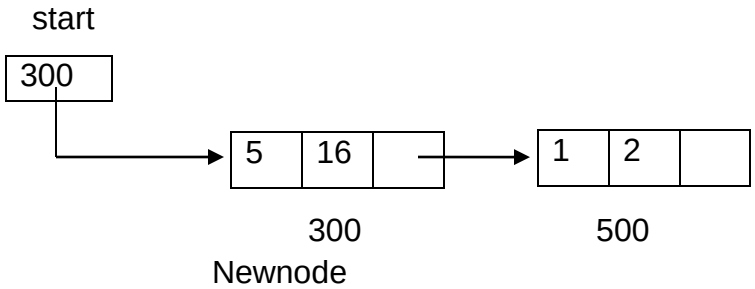


After insertion

If user will insert the exponent value which is greater than the starting node coefficient then the new node will insert at beginning also.



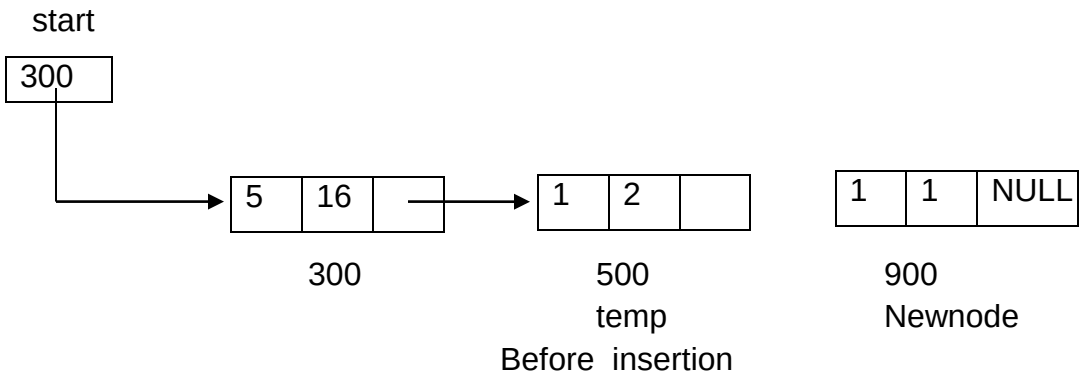
Before insertion



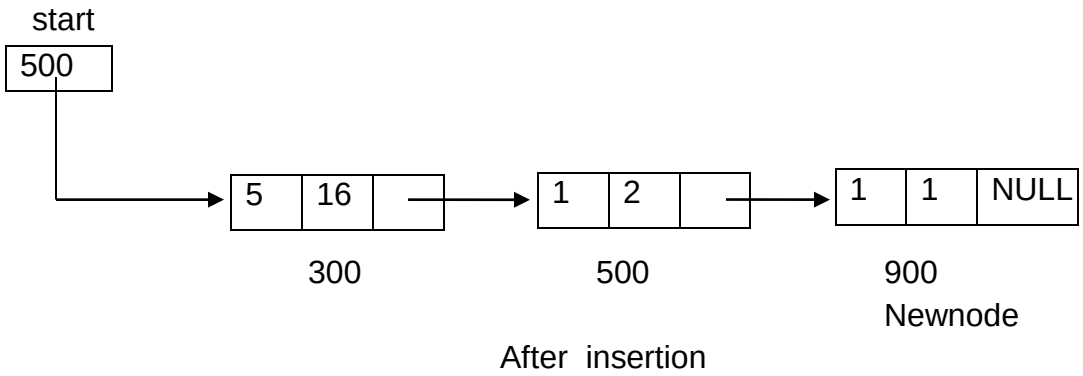
After insertion

```
if(start==null ii start->coeff < newnode->coef)
        {
        newnode->next=start;
        start=newnode;
        }
```

Inserting any other node except first we need to compare the exponent value and find the proper place of a newnode which may be at middle or last.



Before insertion

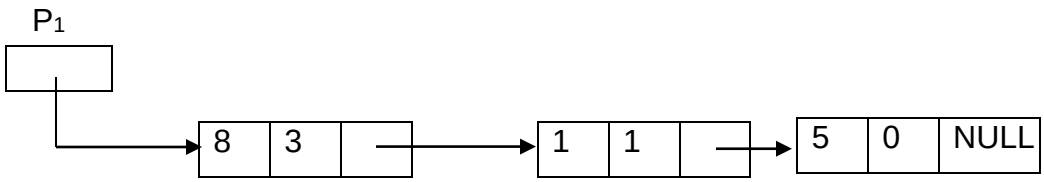


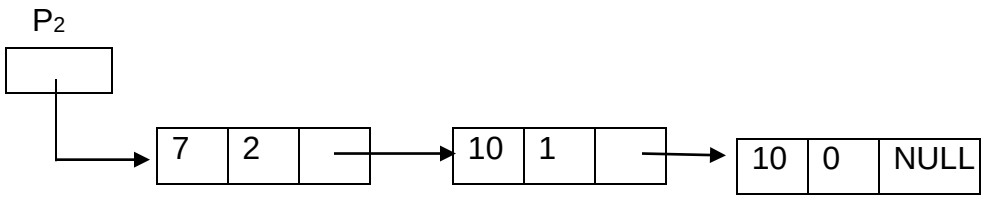
After insertion

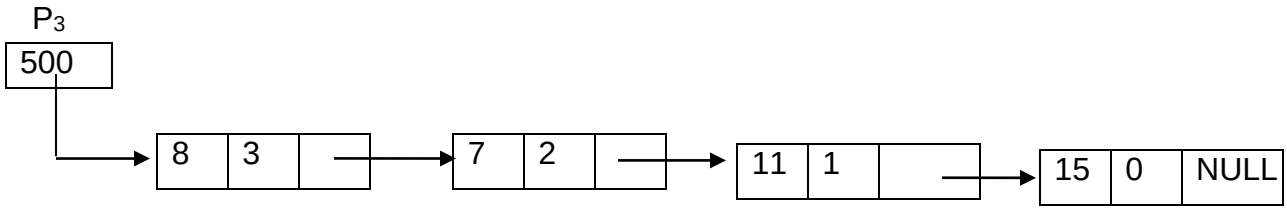
```
temp=prev=start;
if(temp->expo > newnode->expo && temp !=null)
{
        prev=temp
        temp=temp->next;
}
newnode->next=prev->next
prev->next=newnode;
```

```
/*function for creation of polynomials*/
node* create(node *start)
{
	int n,i;
	node *newnode, *temp,*prev;
	start=NULL;
	printf("Enter the no. of nodes : ");
	scanf("%d",&n);
	for(i=0;i<n;i++)
	{
		newnode=(node*)malloc(sizeof(node));
		newnode->next=NULL;
		printf("Enter the coefficient and the exponent of x : \n");
		scanf("%f %d",&newnode->coeff,&newnode->exp);


	if((start==NULL) || start->exp < newnode->exp)
	{
		newnode->next=start;
		start=newnode;
	}
	else
	{
		prev=start;
		temp=start;
		while(temp!=NULL && temp->exp > newnode->exp)
		{
			prev=temp;
			temp=temp->next;
		}
		if(temp!=NULL && temp->exp == newnode->exp)
		{
			temp->coeff+=newnode->coeff;
			free(newnode);
		}
		else
		{
			newnode->next=prev->next;
			prev->next=newnode;
		}
	}
	}
	return(start);
}
```

Addition of polynomials:
Example:
P_1= $8x^3+x+5$
P_2= $7x^2+10x+10$
P_3=$8x^3 + 7x^2 +11x+15$







In an addition we create a third polynomial by appending the higher exponent from the two terms. If there are no more terms in a polynomial, then append all the rest terms of the other polynomial.

```
/*function for addition of polynomials*/
node* add(node *p1, node *p2, node *p3)
{


        //p1,p2,p3 are starting address of the polynomial
        if(p1==NULL && p2==NULL)
                return(p3);
        while(p1!=NULL && p2!=NULL)
        {
                if(p1->exp > p2->exp)
                {
                        p3 = insert_end(p1->coeff,p1->exp,p3);
                        p1=p1->next;
                }
```

```
		else if(p1->exp < p2->exp)
		{
			p2 = insert_end(p2->coeff,p2->exp,p3);
			p2=p2->next;
		}
		else
		{
			p3 = insert_end((p1->coeff+p2->coeff),p1->exp,p3);
			p1=p1->next;
			p2=p2->next;
		}
	}
	while(p1!=NULL)
	{
		p3 = insert_end(p1->coeff,p1->exp,p3);
		p1=p1->next;
	}
	while(p2!=NULL)
	{
		p3 = insert_end(p2->coeff,p2->exp,p3);
		p2=p2->next;
	}

	return(p3);
}

//CODE FOR INSERT_END FUNCTION

node* insert_end(float y,int x,node* start)
{
	node *newnode,*temp;
	newnode=(node*)malloc(sizeof(node));
	newnode->coeff=y;
	newnode->exp=x;
	newnode->next=NULL;
	if(start==NULL)
		start=newnode;
	else
	{
		temp=start;
		while(temp->next!=NULL)
			temp=temp->next;
		temp->next=newnode;
	}
	return(start);
}
```

22. Dynamic Storage Management, Garbage Collection and Compaction

Dynamic Storage Management:

Dynamic memory allocation is performed by the operating system when the program is executing and the program (user) sends a request for additional memory.

Three issues need to be addressed by the operating system:

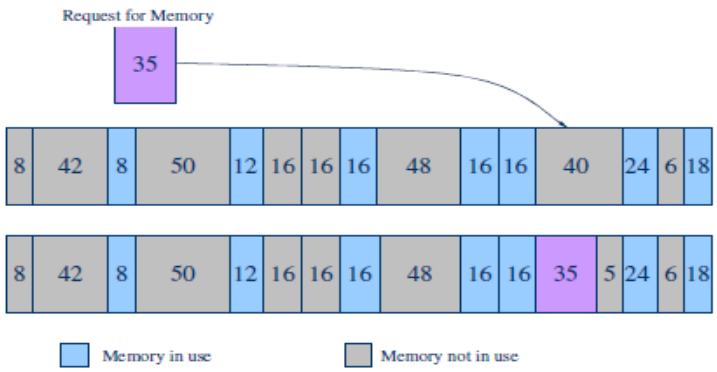
1. allocating variable-length dynamic storage when needed
2. freeing up storage when requested
3. managing the storage (e.g. reclaiming freed up memory for later use)

Heap Allocation Methods:

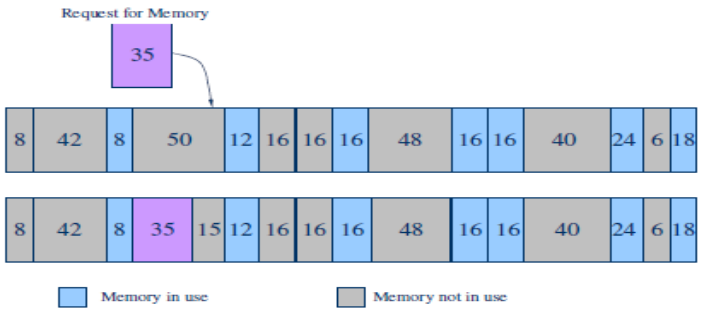
Typically, heap allocation schemes use a free list to keep track of the storage that is not in use.

Algorithms differ in how they manage the free list

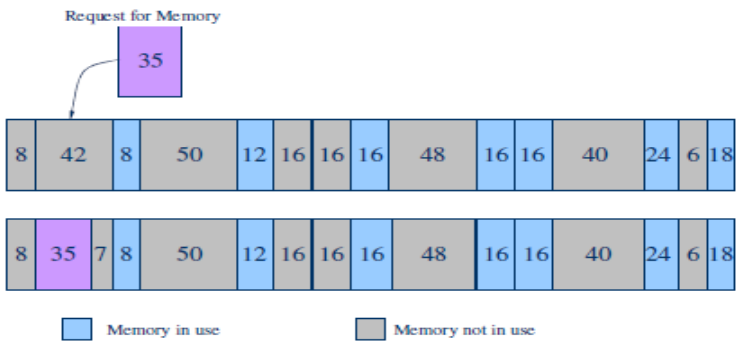
1. **Best Fit**
   - Keep linked list of free blocks.
   - Search the whole list on each allocation.
   - Choose block that comes closest to matching the needs of allocation.
   - Save excess for later.
   - Merge adjacent free blocks during release operations.



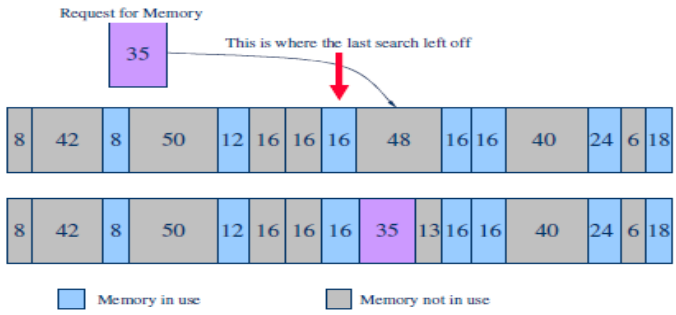
2. **First Fit**
   - Keep linked list of free blocks.
   - Scan the list for the first block that comes closest to matching the needs of allocation.
   - Merge adjacent free blocks during release operations.



3. **Worst Fit**
    - Keep linked list of free blocks.
    - Search the whole list on each allocation.
    - Choose block that worst matches the request.
    - Save excess for later.
    - Merge adjacent free blocks during release operations.



4. **Next Fit**
    - Keep linked list of free blocks.
    - Start where the last search left off.
    - Scan the list for the first block that comes closest to matching the needs of allocation.
    - Merge adjacent free blocks during release operations.



Garbage Collection:
Buddy System (Explicit Garbage Collection):

It is much faster than other heuristics for dynamic memory allocation such as rstt and bestt Its only disadvantage being that blocks must be powers of two in size the buddy system is used in many modern operating systems in particular most versions of UNIX for small block sizes.

At any point in time the memory consists of a collection of blocks of consecutive memory, each of which is a power of two in size. Each block is marked either occupied or free depending on whether it is allocated to the user. The system provides two operations for supporting dynamic memory allocation:

1. Allocate (2^k): Finds a free block of size 2^k marks it as occupied and returns a pointer to it.
2. Deallocate (B): Marks the previously allocated block B as free and may merge it with others to form a larger free block.

Automatic Garbage Collection:

Data that cannot be referenced is generally known as garbage. Garbage collection is the reclamation of chunks of storage holding objects that can no longer be accessed by a program. The garbage collector finds these unreachable objects and reclaims their space by handing them to the memory manager, which keeps track of the free space.
New objects are introduced through object allocations. Parameter passing and assignments can propagate reachability; assignments and ends of procedures can terminate reachability. As an object becomes unreachable, it can cause more objects to become unreachable.

There are two basic ways to find unreachable objects.

1. Reference counting (collects garbage as it is created in each transition)
2. Trace based (periodically locate all the reachable objects)

Reference Counting Garbage Collectors

It is a simple, although imperfect, garbage collector, based on reference counting, which identifies garbage as an object changes from being reachable to unreachable; the object can be deleted when its count drops to zero. With a reference-counting garbage collector, every object must have a field for the reference count.

Reference counts can be maintained as follows:

1. Each time a new pointer refers to the block of memory, the reference count of the block must be increased by 1.Like Object Allocation, Parameter Passing , Reference Assignments, Transitive Loss of Reachability.
2. Each time a pointer that refers the block of memory is changed so it no longer points to that sublist, the reference count of the block must be decreased by 1.like Reference Assignments ,Procedure Returns.

3. When reference count becomes zero, free the memory.

Reference Assignments:

For statement u = v, where u and v are references, the reference count of the object referred to by v goes up by one, and the count for the old object referred to by u goes down by one.

Transitive Loss of Reachability:

Whenever the reference count of an object becomes zero, we must also decrement the count of each object pointed to by a reference within the object.

Disadvantages:

it cannot collect unreachable cyclic data structures, and it is expensive. Cyclic data structures are quite plausible; data structures often point back to their parent nodes, or point to each other as cross references.

Trace-Based Collection

Trace-based collectors run periodically to find unreachable objects and reclaim their space. There are many methods but we discuss only three of them.

i. **Mark-and-Sweep Collectors**
ii. **Mark-and-Compact Collectors**
iii. **Copying collectors**

i. Mark-and-Sweep Collectors

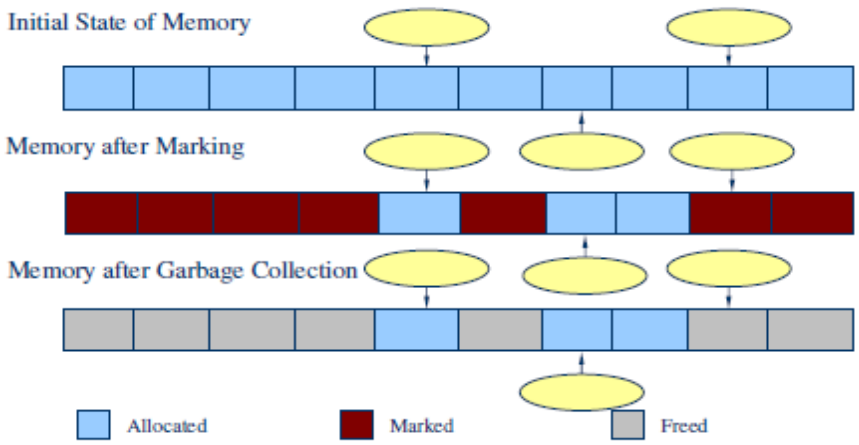
This algorithm visits and "marks" all the reachable objects in the first tracing step and then "sweeps" the entire heap to free up unreachable objects.

Pass 1: Mark

1. Go through all statically-allocated and procedure-level variables, looking for pointers.
2. Mark each object pointed to, and recursively mark all objects it points to.

Pass 2: Sweep

1. Go through all objects, free up those that aren’t marked



ii. Mark-and-Compact Collectors

It is common that the space occupied by reachable objects is much smaller than the freed space. Thus, after identifying all the holes, instead of freeing them individually, one attractive alternative is to relocate all the reachable objects into one end of the heap, leaving the entire rest of the heap as one free chunk.

The mark-and-compact collector in Algorithm 7.15 has three phases:

1. First is a marking phase, similar to that of the mark-and-sweep algorithms described previously.
2. Second, the algorithm scans the allocated section of the heap and computes a new address for each of the reachable objects. New addresses are assigned from the low end of the heap, so there are no holes between reachable objects.
3. The new address for each object is recorded in a structure called NewLocation.
1. Finally, the algorithm copies objects to their new locations, updating all references in the objects to point to the corresponding new locations. The needed addresses are found in NewLocation.

iii. Copying collectors

The memory space is partitioned into two semispaces, A and B. The mutator(A user program, which refer to as the mutator, modifies the collection of objects in the heap) allocates memory in one semispace, say A, until it fills up, at which point the mutator is stopped and the garbage collector copies the reachable objects to the other space, say B. When garbage collection completes, the roles of the semispaces are reversed. The

mutator is allowed to resume and allocate objects in space B, and the next round of garbage collection moves reachable objects to space A.

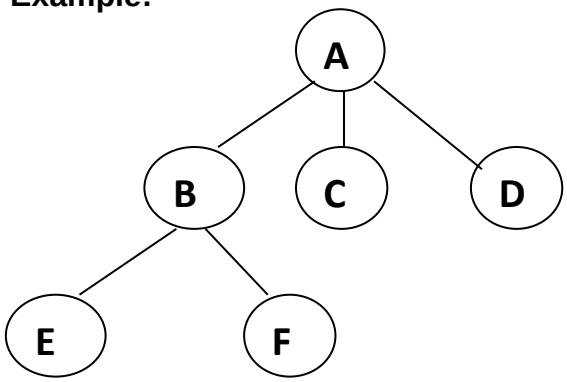
23. Tree: Definition, Terminology

Tree:

A tree is finite set of one or more nodes such that there is a special designated node called root and the remaining nodes partitioned into n disjoint sets, where n>= 0 and T_1, T_2, ….. T_n are the subtrees of the root.

Each node contains some data and linked to its parent and children. The root node has no parent and the nodes have no child called leaf nodes or terminal nodes.

Example:



In the above tree A is the root and B, C, D are children / subtree of it where A is the parent of B,C,D.There is no parent of A as it's the root. Similarly B is the parent of E and F, which are the children of it. As C, D,E and F have no child ,they are the leaf / terminal node.

Degree of a node: The no of subtree is the degree of a node. In the above figure A, B, C, D, E, F has 3, 2, 0, 0, 0 degree respectively.

The node whose degree is zero is called leaf node otherwise they are called non-leaf node.

Sibling: Children of the same parent /node are sibling to each other. In the above figure E & F are siblings of each other. Similarly B is the left sibling and D is the right sibling of C.

Edge: It is the connecting line or link between two nodes

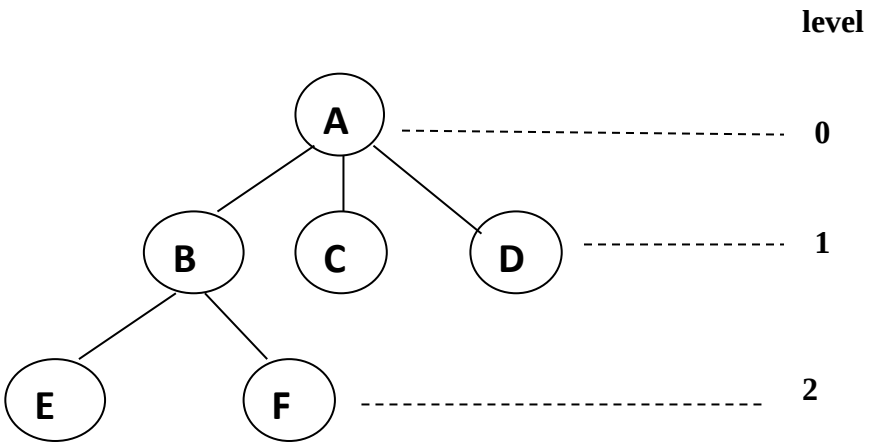
Path: It is a sequence of edges from a source node to destination node.

Predecessor: The nodes which are along the path of root node to the node are predecessor of that node.

Successor: The nodes which are along the paths from the node to the leaf nodes are successor of that node.

Node	Predecessor	Successor
A	NIL	3
B	1	2
C	1	0
D	1	0
E	2	0
F	2	0

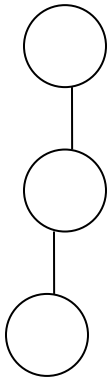
Level: It is the no of edges along the path from root to the node.



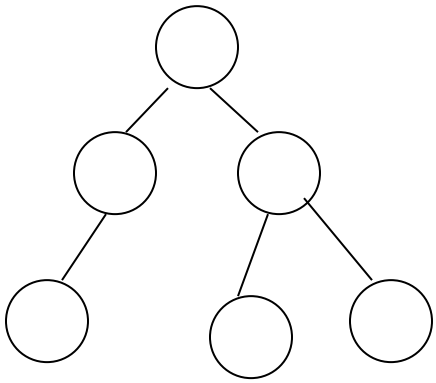
Height: It is the no of edges in the longest path from leaf nodes. The height of all leaf nodes is Zero. The height of root node is the height of tree.

Node	Height
A	2
B	1
C	0
D	0
E	0
F	0

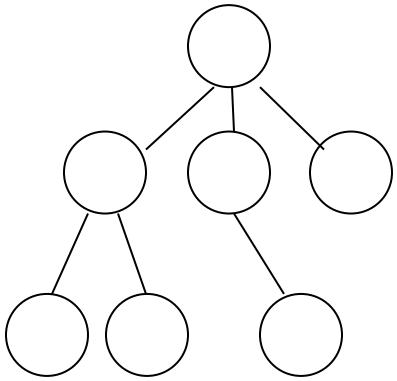
n-ary Tree: Trees having maximum **n** children are known as n-ary Tree. Here n can be any integer value. If n is two it is called binary tree. In a ternary tree n is 3.



Unary Tree



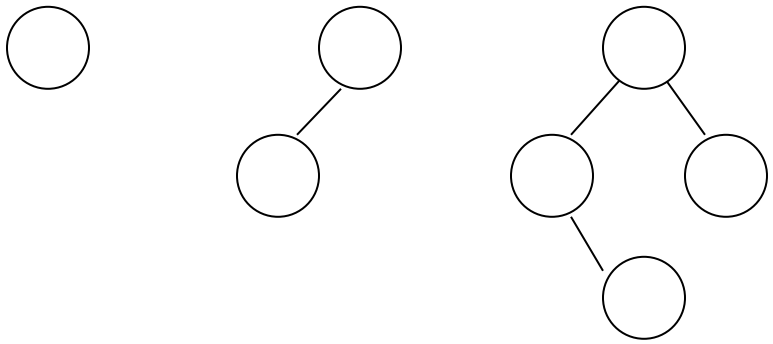
Binary tree



Ternary Tree

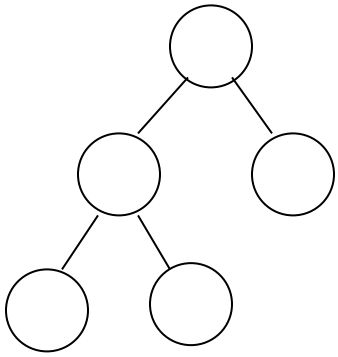
24. Binary Trees: Definition, Representation

Definition:
Trees having maximum two children are called binary tree.

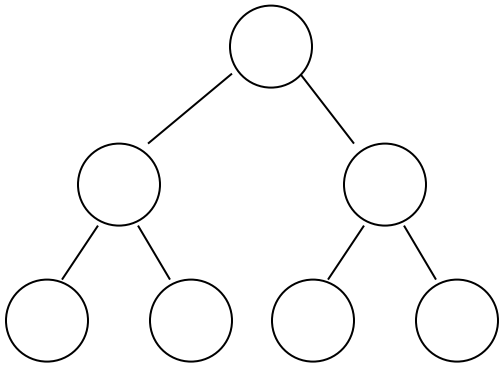


Types of Binary Trees:

Full Binary Tree/Strictly Binary Tree:
It is a tree in which every internal node has exactly two children.

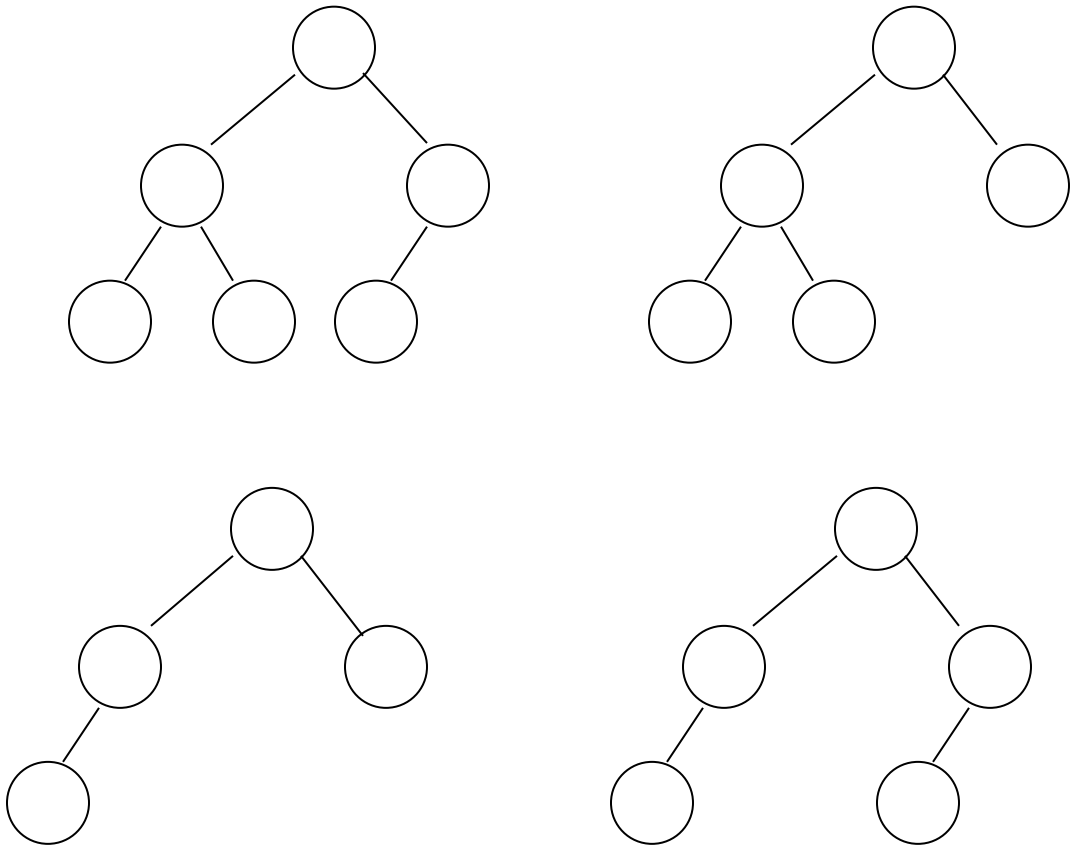


Complete Binary Tree:
It is a tree in which all leaves are at same level.



Almost Complete Binary Tree:

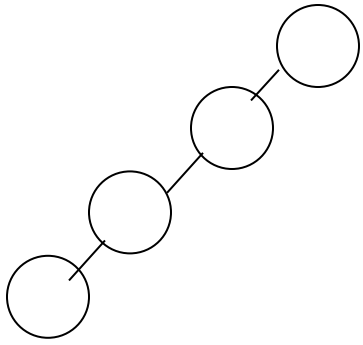
It is a binary tree in which all levels except the last level are completely filled and the last level is as possible left filled.



Out of the above figures the last figure is not an Almost Complete Binary Tree**.**

Skewed Binary Tree:

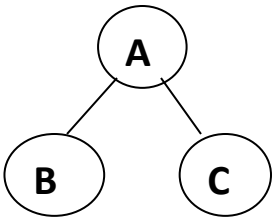
A tree can be skewed towards either left or right. If all internal nodes of a tree have left child only then it is called a left skewed binary tree. If all internal nodes of a tree have right child only then it is called a right skewed binary tree.



Representation of Binary Tree:

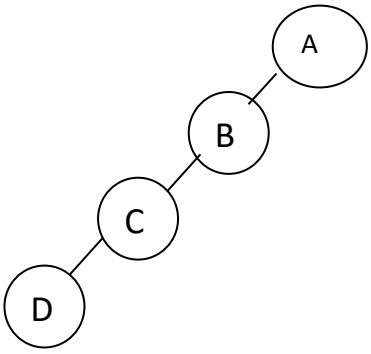
There are two types of representation, array and linked list.

Array representation:



A	**B**	**C**
0	**1**	**2**

In array representation if parent is at index I, then left child will be at 2i+1 and right child will be at 2i+2.The demerit of array representation is the wastage of space in some cases like skewed binary tree.



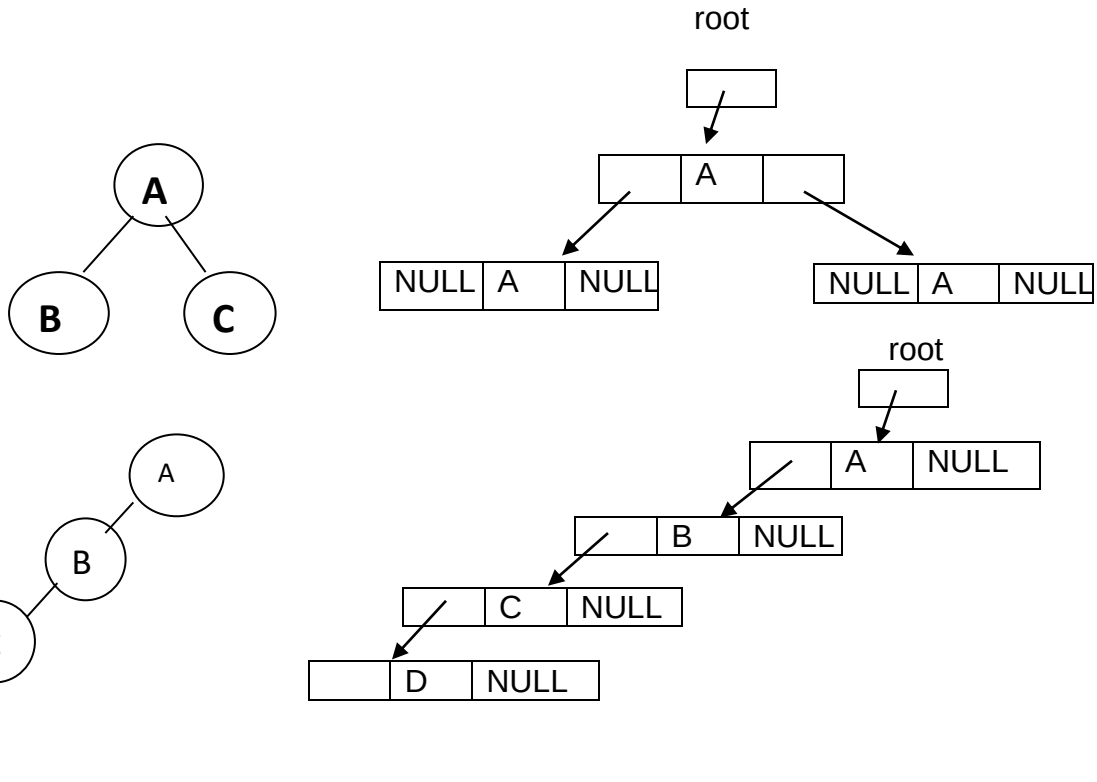
A	B		C				D
0	1	2	3	4	5	6	7

Linked List representation:

The node structure of a tree is represented as follows.

Left Child Address	Data	Right Child Address

Threre is a root pinter which contains the root nodes address. All the operation of a tree is stated from the root node like start pointer in linked list.



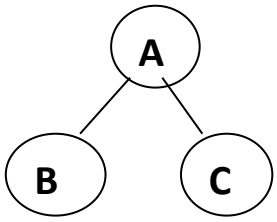
The node structure is defined as follows.

```
typedef struct node
{
struct node *lc;
char data;
strct node *rc;
}node;
```

25. Binary Tree Traversals

Three ways of traversal are there

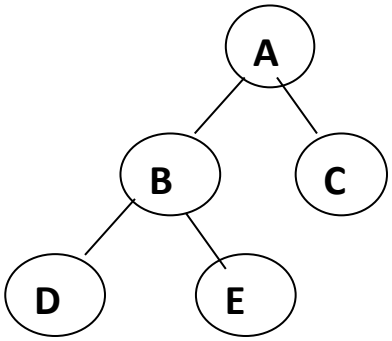
i. Inorder traversal
ii. Preorder traversal
iii. Post order traversal



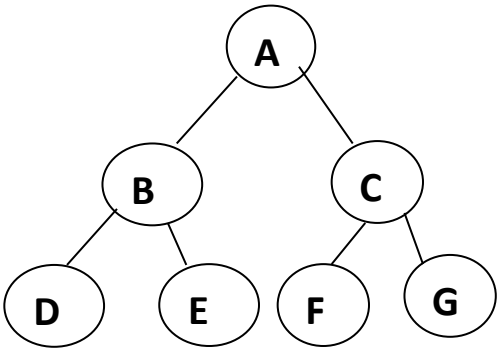
In an inorder traversal the left child / subtree is visited first, then the root and in the last right child / subtree is visited. So the inorder traversal of the above tree is BAC.

In an preorder traversal the the root is visited first, then left child/ subtree and in the last right child/ subtree is visited. So the preorder traversal of the above tree is ABC.

In an postorder traversal the left child/ subtree is visited first, then the right child / subtree and in the last root is visited. So the postorder traversal of the above tree is BCA



Inorder: DBEAC
Preorder: ABDEC
Postorder: DEBCA



Inorder: DBEAFCG
Preorder: ABDECFG
Postorder DEBFGCA

The function to traverse a tree in inorder traversal is as follows.

```
void inorder(node *t)
{
	if(t!=NULL)
	{
		traverse_inorder(t->lc);
		printf("\t%c",t->data);
		traverse_inorder(t->rc);
	}
}
```

The function to traverse a tree in preorder traversal is as follows.

```
void preorder(node *t)
{
	if(t!=NULL)
	{
		printf("\t%c",t->data);
		traverse_inorder(t->lc);
		traverse_inorder(t->rc);
	}
}
```

The function to traverse a tree in postorder traversal is as follows.

```
void postorder(node *t)
{
	if(t!=NULL)
	{
		traverse_inorder(t->lc);
		traverse_inorder(t->rc);
		printf("\t%c",t->data);
	}
}
```

26. Binary Search Tree: Insertion

A binary search tree is a sorted binary tree where the values in the left subtree are smaller than the root node and the values in the right subtree are greater than the root node.

It is a tree which may or may not be empty but it satisfy the following properties.

i. Each node in the tree has unique values/ keys
ii. The keys in the left subtree are smaller than the parent node and the keys in the right subtre are greater than the parent node.
iii. The left and right subtree should satisfy the above properties.

Advantage of BST:

I. The search operation is efficient in a BST as it is sorted.
II. Insertion and deletion take maximum $\log_2 n$

Insertion of nodes into BST:

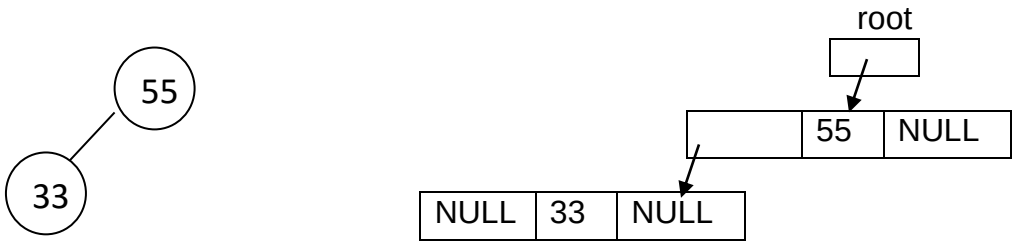
Nodes into BST are inserted as per their position. So the position of the node has to find first then the node is linked to its parent. Initially the tree is empty for which root is assigned to NULL. Then the keys as per their arrival/access order.

Example:

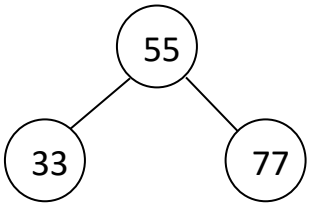
Insert the following values into a BST.
55, 33, 77, 11, 24, 99, 16, 67.

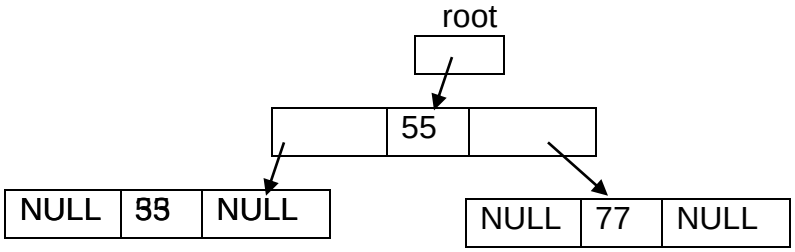
Initially root is empty. When the value '55' is inserted, the root points to its node.

Insert 33

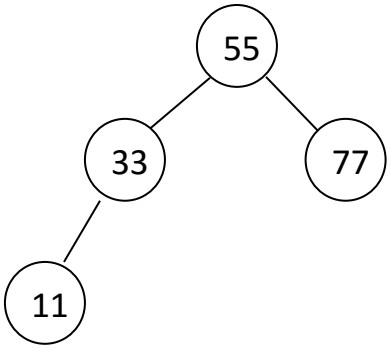


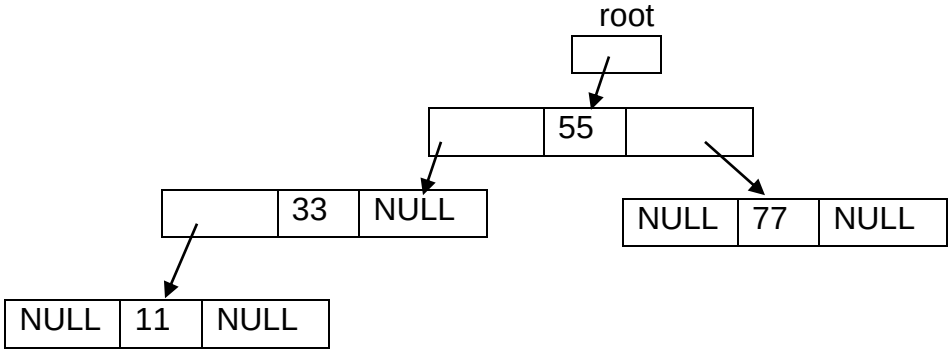
Insert 77



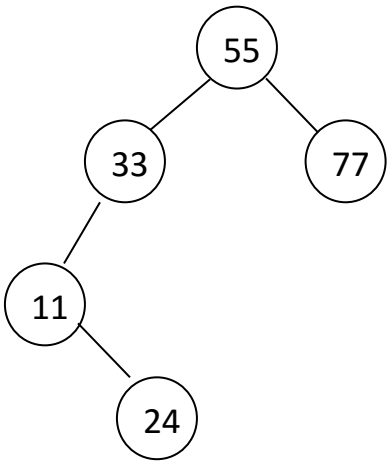


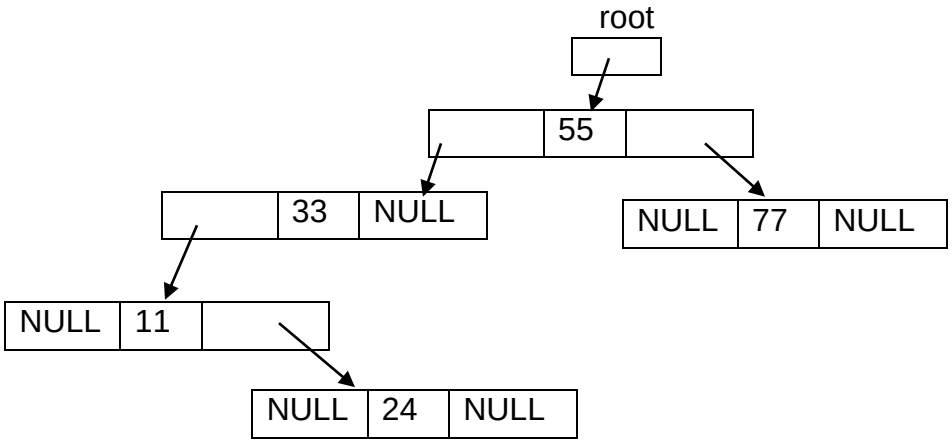
Insert 11



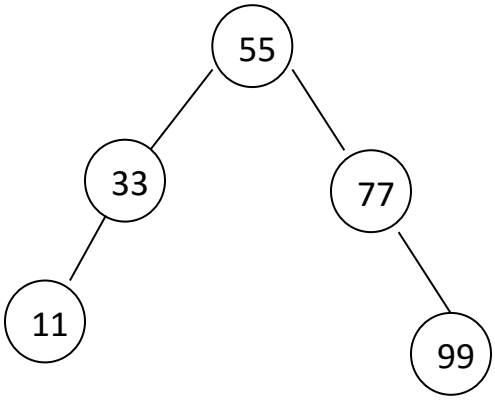


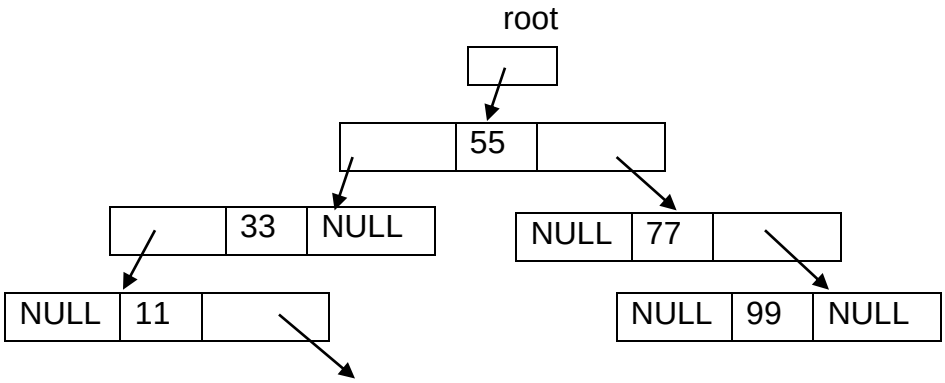
Insert 24

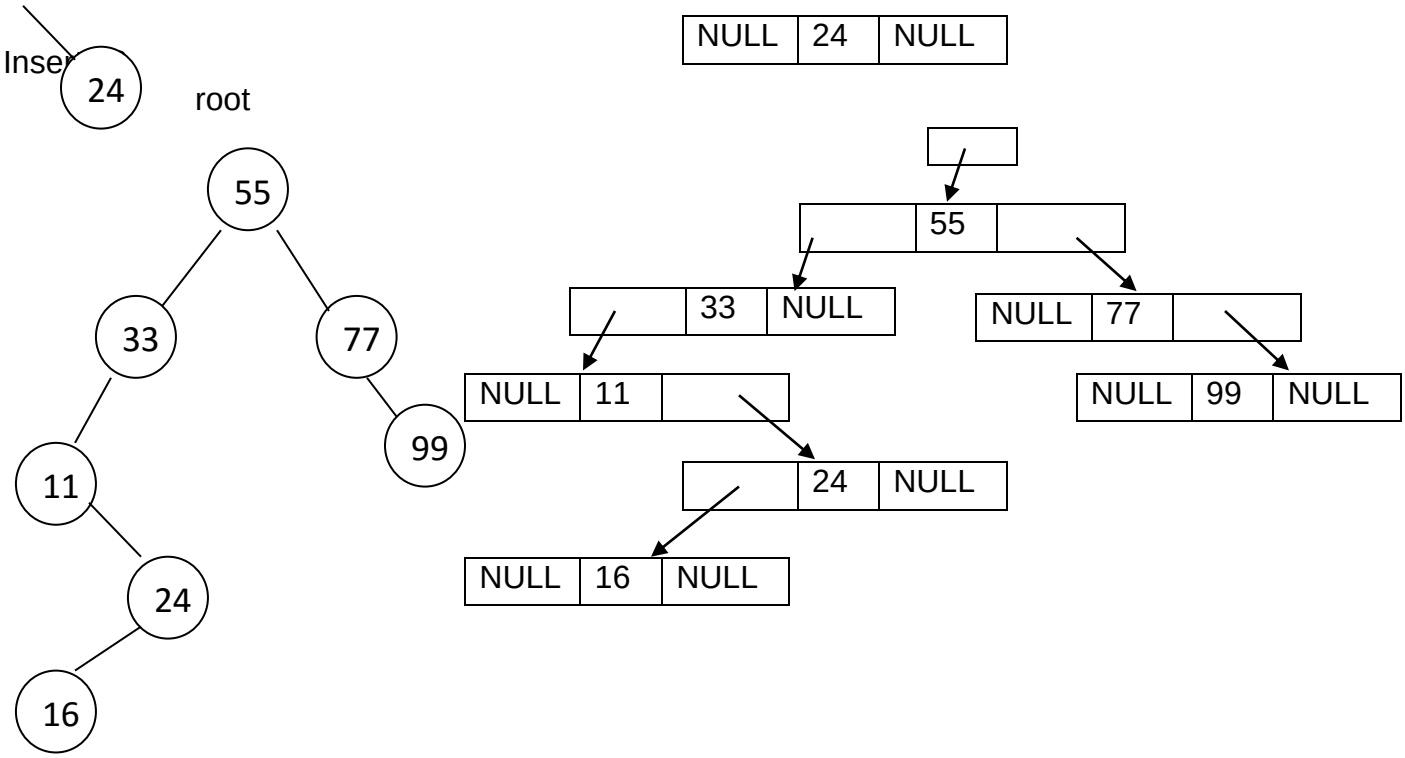




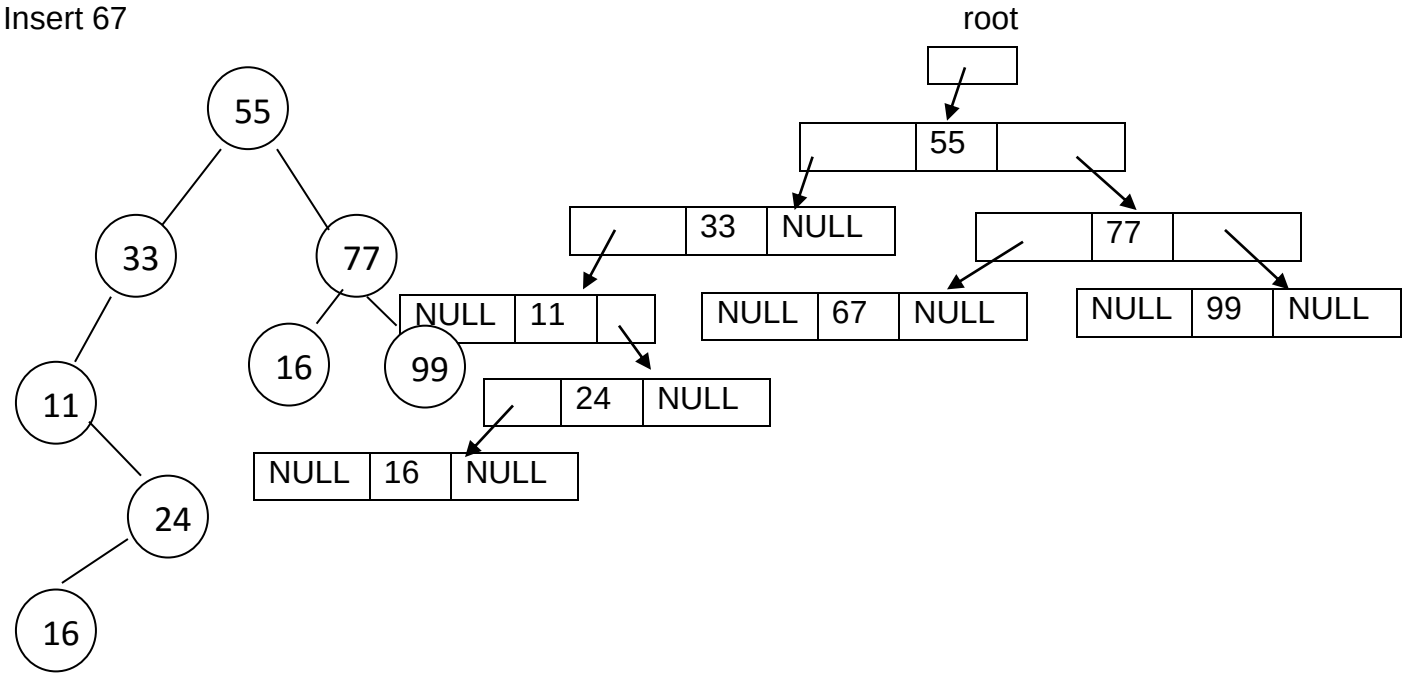
Insert 99






Inser
24
root
55
33
77
11
99
24
16
NULL 24 NULL
55
33 NULL
NULL 77
NULL 11
NULL 99 NULL
24 NULL
NULL 16 NULL

Insert 67


root
55
33
77
11
16
99
24
16
55
33 NULL
77
NULL 11
NULL 67 NULL
NULL 99 NULL
24 NULL
NULL 16 NULL

The c function for inserting node into a tree is as follows.

```
node* insert(node *t,char x)
{
	node *temp=t;
	if(t==NULL)
	{
		t=(node*)malloc(sizeof(node));
		t->data=x;
		t->lc=NULL;
		t->rc=NULL;
	}
	else if(t->data > x)
		t->lc=insert(t->lc,x);
	else if(t->data < x)
		t->rc=insert(t->rc,x);
	else
	{
		printf("Duplicate data not allowed\n");
	}

		return t;
}
```

27. Binary Search Tree: Deletion

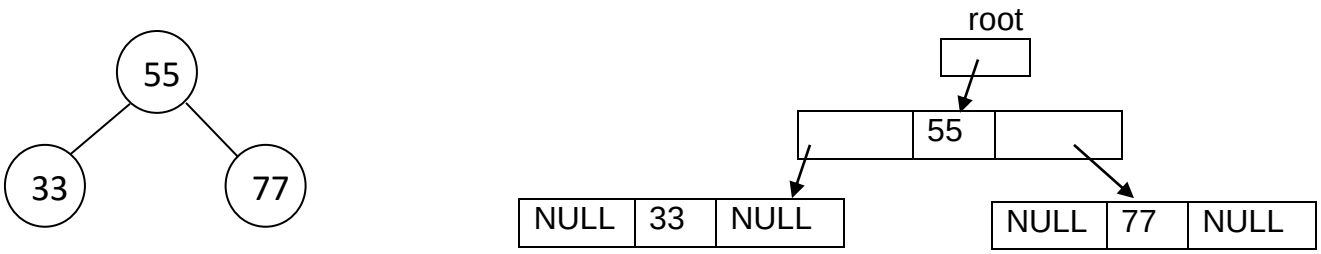
If there is a Tree T and the node to delete is N and the parent of N is P(N) then the node can be deleted in three ways.

i. If N is a leaf node
ii. If N has one subtree either left or right
iii. If N has both subtrees

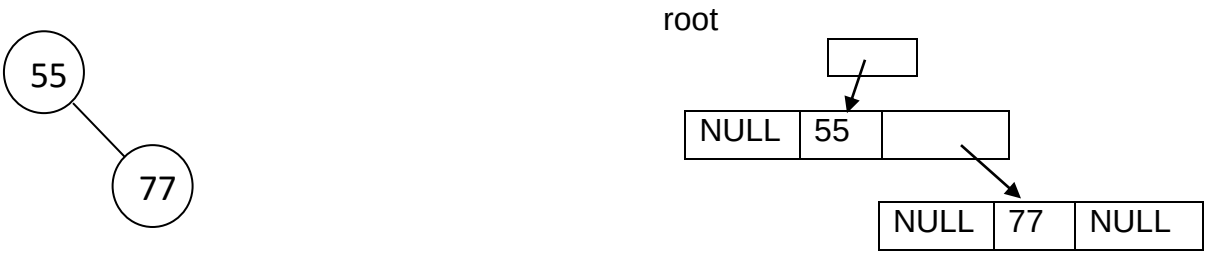
If N is a leaf node, simple delete N from T by replacing the address of N at P(N) with NULL.

Example:

Delete 33 from the Tree below.



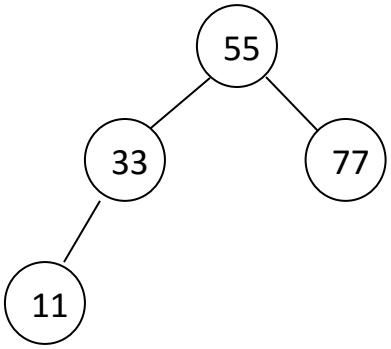
Before deletion

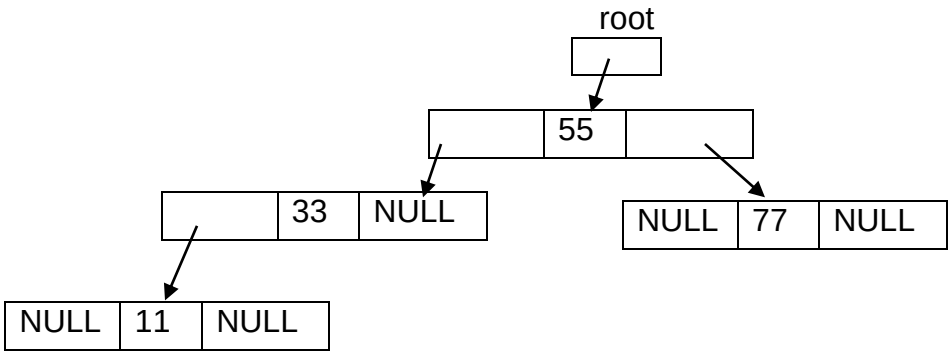


After deletion

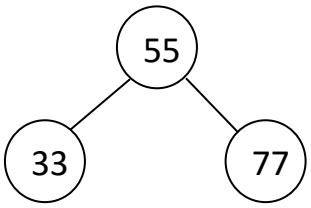
If N has exactly one child / subtree, then delete N from T by replacing the address of N at P(N) with the address of its only child.

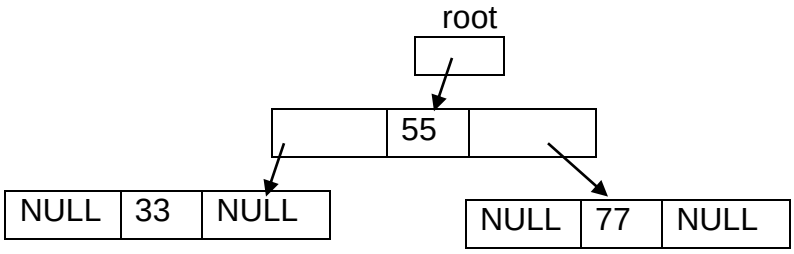
Example
:
Delete 11 from the Given Tree.





Before deletion



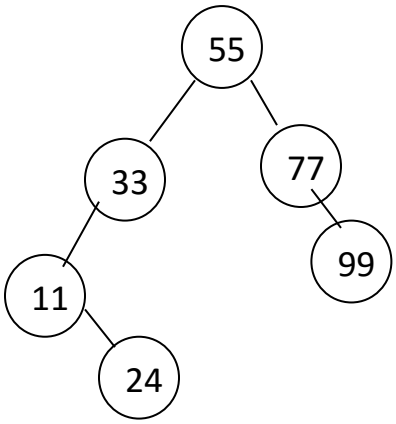


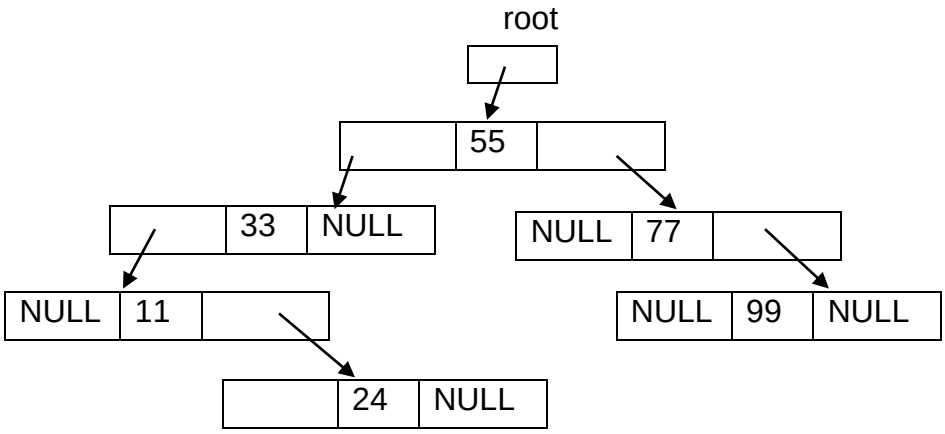
After deletion

If N has both children / subtrees ,then delete N by replacing the value of N with ,either the largest value node from its left subtree or by deleting either the smallest value node from its right subtree and delete that node from which the value is replaced..

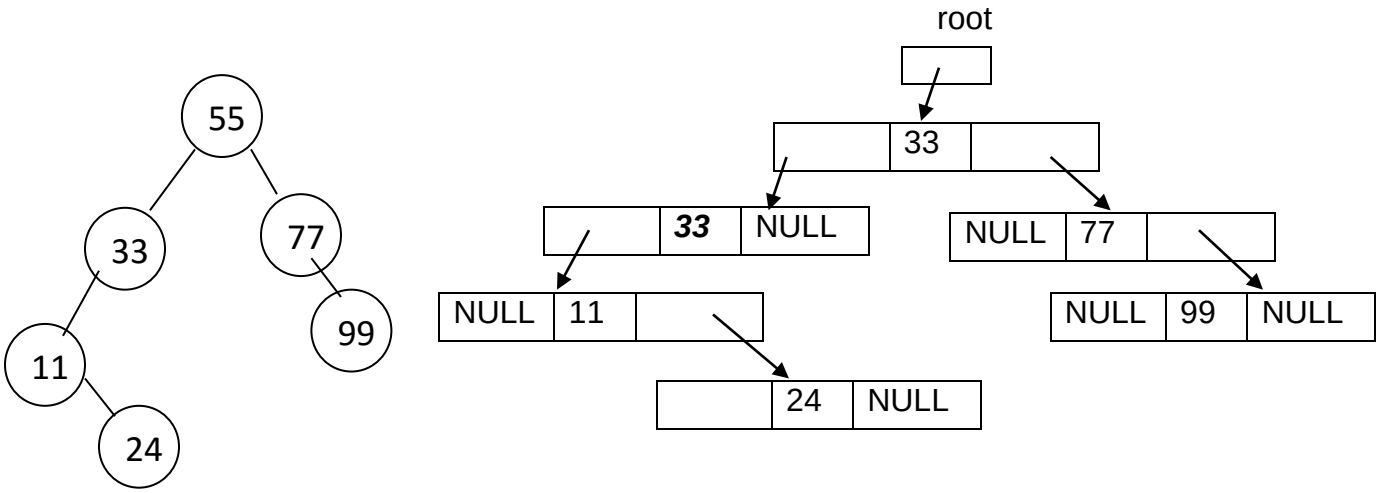
Example:

Delete 55 from the tree.

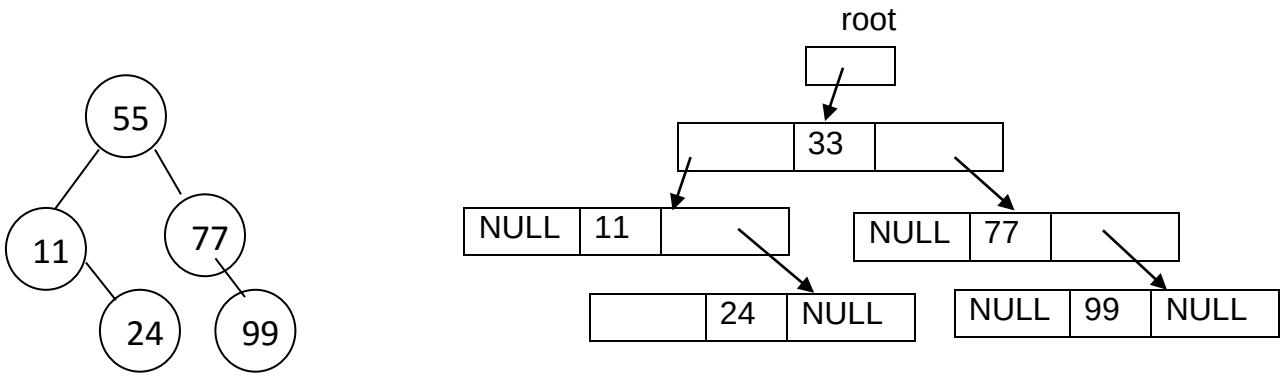




For deleting the node 55 ,we have to replace 55 with either 33 or 77 and that node will be deleted.



After replacement in Deletion



After Deletion

The C code for delteng nay node is as follows.

```
node *delete(node *t,char x)
{
	node *delnode;
	if(t==NULL)
		printf("The tree is empty\n");
	else if(t->data > x)
		t=delete(t->lc,x);
	else if(t->data < x)
		t=delete(t->rc,x);
	else if(t->lc!=NULL && t->rc!=NULL)
	{
		delnode=findmax(t->lc);
		t->data=delnode->data;
		t->lc=delete(t->lc,delnode->data);
		free(delnode);
	}
	else
	{
		delnode=t;
		if(t->lc==NULL)
			t=t->rc;
		else
			t=t->lc;
		free(delnode);
	}

	return(t);
}

//code to find maximum node
node* findmax(node *t)
{
	if(t==NULL)
		return t;
	else if(t->rc==NULL)
		return t;
	else
		findmax(t->rc);
}
```

28. General Tree to Binary Tree Conversion, Threaded Binary Tree

General Tree to Binary Tree Conversion:

General trees are those in which the number of subtrees for any node is not required to be 0, 1, or 2. The tree may be highly structured and therefore have 3 subtrees per node in which case it is called a ternary tree.
However, it is often the case that the number of subtrees for any node may be variable. Some nodes may have 1 or no subtrees, others may have 3, some 4, or any other combination. The ternary tree is just a special case of a general tree (as is true of the binary tree).

General trees can be represented as ADT's in whatever form they exist. However, there are some substantial problems. First, the number of references for each node must be equal to the maximum that will be used in the tree.
Obviously, some real problems are presented when another subtree is added to a node which already has the maximum number attached to it. It is also obvious that most of the algorithms for searching, traversing, adding and deleting nodes become much more complex in that they must now cope with situations where there are not just two possibilities for any node but multiple possibilities. It is also possible to represent a general tree in a graph data structure (to be discussed later) but many of the advantages of the tree processes are lost.

Fortunately, general trees can be converted to binary trees. They don't often end up being well formed or full, but the advantages accrue from being able to use the algorithms for processing that are used for binary trees with minor modifications. Therefore, each node requires only two references but these are not designated as left or right. Instead they are designated as the reference to the first child and the reference to next sibling. Therefore the usual left pointer really points to the first child of the node and the usual right pointer points to the next sibling of the node. One obvious saving in this structure is the number of fields which must be used for references. In this way, moving right from a node accesses the siblings of the node (that is all of those nodes on the same level as the node in the general tree). Moving left and then right accesses all of the children of the node (that is the nodes on the next level of the general tree).

Creating a Binary Tree from a General Tree

Since the references now access either the first child or successive siblings, the process must use this type of information rather than magnitude as was the case for the

binary search tree. Note that the resulting tree is a binary tree but not a binary search tree.

The process of converting the general tree to a binary tree is as follows:

* use the root of the general tree as the root of the binary tree

* determine the first child of the root. This is the leftmost node in the general tree at the next level

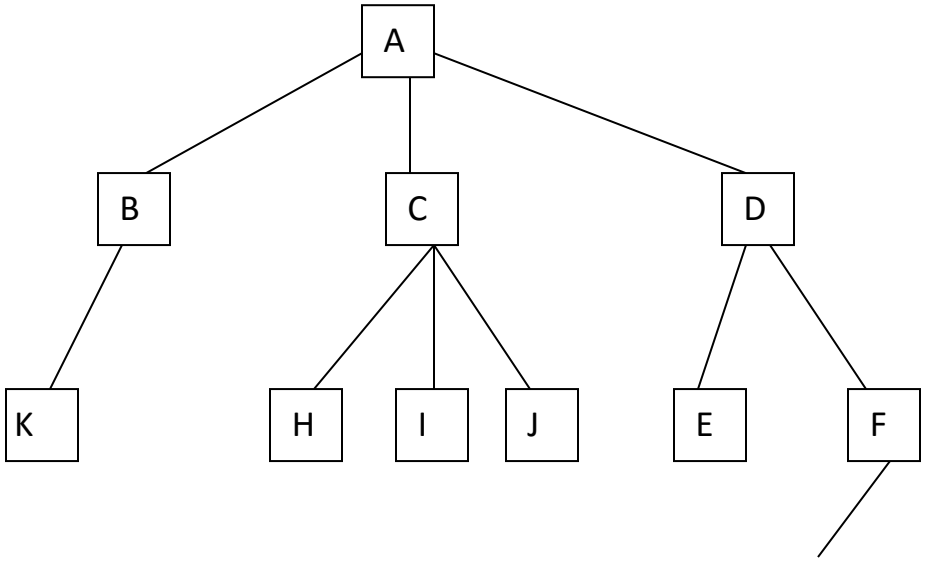
* insert this node. The child reference of the parent node refers to this node

* continue finding the first child of each parent node and insert it below the parent node with the child reference of the parent to this node.

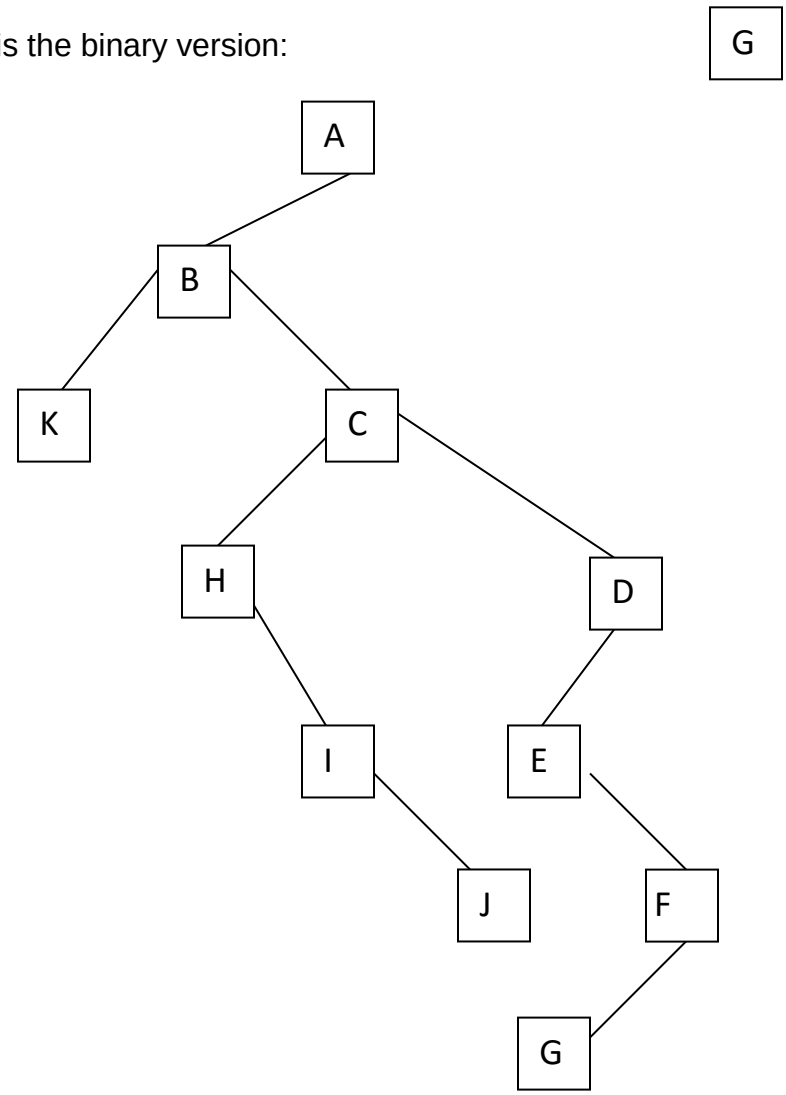
* when no more first children exist in the path just used, move back to the parent of the last node entered and repeat the above process. In other words, determine the first sibling of the last node entered.

* complete the tree for all nodes. In order to locate where the node fits you must search for the first child at that level and then follow the sibling references to a nil where the next sibling can be inserted. The children of any sibling node can be inserted by locating the parent and then inserting the first child. Then the above process is repeated.

Given the following general tree:



The following is the binary version:



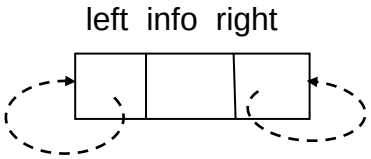
Threaded Binary Trees:

In a binary tree representation approximately half of the pointer fields (left and right) contains null element. This null space may be used by replacing some other pointers which point to higher nodes in the tree. These special pointers are called *threads* which are distinguished in some way from ordinary pointers. Binary trees with such pointers are called *threaded trees*.

Threads may be denoted by negative integers when ordinary pointers are denoted by

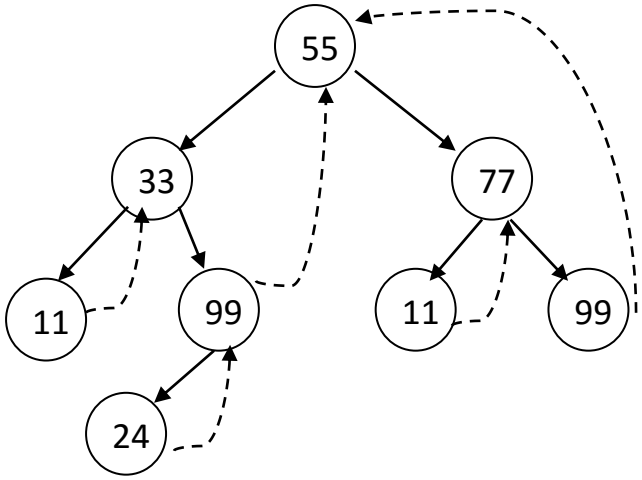
positive integers. They are many ways to thread a binary tree T, such as one-way threading or a two-way threading. Each threading will correspond to a particular traversal of a binary tree.

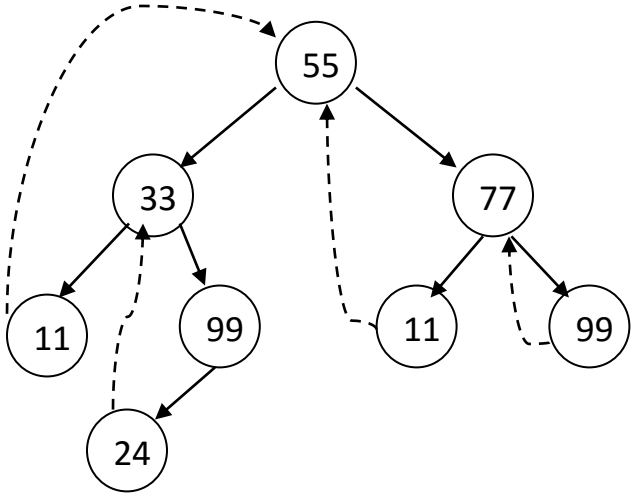
A node Structure



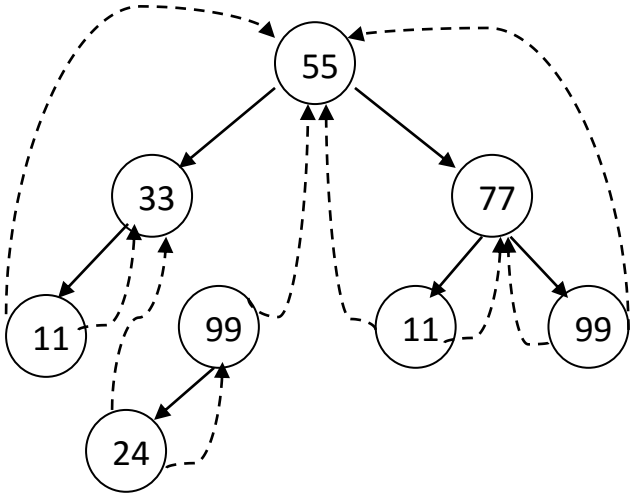
One-way inorder threading of a binary tree.

Inorder: 11 33 24 99 55 11 77 99





Two-way inorder threading of a binary tree.



29. Expression Tree, Manipulation of Arithmetic Expressions

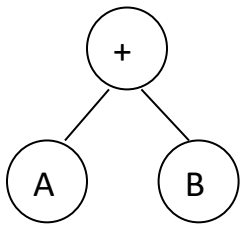
Expression Tree:

It is strictly binary tree for evaluating binary arithmetic expression in which operators are maintained as interior nodes and their operands are leaf nodes.
It is drawn from bottom up manner where bottom level contains highest precedence operators and the root node contains the operator having least precedence.

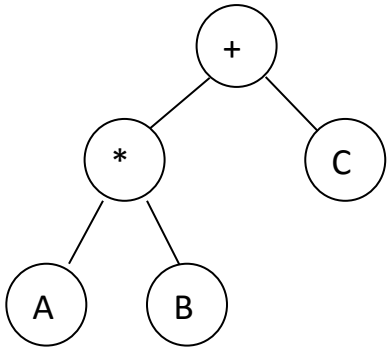
Example: Draw an expression trees for the following expressions.

a. A + B
b. A+B*C
c. A*B+C/D
d. A*(B+C)/D

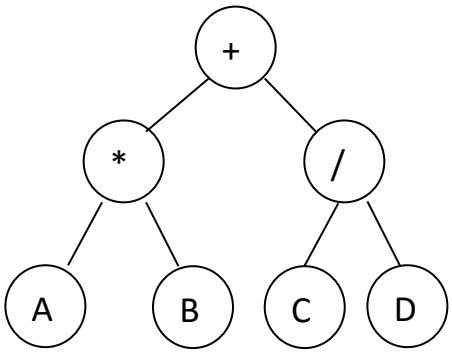
a. A+B



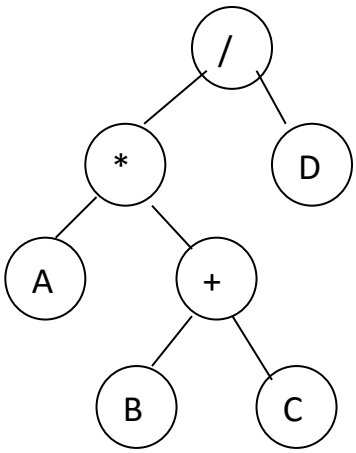
b. A+B*C



c. A*B+C/D

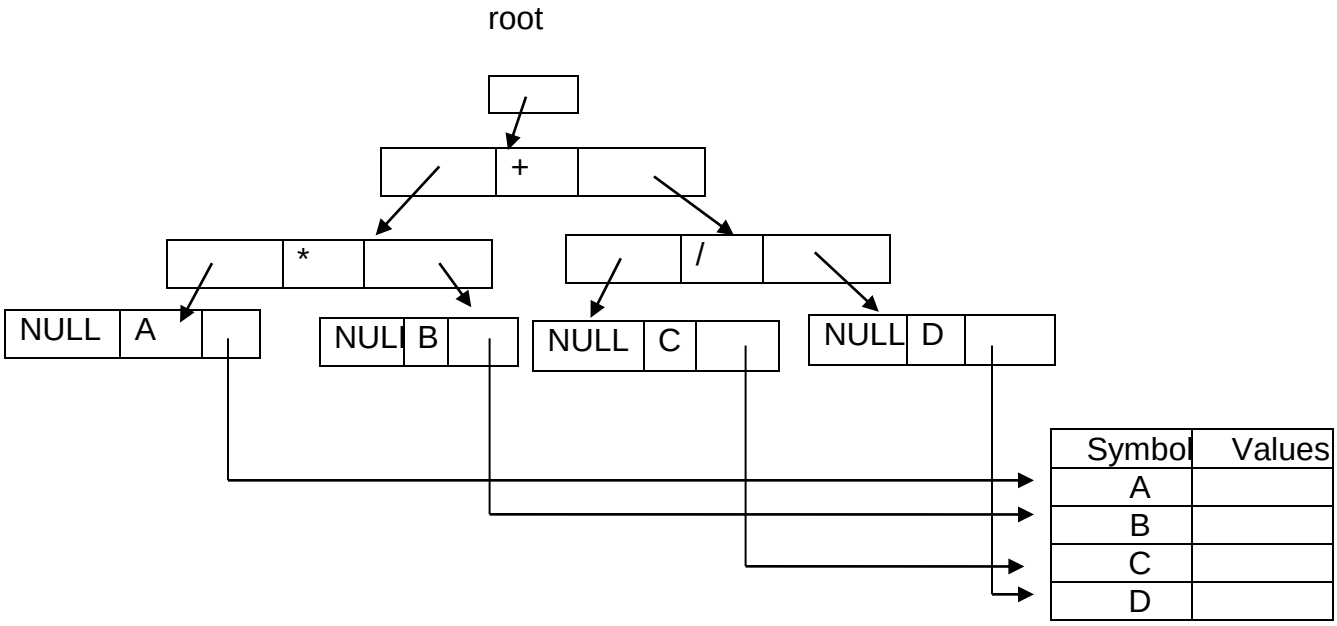


d. A*(B+C)/D



Manipulation of Arithmetic Expressions:

To manipulate arithmetic expression a symbol table is used for storing their value and types. Consider the expression A*B+C/D.A table is used to store values of each operand.



The following table is used to assign type to each operand.

Symbol	Type
Values or constant	0
+	1
-	2
*	3
/	4
^	5

The following procedure is used to evaluate arithmetic expression.

```
switch(type(E))
{
case 0:
        F=Rptr(E)
        return (value(F))
case 1:
        return (E.val(Lptr (E)) + E.val(Rptr (E)))
case 2:
        return (E.val(Lptr (E)) - E.val(Rptr (E)))
case 3:
        return (E.val(Lptr (E)) * E.val(Rptr (E)))
case 4:
        return (E.val(Lptr (E)) / E.val(Rptr (E)))
case 5:
        return (E.val(Lptr (E)) ^ E.val(Rptr (E)))
default:
        printf("Invalid expression");
}
```

In a binary tree representation of an expression, each leaf node's Rptr fields points to an entry in the symbol table. It has a value which may be either a variable or a constant. Each non leaf node has an operator. Here the left subtree of this non-leaf node has a left operand. the right subtree of this non-leaf node has a right operand.

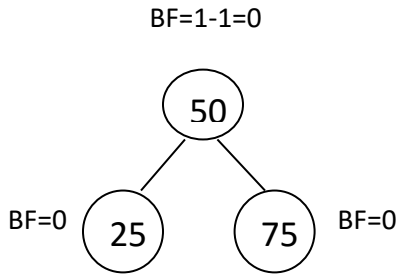
30. Height Balanced Trees (AVL Tree): Insertion

In order to have a worst case running time for insert and delete operations to be O(log n), we must make it impossible for there to be a very long path in the binary search tree.

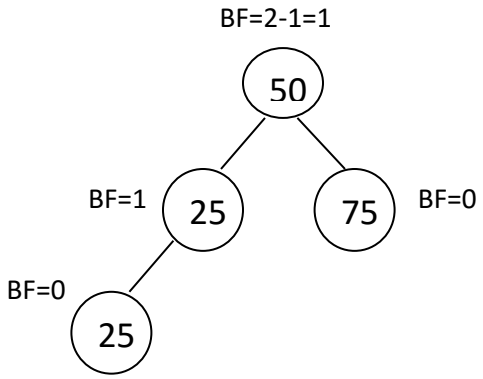
The first balanced binary tree is the AVL tree, named after it's inventors, Adelson-Velskii and Landis.

A binary search tree is an AVL tree if each node has balanced factor which can be 1, 0 or -1.The balanced factor value of a node is the difference between the height of its left subtree and its right subtree.

Example:



Example:



If the balance factor of a node is 1 then it is called left heavy. If the balance factor of a node is -1 then it is called right heavy. If a node is left heavy or right heavy then there is a chance of unbalance of the node if a new node is to be inserted as its successor.

.

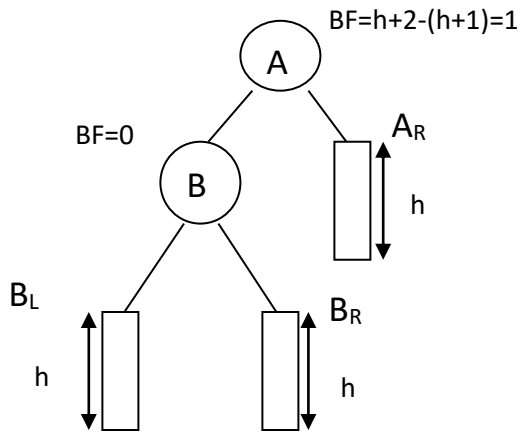
Insertion of a node to AVL tree:

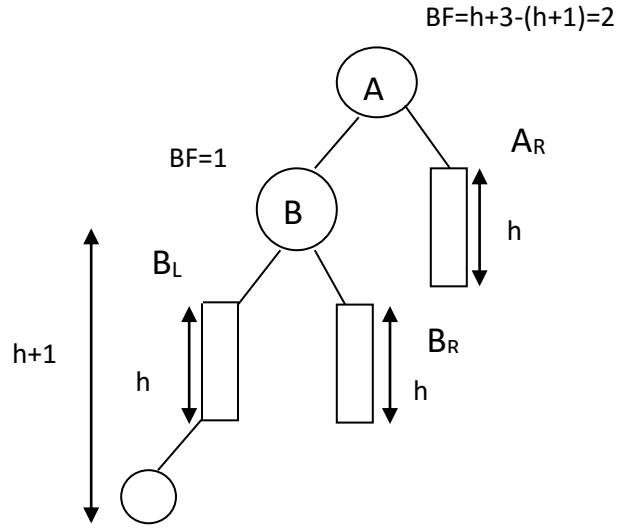
Inserting a node to AVL tree is similar to BST insertion, but we need to balance the tree if it becomes unbalance. There a different rebalancing technique as follows

i. LL rotation
ii. RR rotation
iii. LR rotation
iv. RL rotation

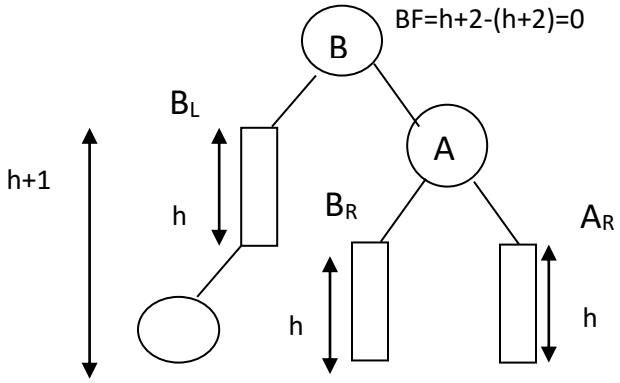
All these rotation are carried out around a pivot node which is the nearest unbalanced node from the newly inserted node.

LL rotation is applied to rebalance the pivot node when the new node is inserted at the left subtree of the left child of the pivot node. Let A is pivot node, A_R is the right subtree of A and B_L , B_R are the left and right subtree of B respectively.



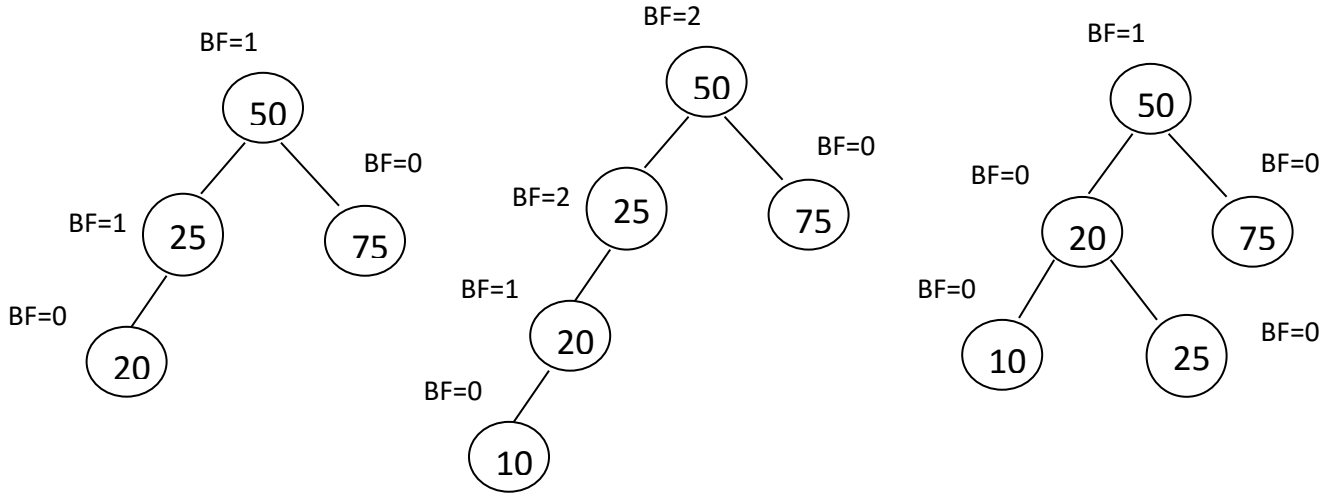


Before insertion to a left heavy node. After insertion to a left heavy node.



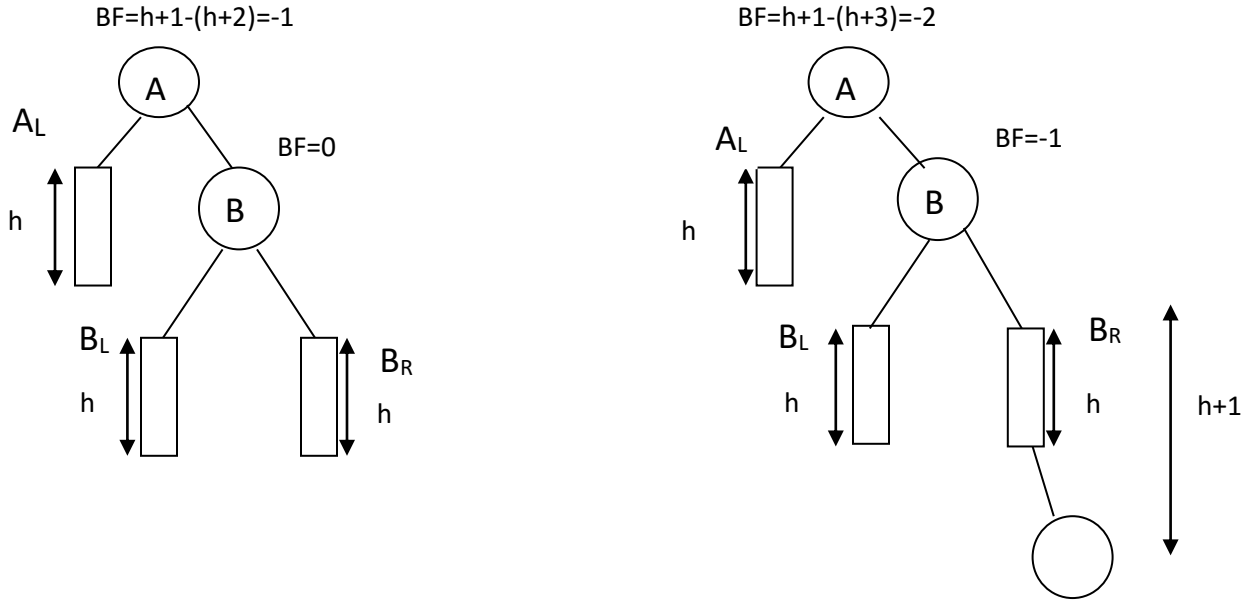
After Rebalance

Example:

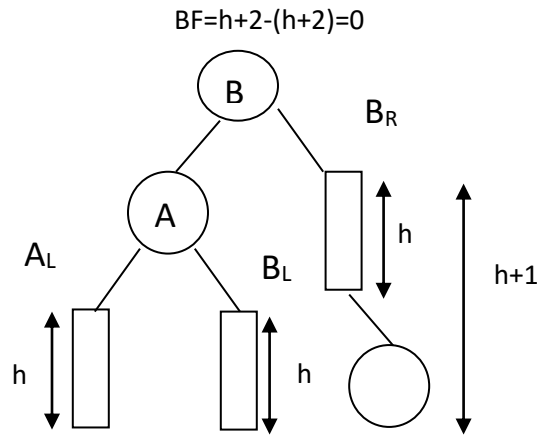


Before insertion After insertion After rebalance

RR rotation is applied to rebalance the pivot node when the new node is inserted at the right subtree of the right child of the pivot node. Let A is pivot node, A_R is the right subtree of A and B_L , B_R are the left and right subtree of B respectively

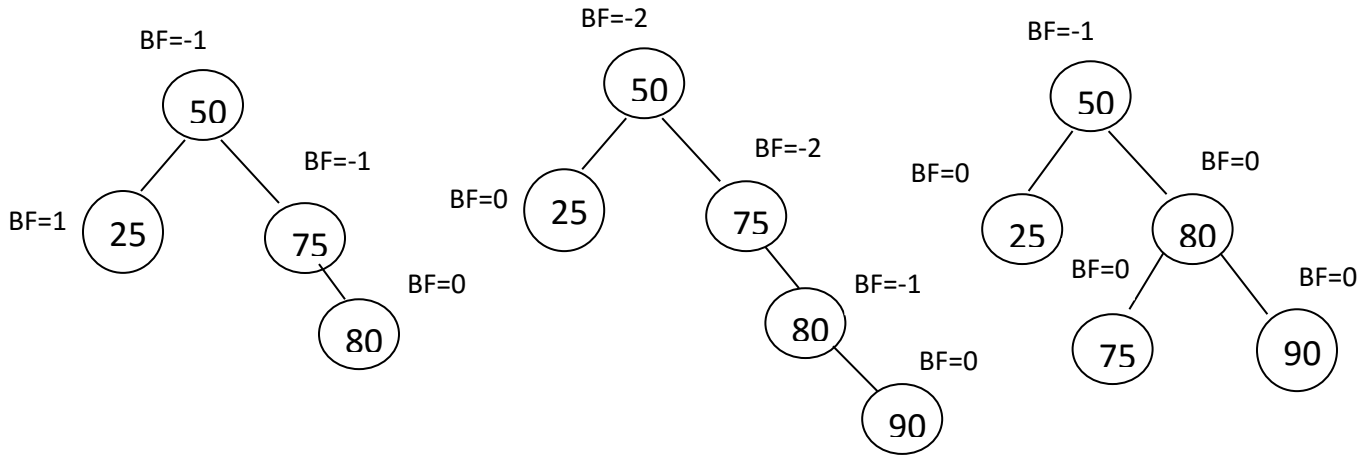


Before insertion to a right heavy node. After insertion to a rightheavy node.



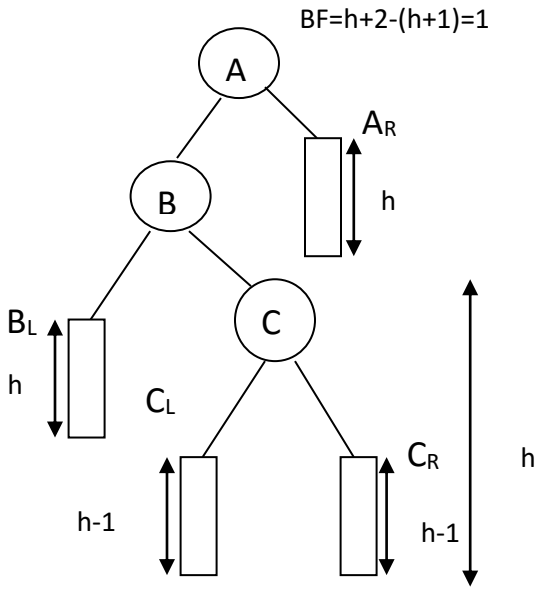
After Rebalance

Example:

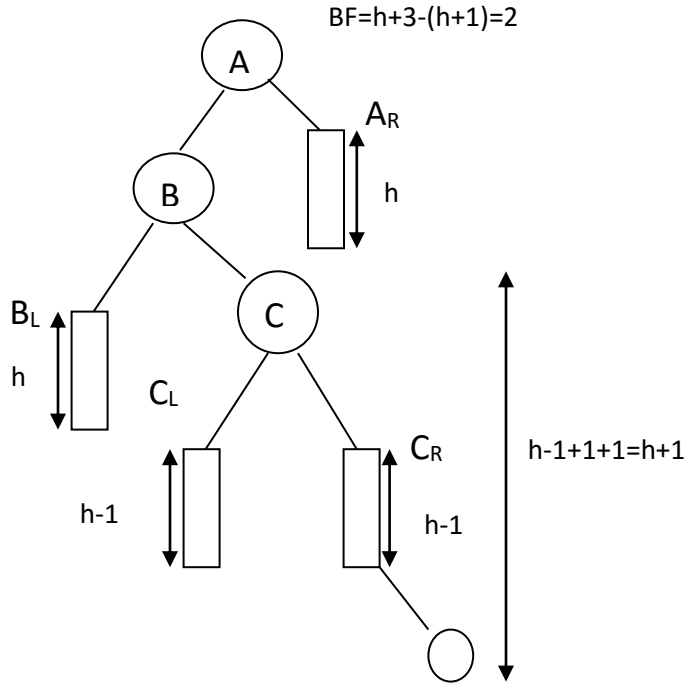


Before insertion After insertion After rebalance

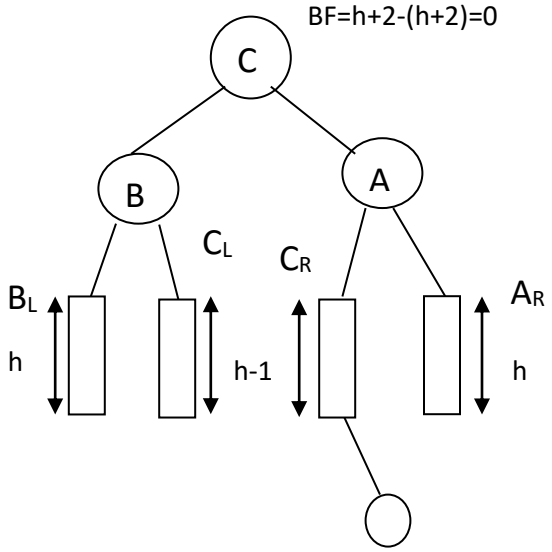
In a LR rotation rebalance is applied to the pivot node when the new node is inserted at the right subtree of the Left child of the pivot node. Let A is pivot node, A_R is the right subtree of A and B_L, B_R are the left of B and C is the right child of left child of pivot node A.



Before Insertion

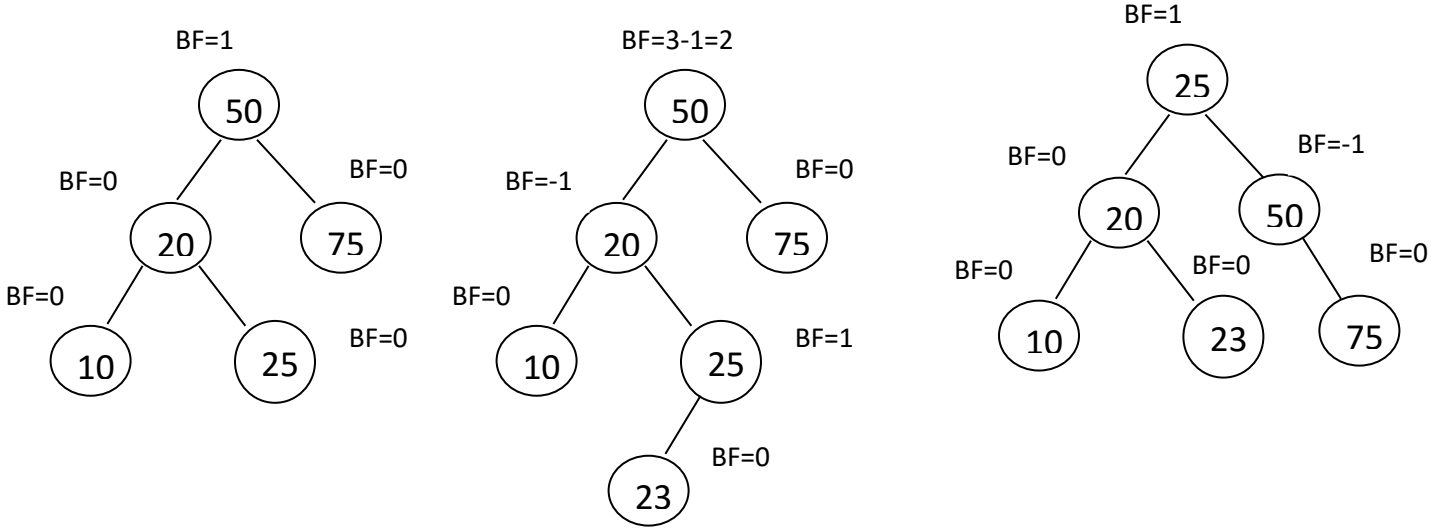


After Insertion



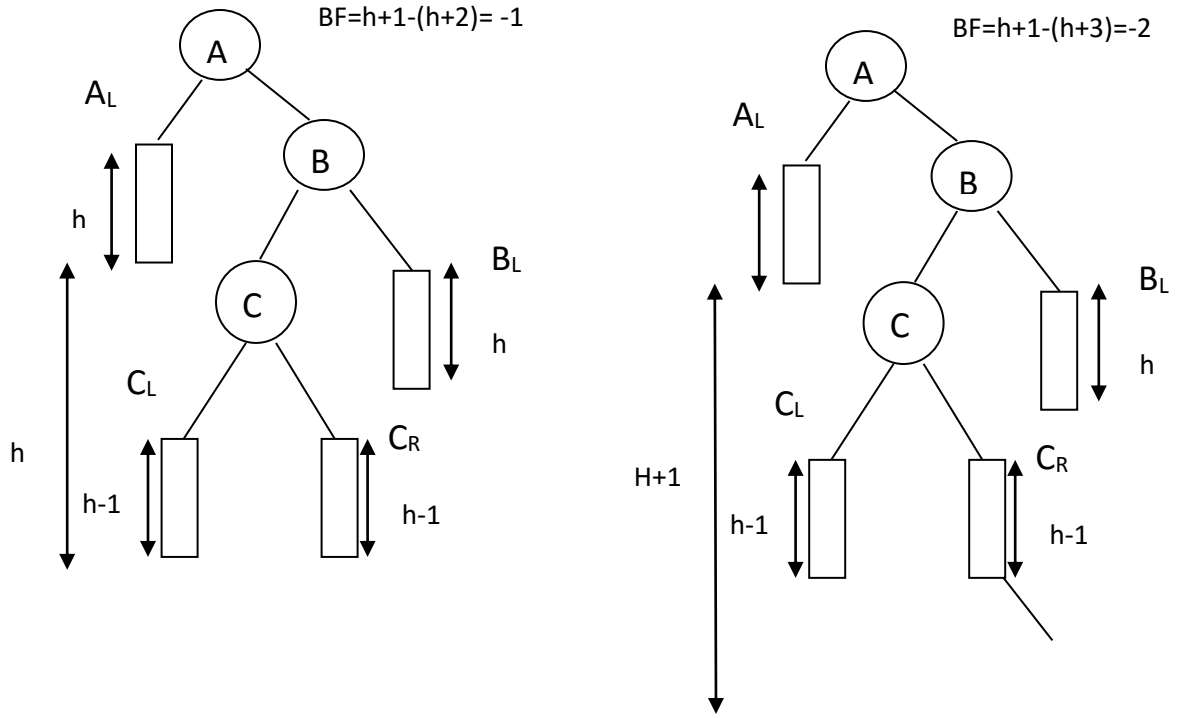
After rebalance

Example:



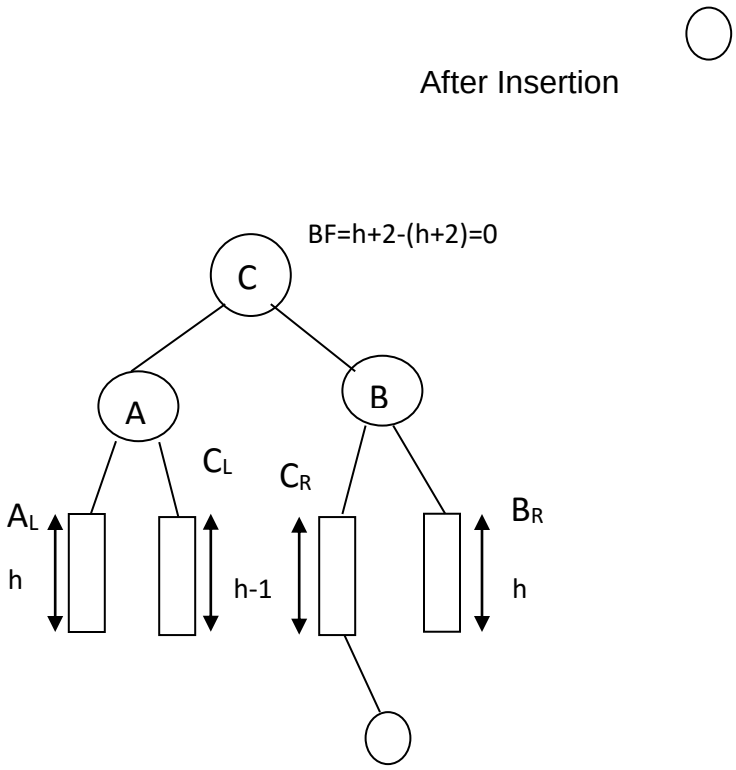
Before insertion After insertion After rebalance

In a RL rotation rebalance is applied to the pivot node when the new node is inserted at the left subtree of the right child of the pivot node. Let A is pivot node, A_L is the left subtree of A and B_L, B_R are the left of B and C is the left child of rghi child of pivot node A.



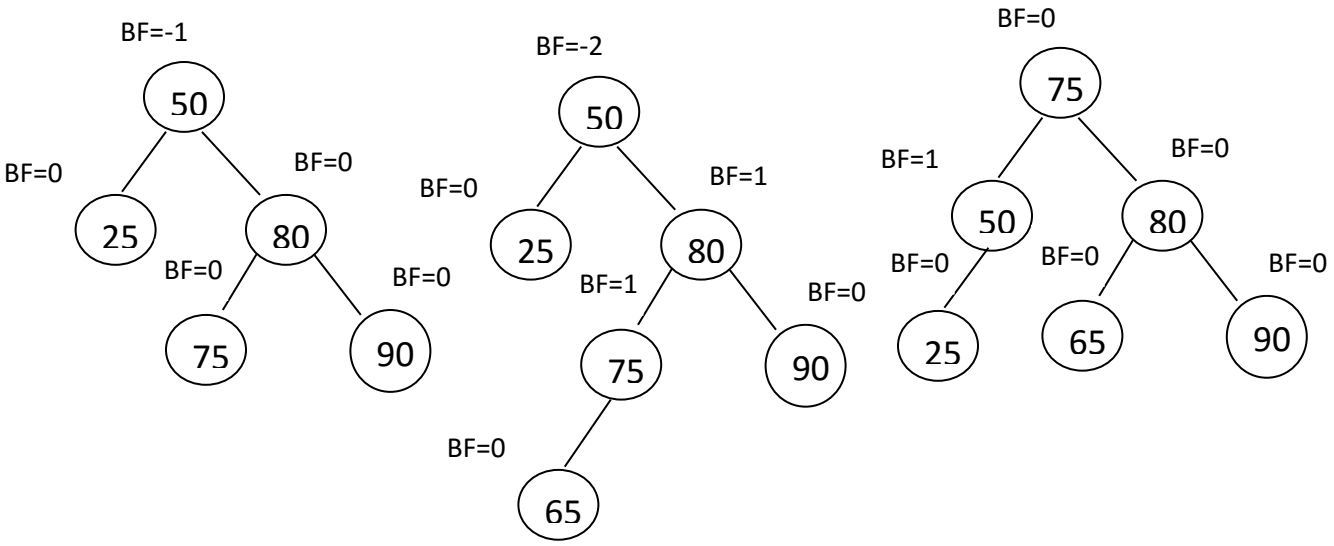
Before Insertion

After Insertion



After rebalance

Example:



Before insertion

After insertion

After rebalance

31. Height Balanced Trees (AVL Tree): Deletion

First we will do a normal binary search tree delete. If a delete causes a violation of the AVL Tree height property, this would have to restructure the tree by some node on the path from the parent of the deleted node to the root node.

One thing to note: whereas in an insert there is at most one node that needs to be unbalanced, there may be multiple nodes in the delete that need to be rebalanced.

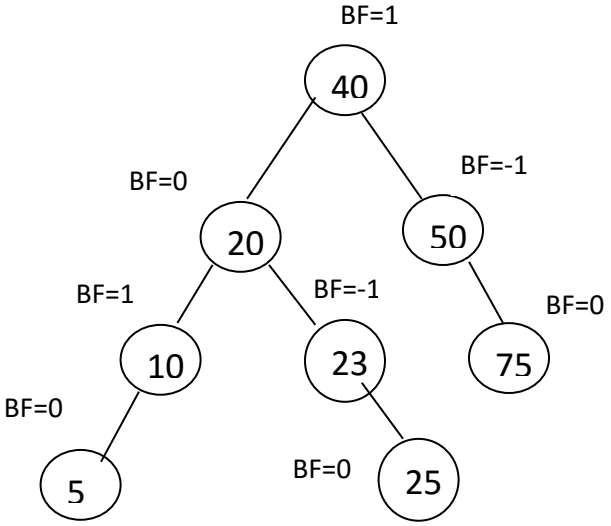
Deletion of a node from an AVL tree has the following two cases.

i. Deleting a node from right subtree of pivot node
ii. Deleting a node from left subtree of node

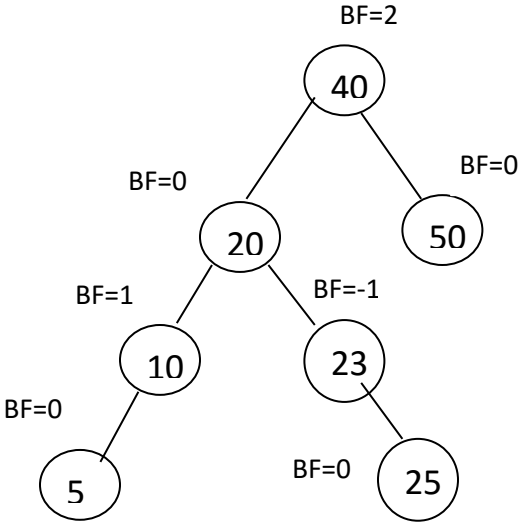
If we delete a node from the right subtree of pivot node and the balance factor of a left child of pivot node is 0 or 1, then apply LL rotation. If it is -1 then apply LR rotation.

If we delete a node from the left subtree of pivot node and the balance factor of a right child of pivot node is 0 or 1, then apply RR rotation. If it is -1 tehn apply RL rotation.

Example

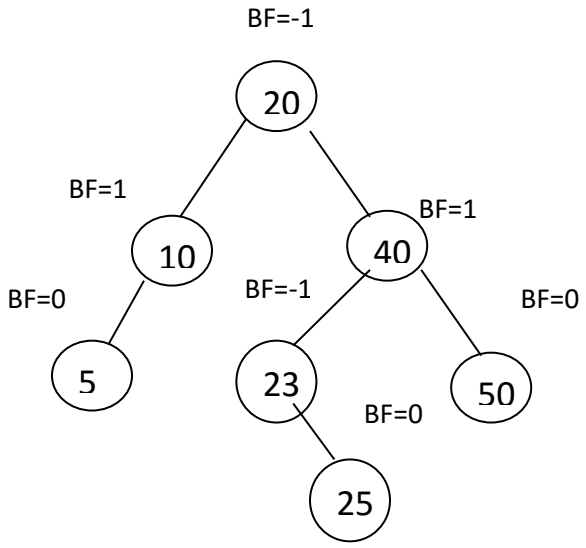


Node to Delete 75



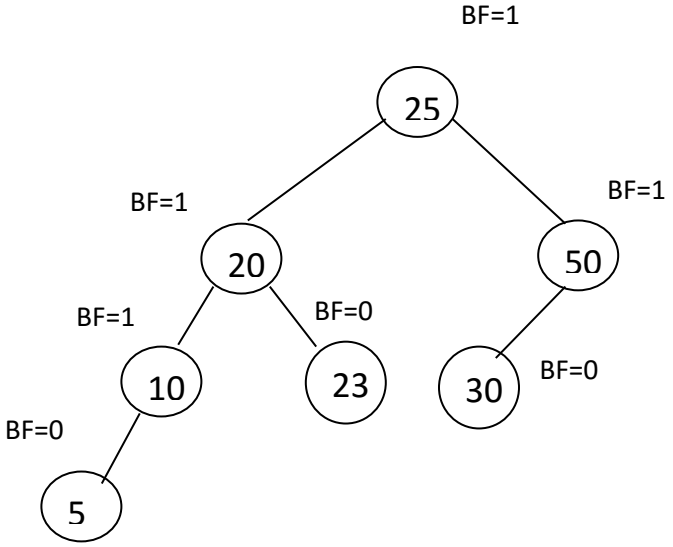
After deletion of 75

In the above figure after deletion of 75 pivot node is 25 , and we have deleted the node from its right sub tree . So we perform LL rotation as the balance factor of a left child of pivot node is 0.

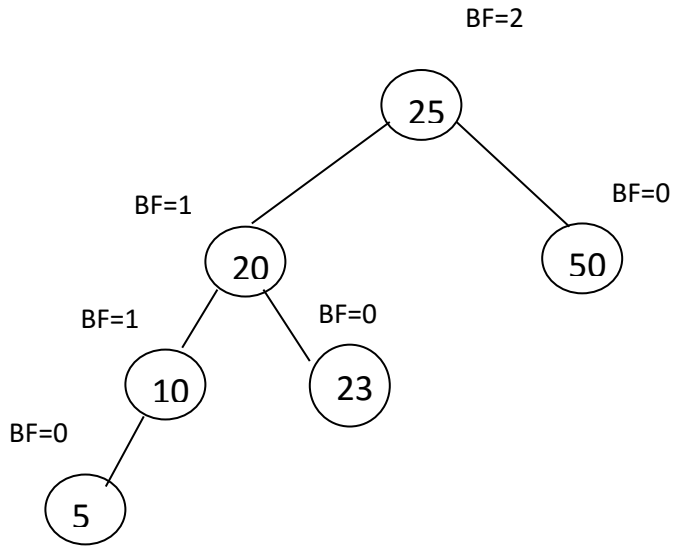


After LL rotation

Example

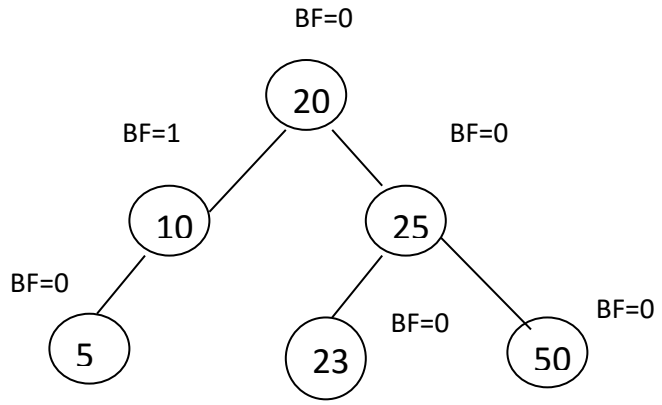


Node to Delete 30



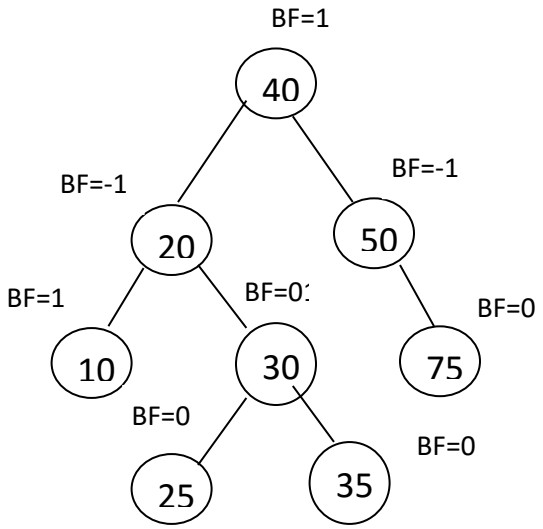
After deletion of 30

We have deleted the node from its right sub tree we checked the balance factor of the left child of pivot node 25 and found 1 for which we will apply LL rotation.

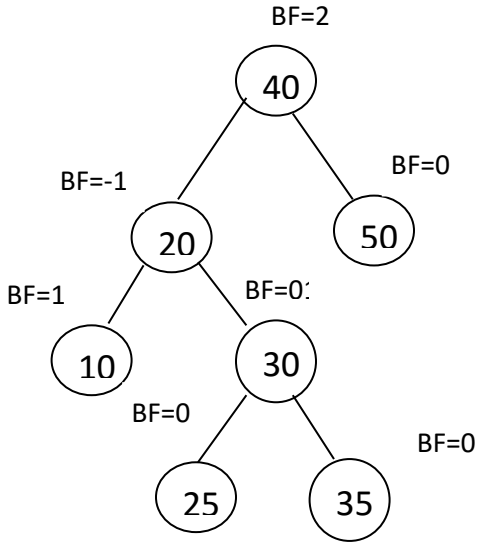


After LL rotation

Example

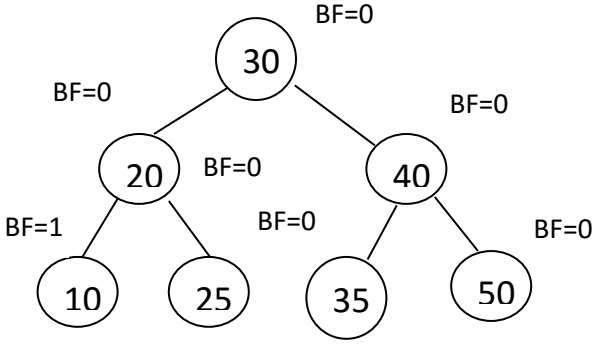


Node to Delete 75



After deletion of 75

As the balance factor of the left child of pivot node 25 is - 1 so which we will apply LR rotation.



After LL rotation

32. M-Way Search Tree

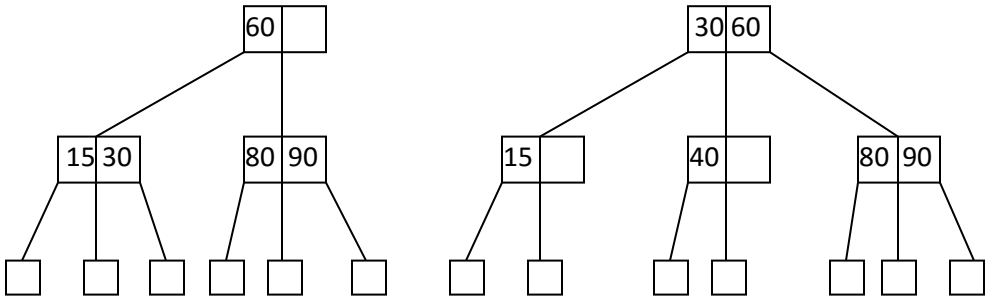
A m-way search tree is a tree in which

a. The nodes hold between 1 to m-1 distinct keys
b. The keys in each node are sorted
c. A node with k values has k+1 subtrees, where the subtrees may be empty.
d. The i'th subtree of a node $[v_1, ..., v_k]$, $0 \leq i \leq k$, may hold only values v in the range $v_i \leq v \leq v_{i+1}$.

1. 2-3 Trees

A 2-3 tree is a search tree that is either empty tree or has the following properties :

(1) Each internal node is either a 2-node or a 3-node. A 2-node has one element while a 3-node has two elements.
(2) Let left-child and middle-child denote the children of a 2-node. Let data_l be the element in this node and let data_l.key be its key.
   (a) All elements in the 2-3 subtree with root left-child have key less than data_l.key.
   (b) All element in the 2-3 subtree with root middle-child have key greater than data_l.key.
(3) Let left-child, middle-child, and right-child denote the children of a 3-node. Let data_l and data_r be the two elements in this node. Then, data_l.key is less than data_r.key;
   (a) All keys in the 2-3 subtree with root left-child are less than data_l.key.
   (b) All keys in the 2-3 subtree with root middle-child are less than data_r.key and greater than data_l.key; and
   (c) all keys in the 2-3 subtree with root right-child are greater than data_r.key.
(4) All external nodes are at the same level



40 inserted

2. 2-3-4 Trees

A 2-3-4 tree extends a 2-3 tree so that 4-nodes may have up to four children. A 2-3-4 tree is a search tree that is either empty or satisfies the following properties:

(1) Each internal node is a 2-, 3-, or 4-node. A 2-node has one element, a 3-node has two elements, and a 4-node has three elements.
(2) …
(3) All external nodes are at the same level.

3. Red-Black Trees

A red-black tree is a binary tree representation of a 2-3-4 tree. The child pointers of a node in a red-black tree are of two types: red and black. If the child pointer was present in the original 2-3-4 tree, it is a black pointer. Otherwise, it is a red pointer.

(1) A red-black tree is a binary search tree
(2) Every root to external node path has the same number of black links.
(3) No root to external node path has two or more consecutive red pointers.

4. B-Trees

- Multi-way (m-way) search trees
- AVL trees are 2-way search trees, 2-3 trees are 3-way search trees, and 2-3-4 trees are 4-way search trees. On the contrary, they are not. Such as, 2-way search trees are not AVL trees.
- B-tree Definition : A B-tree of order m is a m-way tree that is either empty or has the following properties:
  (a) The root node has at least 2 children.
  (b) All nodes other than the root node and failure nodes have at least m/2 children.
  (c) All failure nodes are at the same level.

33. B Tree

It is a balanced multi way search tree. A B tree has an order which satisfies / specifies the no of children of node in the tree.

A B tree having order m should satisfy the following properties:

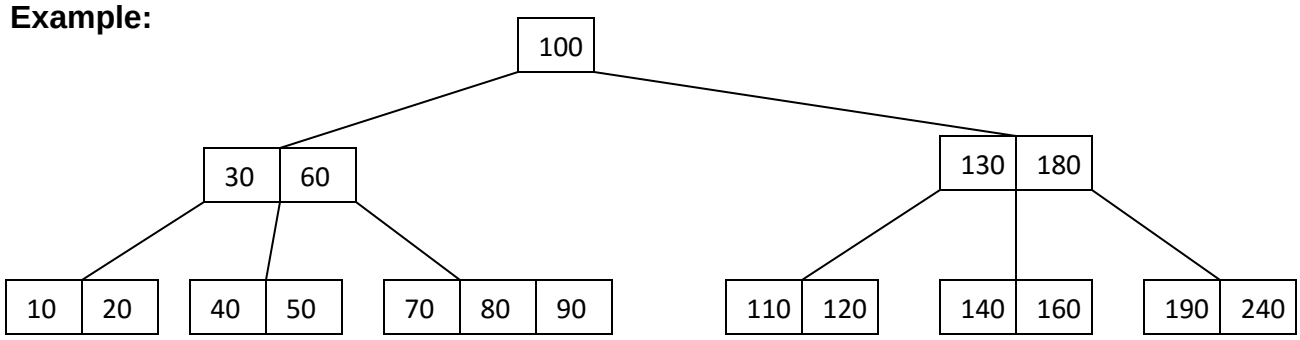
i. The root node has at least two children and at most m child.

ii. The internal nodes should have at least $\lceil m/2 \rceil$ children and at most m child.

iii. The no of data or keys in each internal node is one less than the no of child and keys partitioned the data in left and right subtree.

iv. All leaves should be at same level.

Example:



B tree of order 3.

For an order 2 B tree the root should have minimum one key and maximum three keys and all nodes should have minimum two keys and maximum three keys.

Insertion into a B tree:

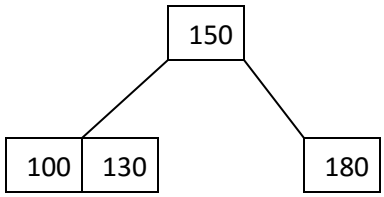
A key in a B Tree is inserted at leaf nodes only. Initial key of a B tree is the root node which a leaf also.

If a node contains less than maximum number of keys m-1, then simply insert the key to the node in order otherwise split the node into two nodes at median by moving up the median to its parent node. While moving up the median key to parent node, if it is full then spilt it. The splitting process can propagate to the root node.

Example: For an order 4 B tree, a node can have maximum 4 keys when we try to insert a new key to it has to split.

100	150	180

Let insert 130 to this node, as the node is filled, we split is at median key which is 150.



After splitting

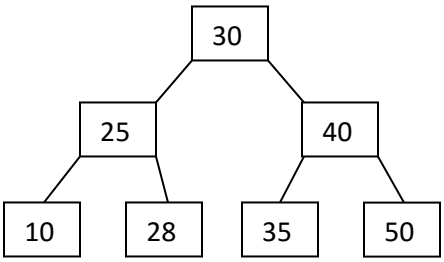
Deletion of a key from a B tree:

A key can be delete from any node of a b Tree unlike insertion. If a node has more than $\lceil m/2 \rceil$-1 key, then simple delete the key. If a node has exactly $\lceil m/2 \rceil$-1 key then deleting a key from the node requires borrowing of keys from siblings. If all the siblings have exactly $\lceil m/2 \rceil$-1 key then it requires merging of nodes. The process of merging can propagate to root.

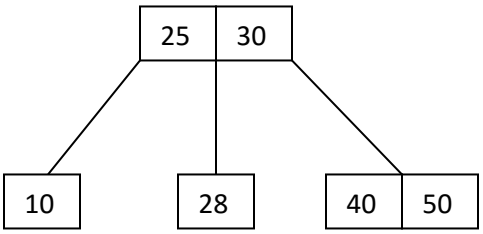
Example:

Let take an order 3 B tree, in which all nodes can have minimum 1 key and maximum two keys.

Delete 35.



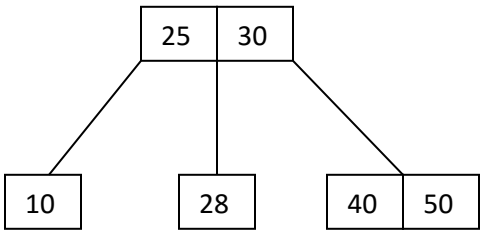
Before deletion

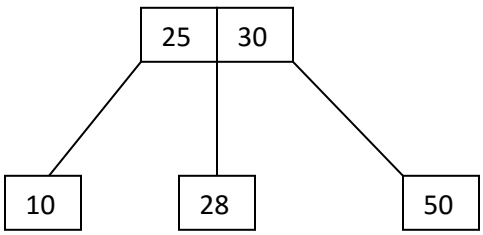


After deletion

In the above example while deleting 35, there are minimum keys in its sibling so it needs merge with its parent key. As there are minimum no of key in its parent, so it need to merge with parents also.

Delete 40.

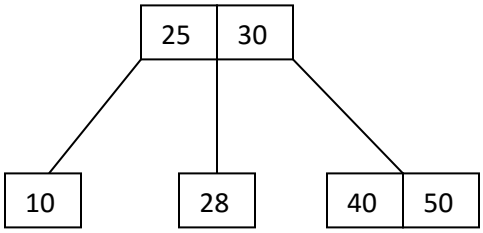


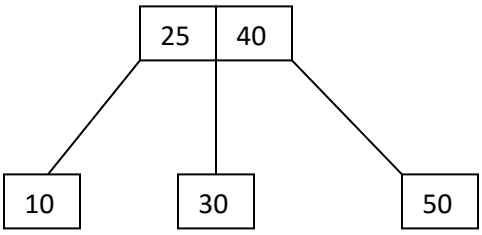


Before deletion After deletion

In the above figure as there are more than minimum no of keys in the node ,simple the key is deleted without any borrow and merge.

Delete 28





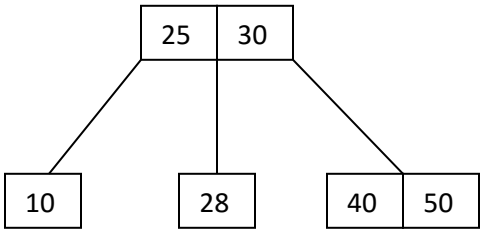
Before deletion After deletion

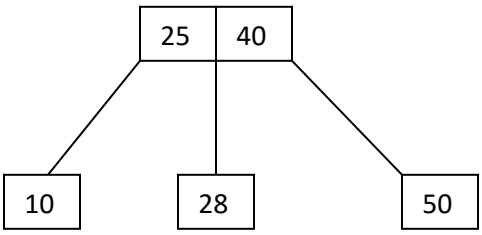
As there are more than the number of keys in the sibling of 28, a key is borrowed to its parent key by moving down its parent to its place.

Suppose a node has to delete from a non leaf node borrowing of keys done from children having more than minimum keys otherwise its children will be merged.

Example:

Delete 30

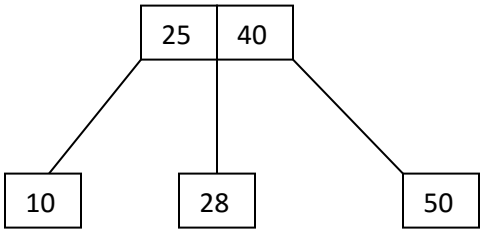




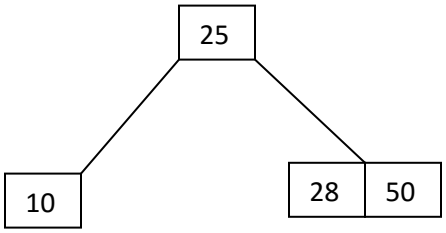
Before deletion After deletion

In case deleting 30 from a non leaf node, we found that there are more than minimum keys from its children. So we have borrowed / moved up a key from it. If there are exactly minimum numbers of children then we merge them as follows.

Delete 40



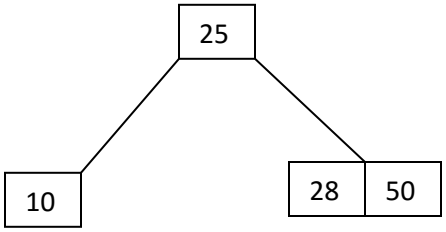
Before deletion



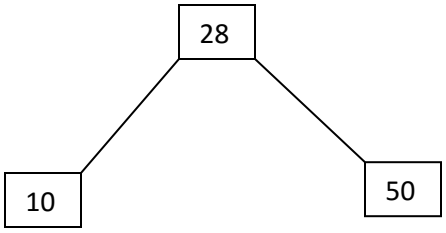
After deletion

If there are minimum number of keys in the non leaf node and we have to delete a key from it then we have either borrow or merge as before.

Delete 25

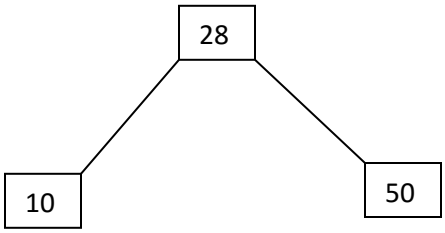


Before deletion

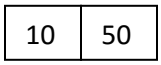


After deletion

Delete 28



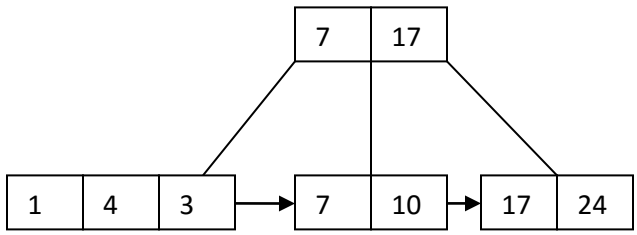
Before deletion



After deletion

34. B^+ Tree

It is a variation of B Tree .In a B^+ Tree all keys are mentioned in the leaf level which are also in the non leaf node. The duplicate key in the non leaf node defines the path to reach the key. Unlike B tree all Leaf nodes are linked together for providing a sequential path.



B^+ Tree of order 4

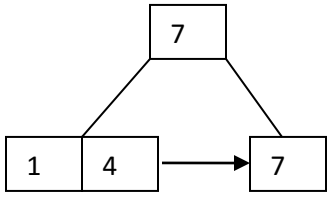
Insertion in B^+ Tree:

It is similar to B Tree Insertion but when a new node splits the median remains in both parent node and any one child node. The splitting process can propagate to root.

Example:

In an order 4 B Tree there can be maximum 3 keys and minimum 2 keys should be present in a node but root node contains minimum 1 key.

Before splitting

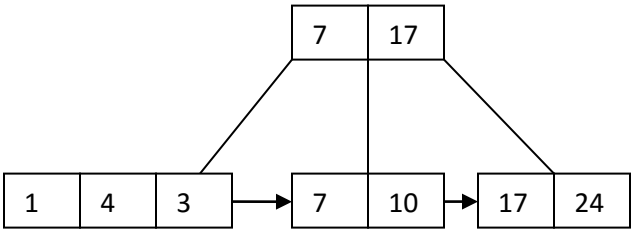


After splitting

Deletion in B^+ Tree:

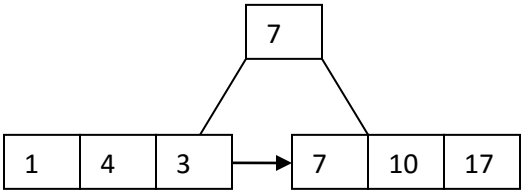
Deletion is similar to B tree. But when a key is to be deleted from a leaf appears at non leaf then changes that key also to the key which is replaced for the deleted key. Like B tree, if we are deleting a key then either borrow or merge the keys. The merging process can propagate to root.

Example:



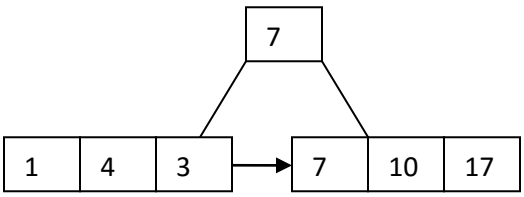
Before deletion

It is an order 4 B^+ Tree if we delete 24 then we are shortage of keys in that node. Borrowing of keys fails as there are shortage of keys in sibling and point. So we merge it with its parent. After merging the figure is as follows.

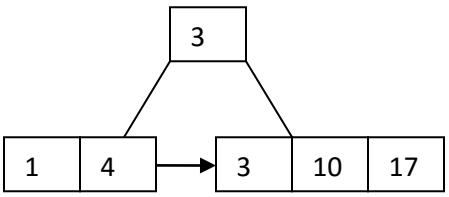


After deletion

If we delete 7 which is there at both parent and leaf node. Then we have o delete from both the places and borrow key from siblings to those place if available otherwise it needs to merge.



Before deletion



After deletion

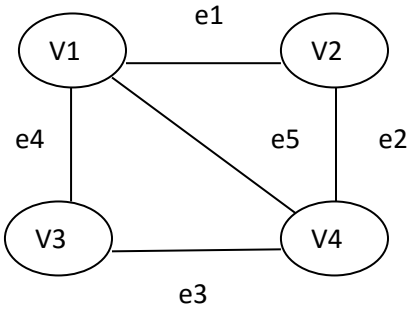
35. Graph: Definition, Terminology

Graph:

A graph G consist of two sets, a set V of elements called as nodes or vertices and a set E of edges such that each edge e Є E is identical with unique pair [u,v] of nodes in V and is represented as e=[u,v] where the Graph is represented as G=(V,E).

The vertex set in the figure is V(G)= { v1, v2, v3, v4 }

The edge set in the figure is E(G)= { e1 , e2 , e3 , e4, e5 }



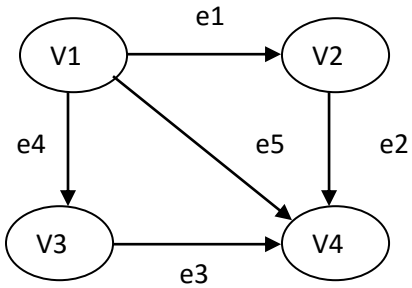
Undirected Graph

Graph Terminology:

Undirected Graph:

A Graph is called undirected if no edge has a direction.

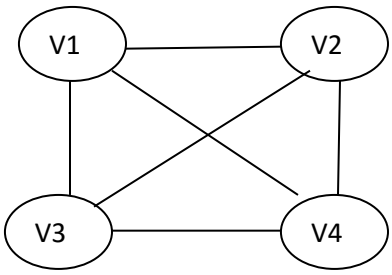
Directed Graph: A Graph is called directed Graph if every edge has a direction.



Directed Graph

Weighted Graph:

A Graph is called weighted Graph if every edge has assigned a numerical value called weight or cost of the edge.



Complete Graph

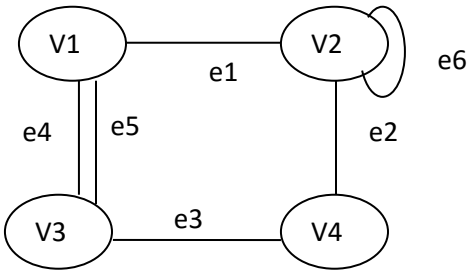
Adjacent Nodes: A node is adjacent to another if they have a common edge.

Incident Nodes: If (u,v) is an edge then it is incident on u and v vertex which are adjacent to each other.

Complete Graph: A Graph is said to be complete if every node is adjacent to every other node.

Loop - An edge connecting a vertex to itself. In the multi graph figure e6 is a loop edge.

Multi-edges – If there are parallel edges between two nodes, Then it is called multi-edge. In the multi graph figure e4 and e5 are multi-edge.



Multi graph

Simple Graph - Graph with no loops or multi-edges.

Degree of a node: The number of edges connected to it.

Degree of a Directed Graph:

There are two types of degree of a directed Graph.

i. In-degree: The number of edges connected towards the node
ii. Out-degree: The number of edges out from the node.

In the above directed graph the in-degree and out-degree is as follows.

Vertex	In-degree	Out-degree
V1	**0**	**3**
V2	**1**	**1**
V3	**1**	**1**
V4	**3**	**0**

Regular Graph: A graph is said to be regular if degree of every vertex is same.

Walk – A walk is a sequence of vertices and edges like $v_1\ e_1\ v_2\ \ldots\ e_{n-1}\ v_n$.

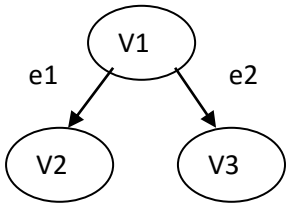
Trail - It is a walk with distinct edges (not include same edge twice, vertex may repeat).

Path - It is a trail with distinct vertices (not include any vertex and edge twice).

Cycle: It is a path that begins and ends on the same vertex.

Circuit: It is a trail that begins and ends on the same vertex.

Acyclic Graph: A graph with no cycle.



Tree- A directed acyclic Graph

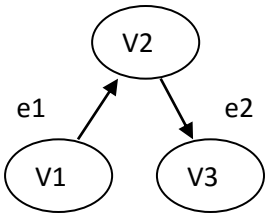
Tree

Connected Graph -A graph where there exists a path between any two vertices.

Component of a Graph- It is a maximal connected sub-graph .

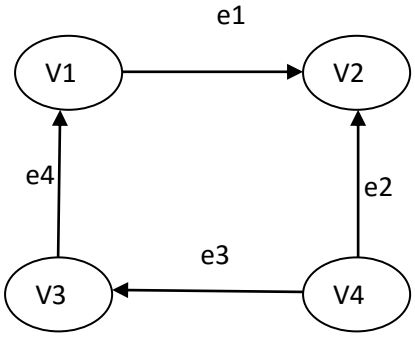
Connectedness in a Directed Graph: There are three types of Connectedness in a directed Graph.

Weakly connected: A Directed Graph is called weakly connected, if there is a path between every pair of vertices after removing the direction of its edges.



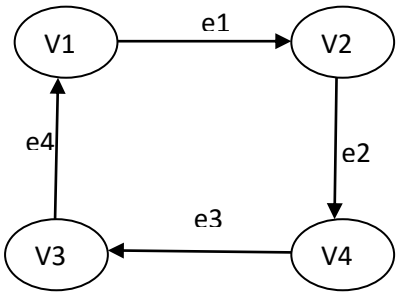
Weakly Connected Graph

Unilaterally connected: A Directed Graph is unilaterally connected if there is a path either from source to destination or from destination to source.



Unilaterally connected Graph

Strongly connected: A Directed Graph is strongly connected if there is a path between every pair of vertices both from source to destination or from destination to source.



Strongly connected Graph

36. Representation of Graph: Adjacency Matrix, Incidence Matrix, path Matrix

Representation of Graph: A is graph is represented in two ways as follows.

i. Adjacency matrix
ii. Adjacency list

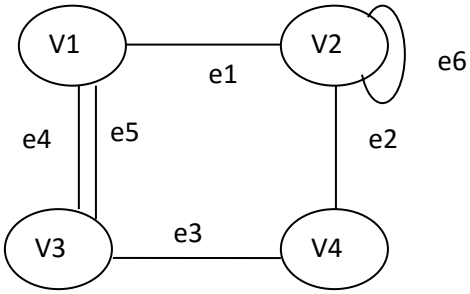
Adjacency Matrix:

Let G=(V,E) be a graph with n vertices, n>=1.The adjacency matrix of G is a two dimensional n x n array or matrix.

Mat [i] [j] = 1, If there is an edge between v_i and v_j

Mat [i] [j] = 0 , If there is no edge between v_i and v_j

The adjacency matrix for an undirected graph is symmetric; the adjacency matrix for a digraph need not be symmetric.

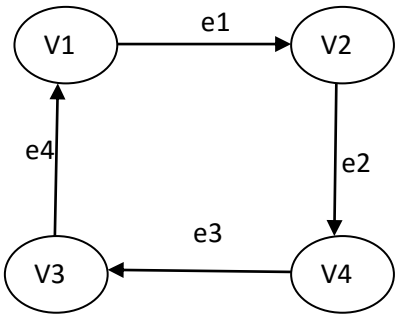


Undirected graph

Mat [4] [4] =

$$\begin{array}{c|cccc} & V1 & V2 & V3 & V4 \\ \hline V1 & 0 & 1 & 1 & 0 \\ V2 & 1 & 1 & 0 & 1 \\ V3 & 1 & 0 & 0 & 1 \\ V4 & 0 & 1 & 1 & 0 \end{array}$$

Adjacency Matrix



Directed graph

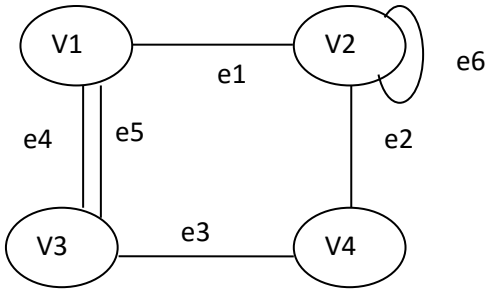
Mat [4] [4] =

$$\begin{array}{c|cccc} & V1 & V2 & V3 & V4 \\ \hline V1 & 0 & 1 & 0 & 0 \\ V2 & 0 & 0 & 0 & 1 \\ V3 & 1 & 0 & 0 & 0 \\ V4 & 0 & 0 & 1 & 0 \end{array}$$

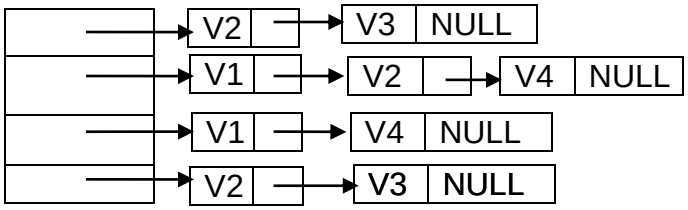
Adjacency Matrix

Adjacency List:

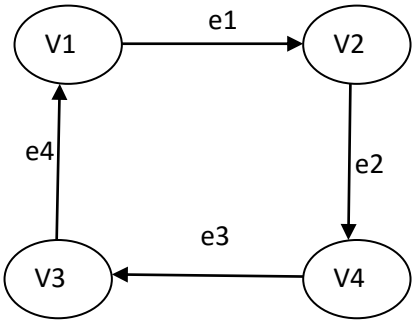
In this representation of graph, there are n chains for n vertices of the graph G. Each chain contains the list of adjacent nodes of the corresponding vertex. An array of adjacent list is used so that we can access any adjacency list in O (1) times.



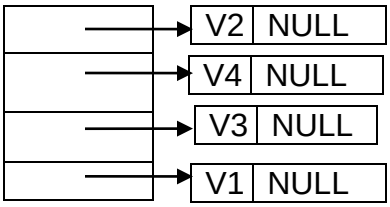
Undirected graph



Adjacency List



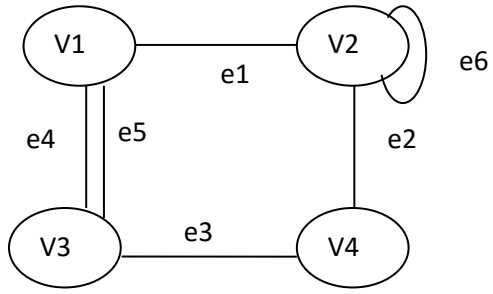
Directed graph



Adjacency List

Incidence Matrix

The incidence matrix of an undirected graph gives the (0 , 1)-matrix which has a row for each vertex and column for each edge, and (v, e) = 1 iff vertex v is incident upon edge e. The size f the matrix is v x e.

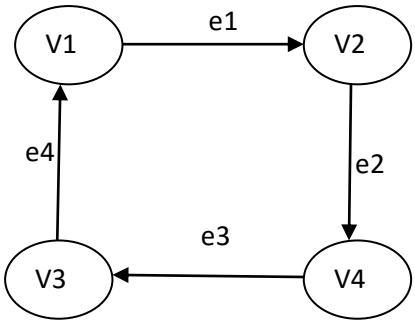


Undirected graph

$$\begin{array}{c} \\ V1 \\ V2 \\ V3 \\ V4 \end{array} \begin{array}{c} \begin{array}{cccccc} e1 & e2 & e3 & e4 & e5 & e6 \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

incidence matrix

The incidence matrix of a directed graph gives the (0 , 1)-matrix which has a row for each vertex and column for each edge, and (v , e) = 1 iff vertex v is incident to (in) edge e and (v , e) = -1 iff vertex v is incident from (leaves) otherwise zero. The size f the matrix is v x e.



Directed graph

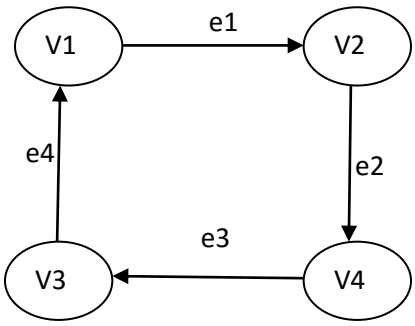
$$\begin{array}{c} \\ V1 \\ V2 \\ V3 \\ V4 \end{array} \begin{array}{c} \begin{array}{cccc} e1 & e2 & e3 & e4 \end{array} \\ \begin{pmatrix} -1 & 0 & 0 & 1 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 1 & -1 & 0 \end{pmatrix} \end{array}$$

incidence matrix

Path Matrix:

As we know that a path is a sequence of distinct vertices (not include any vertex and edge twice) from a source node to a destination node.

Path Matrix is a V X V matrix which contains 1 if there is a path between nodes else zero.



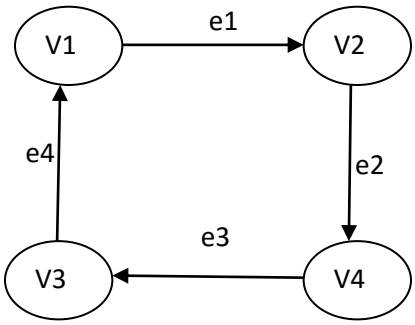
P[4][4]=

$$\begin{array}{c} \\ V1 \\ V2 \\ V3 \\ V4 \end{array} \begin{array}{c} \begin{array}{cccc} V1 & V2 & V3 & V4 \end{array} \\ \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \end{array}$$

Directed graph

path matrix

In some books, the path represents zero form if source node and destination node is same as follows.



P[4][4]=

$$\begin{array}{c} \\ V1 \\ V2 \\ V3 \\ V4 \end{array} \begin{array}{c} \begin{array}{cccc} V1 & V2 & V3 & V4 \end{array} \\ \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \end{array}$$

Directed graph

path matrix bi considering no path for same vertex

37. Graph Traversal Algorithm: Breadth first Search (BFS)

Graph Traversal:

It is used to find whether a node is reachable from another node. Traversal of a graph is not unique. There are two ways of graph traversal.

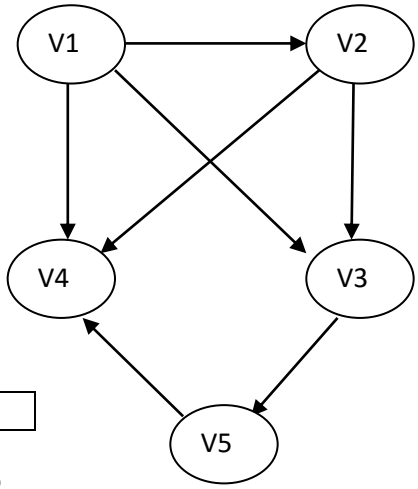
i. Breadth First search
ii. Depth First Search

Breadth First search

It uses queue data structure to traverse / visit the adjacent nodes of the graph level / breadth wise. The queue is used to keep track of the unvisited adjacent nodes. A Boolean array which contains 0's and I's only is used to keep track of visited nodes. The size of the array is equal to the no of nodes in the graph.

Algorithm:

i. Take any node as the starting node.
ii. Insert the starting node in the queue and mark it visited.
iii. Delete a node from the queue and insert all its unvisited adjacent nodes into the queue.
iv. Mark all these unvisited adjacent nodes as visited.
v. Repeat step III and IV until the queue is empty.



Example:

Initially the queue is empty and the Boolean array is filled with zero to represent that node is visited.

-1 0 1 2 3 4
R
F

Queue

0	**0**	**0**	**0**	**0**

v1 v2 v3 v4 v5

Boolean Array

As we can start traversal from any node, let us take V1 as the starting node. So V1 is inserted in the queue and mark V1 as visited (1) in the Boolean array.

V1				

-1 0 1 2 3 4
R
F

Queue

1	**0**	**0**	**0**	**0**

v1 v2 v3 v4 v5

Boolean Array

Next step is to delete the node V1 and insert all its unvisited adjacent nodes V1, V2, V3 in the queue and mark them visited. The insertion of the node can be done in any order.

V2	**V3**	**V4**		

-1	**0**	**1**	**2**	**3**	**4**
	F		**R**		

Queue

1	**1**	**1**	**1**	**0**
v1	**v2**	**v3**	**v4**	**v5**

Boolean Array

Similarly, v2 is deleted but it doesn't have any unvisited adjacent node.

	V3	**V4**		

-1	**0**	**1**	**2**	**3**	**4**
		F	**R**		

Queue

1	**1**	**1**	**1**	**0**
v1	**v2**	**v3**	**v4**	**v5**

Boolean Array

Now V3 is in the front and it has one unvisited node V5.So v3 is deleted and V5 is inserted and v5 is marked visited.

		V4	**V5**	

-1	**0**	**1**	**2**	**3**	**4**
			F	**R**	

Queue

1	**1**	**1**	**1**	**1**
v1	**v2**	**v3**	**v4**	**v5**

Boolean Array

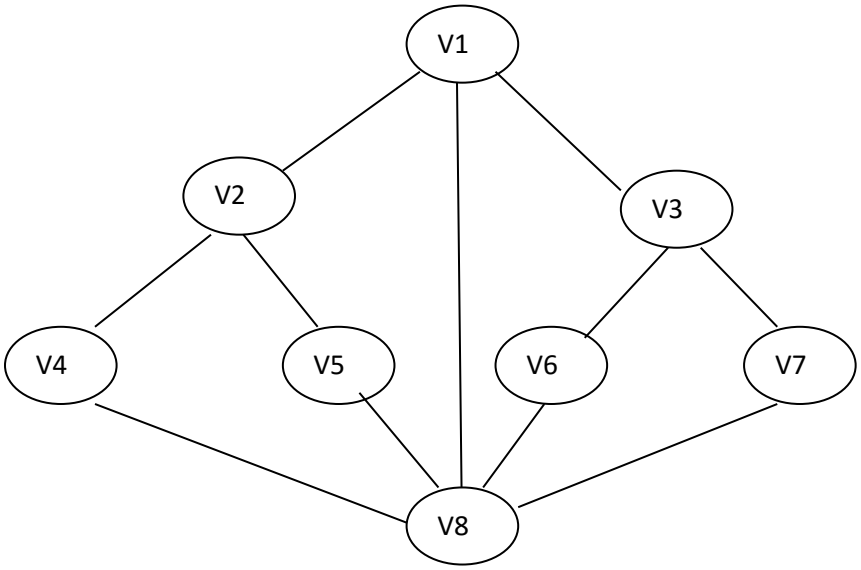
As V4 and V5 have no unvisited node, they will be simple delete. The traversal of the graph the order they are delete. Hence V1 V2 V3 V4 V5 is one of the breadths first search traversal sequence.

38. Graph Traversal Algorithm: Depth first search (DFS)

It visits all the nodes depth wise or path wise. It uses stack data structure which is used to keep track of adjacent nodes. A Boolean array is used to keep tracks of the unvisited nodes. If the node is visited it marks 1 otherwise 0.

Algorithm:

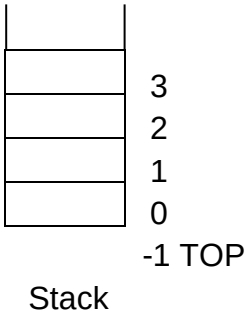
i. Take any node as the starting node.
ii. Push the starting node into the stack.
iii. Pop an element from the stack.
iv. If it is visited ,go to step III else mark it as visited and push the adjacent nodes of the popped element to the stack
v. Repeat step III to V until the stack is empty.



Example:

Let V1 be the starting node and the stack is empty initially. The Boolean array is assigned to zero which represent that none of the node is visited.

Initially the scenario is as follows.



Stack

0	0	0	0	0	0	0	0
v1	v2	v3	v4	v5	v6	v7	v8

Boolean Array

Push start node V1 to the stack,

	3
	2
	1
V1	0 TOP
	-1

Stack

0	0	0	0	0	0	0	0
v1	v2	v3	v4	v5	v6	v7	v8

Boolean Array

Pop an element, mark it as visited and push all its adjacent nodes V2 V3 V8 at any order.

	3
V3	2 TOP
V8	1
V2	0
	-1

Stack

1	0	0	0	0	0	0	0
v1	v2	v3	v4	v5	v6	v7	v8

Boolean Array

If we repeat the same process where v3 is popped and marked as visited and the scenario is as follows.

V7	4 TOP
V6	3
V8	2
V2	1
	0
	-1

Stack

1	0	1	0	0	0	0	0
v1	v2	v3	v4	v5	v6	v7	v8

Boolean Arrays

Now V7 is popped which is marked as 1 and its adjacent nodes v3, v8 are pushed to the stack now.

	5 TOP
V8	4
V3	3
V6	2
V8	1
V2	0

1	0	1	0	0	0	1	0
v1	v2	v3	v4	v5	v6	v7	v8

Boolean Arrays

Similarly V8 is popped and V4, v5, v6, v7 are pushed to the stack.V8 is marked as 1.

V7	7 TOP
V6	6
V5	5
V4	4
V3	3
V6	2
V8	1
V2	0

1	0	1	0	0	0	1	1
v1	v2	v3	v4	v5	v6	v7	v8

Boolean Arrays

Next V7 is popped. As it is visited already we pop the next one V6 and V3, V7, v8 are pushed to stack.

V8	8 TOP
V7	7
V3	6
V5	5
V4	4
V3	3
V6	2
V8	1
V2	0

1	0	1	0	0	1	1	1
v1	v2	v3	v4	v5	v6	v7	v8

Boolean Arrays

As v8, v7 and V3 are visited already, they are popped and ignored. So V5 is popped and marked as 1 and its adjacent nodes V2, V4, V8 are pushed.

V8	7 TOP
V4	6
V2	5
V4	4
V3	3
V6	2
V8	1
V2	0

1	0	1	0	1	1	1	1
v1	v2	v3	v4	v5	v6	v7	v8

Boolean Arrays

V8 is popped and ignored.V4 is popped and marked as 1 and its adjacent nodes V2, V8 are pushed to stack.

V8	7 TOP
V2	6
V2	5
V4	4
V3	3
V6	2
V8	1
V2	0

1	0	1	1	1	1	1	1
v1	v2	v3	v4	v5	v6	v7	v8

Boolean Arrays

Again V8 is popped and ignored. Then V2 is popped and marked as 1 and its adjacent nodes V4, V5, v1 are pushed to stack.

V1	8 TOP
V5	7
V4	6
V2	5
V4	4
V3	3
V6	2
V8	1
V2	0

1	1	1	1	1	1	1	1
v1	v2	v3	v4	v5	v6	v7	v8

Boolean Arrays

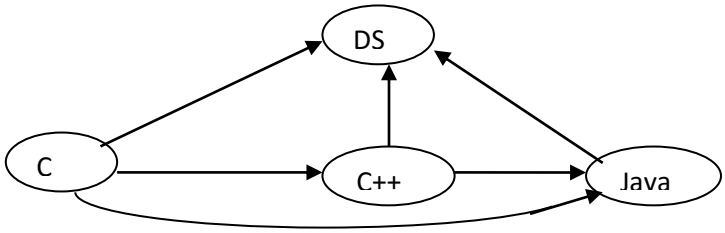
As we will pop one by one ,all nodes are ignored as visited .

Finally, the traversal is the order they are visited. That is as follows.

V1, V3, V7, V8, V6, V5, V4, V2

39. Topological Sorting

It is applicable to an acyclic graph only. It is a linear order of nodes of a graph.



University course curriculum

An appropriate example of topological sort is the university course curriculum design. Students should learn C course first and next they can learn any other programming language like c++ and java. The course DS (Data structure) can be learnt after studying any programming language only. Because we have to implement the algorithms using any language. Similarly java can be understood easily if we have studied c++ before it.

As C is the source node it's in degree is zero which does not depends on any other node. Arrows represent the dependency between nodes. So we need to arrange the nodes as per the degree of their dependency.

Algorithm:

1. Find any node having in degree 0.
2. Insert the node into the queue.
3. Delete the element from queue and mark it as visited
4. Decrease the degree of its adjacent node by 1.
5. Repeat from step 1 until the queue is empty.

Example:

In the above graph the indegree of each node is mentioned in the following table.

	C	DS	C++	Java
Indegree	0	3	1	2
Visit Status	0	0	0	0

	C	DS	C++	Java
C	0	1	1	1
DS	0	0	0	0
C++	0	1	0	1
Java	0	1	0	0

Adjacency Matrix

As node C is having degree zero let insert it into the queue and decrease the degree of its adjacent nodes and mark as visited.

	C			

Front 0
Rear 0

Indegree
Visit Status

C	DS	C++	Java
0	2	0	1
1	0	0	0

We can see that now c++ node has indegree in between the unvisited nodes. So let insert it into the queue and decrease the degree of its adjacent nodes and mark as visited.

C	C++		

Front 1
Rear 0

Indegree
Visit Status

C	DS	C++	Java
0	1	0	0
1	0	1	0

Now Java node has indegree in between the unvisited nodes. So let insert it into the queue and decrease the degree of its adjacent nodes and mark as visited.

C	C++	Java	

Front 2
Rear 0

Indegree
Visit Status

C	DS	C++	Java
0	0	0	0
1	0	1	1

Similarly DS is inserted to the queue.

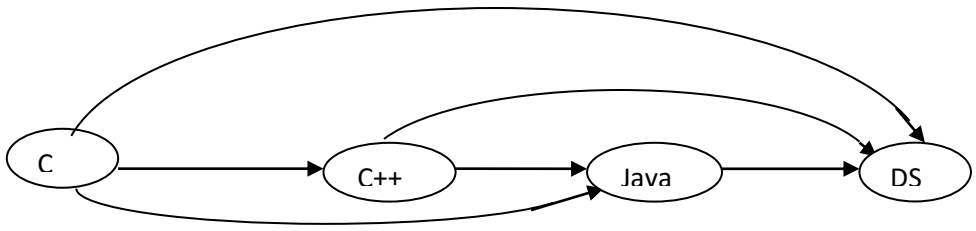
C	C++	Java	

Front 2
Rear 0

Indegree
Visit Status

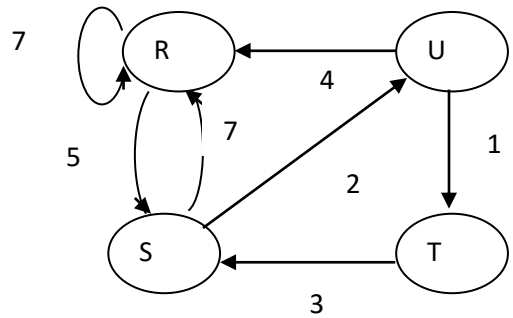
C	DS	C++	Java
0	0	0	0
1	1	1	1

The linear order of the graph is the order they are deleted and shown as follows.



40. Shortest Path Problems (Floyd Warshall's Algorithm)

Shortest Path problem is defined based on a Weighted Graph. A Graph is called weighted Graph if every edge has assigned a numerical value called weight or cost of the edge. A weighted graph is represented using a weight matrix which is defined as follows.



Weighted Graph

	R	S	T	U
R	7	5	0	0
S	7	0	0	2
T	0	3	0	0
U	4	0	1	0

Weight matrix

Shortest Path:

It is the minimum cost of the path which is sum of the weight of the edges along the paths from source to destination. For example in the above figure the shortest path from U to S is 4 where there are two costs 9 and 4.

Warshall's is used to find the shortest path among all pair of nodes in a graph. The input to the algorithm is the Weight matrix and the output is the all pair shortest path matrix.

The initial matrix q_0 is the weight matrix with infinite value if there is no edge between two vertices**.** We find the solution matrix by considering all vertices as an intermediate vertex. The idea is to one by one pick all vertices and update all shortest paths which include the picked vertex as an intermediate vertex in the shortest path. When we pick vertex number k as an intermediate vertex, we already have considered vertices {0, 1, 2, …… ,k-1} as intermediate vertices.

$$\mathbf{q_k [I][j] = MIN (q_{k-1} [I][j] , q_{k-1} [i][k] + q_{k-1} [k][j])}$$

Where k takes value from 0 to n.

q_0 is the modified weight matrix and q_1 , q_2 , q_3 , q_4 respectively q_R , q_S , q_T , q_U .

$q_0 =$

	R	S	T	U
R	7	5	∞	∞
S	7	0	∞	2
T	∞	3	0	∞
U	4	∞	1	0

Consider shortest path to all pair through R node. Then the matrix q_R is as follows.

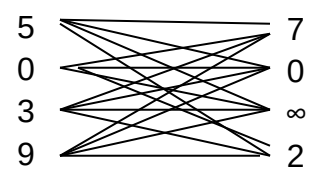
$q_R =$

	R	S	T	U
R	7	5	∞	∞
S	7	0	∞	2
T	∞	3	0	∞
U	4	9	1	0

Similarly we can consider shortest path to all pair through S, T and U node and the matrices are as follows.

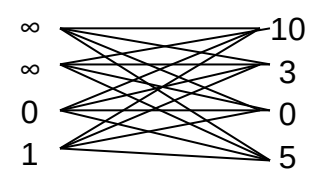
$q_S =$

	R	S	T	U
R	7	5	∞	7
S	7	0	∞	2
T	10	3	0	5
U	4	9	1	0



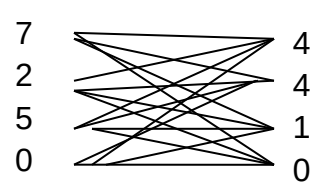
$q_T =$

	R	S	T	U
R	7	5	∞	7
S	7	0	∞	2
T	10	3	0	5
U	4	4	1	0



$q_U =$

	R	S	T	U
R	7	5	8	7
S	6	0	3	2
T	9	3	0	5
U	4	4	1	0



The c function for warshall's shortest path algorithm is as follows.

```
void warshall_shortest_path (  int m[ 10 ][ 10 ],int n )
{
        int i,j,k;
        int q [ n ][ n ];
        for (  i = 0 ; i < n ; i ++  )
        for (  j = 0 ; j < n ; j++ )
                if  (  i != j && m [ i ][ j ] == 0  )
                        m [ i ][ j ] == 99999;
                else
                        q[ I ][ j ] = m[ I ][ j ];
        for (  k = 0  ;k < n ; k ++ )
                for (  i = 0 ; i < n ; i ++  )
                        for (  j = 0 ; j < n ; j ++ )
                                q[ i ][ j ] =min (  q[ i ][ j ], q[ i ][ k ] + q[ k ][ j ]  ) ;
}
```

41. Sorting Techniques: Bubble, Selection, Insertion Sort

Sorting:

It is a technique to arrange some values in an order either ascending or descending order. There are different types of sorting technique as follows.

1. Selection Sort
2. Bubble Sort
3. Insertion Sort
4. Merge Sort
5. Quick Sort
6. Radix Sort
7. Heap Sort

Best case, worst case and average case input of sorting techniques:

When values are given in the desired order to sort, it is the best case input. If the values are completely given in the reverse order of the desired order, then it is the worst case input. Otherwise it's an average case input.

Some tips on sorting technique:

1. We don't need to sort, if there is a single element to sort.
2. If there are n elements, then we need n-1 values to arrange where the nth value is automatically arranged. So we conclude that if n elements are there we need n-1 pass to sort,
3. In each pass, one element is placed it its desired place. So we compare an element with other elements. So we need maximum n-1 comparison operation in each pass.
4. Form the above two statement, we sure that we need two loop where one is nested within another. Outer loop is for each pass and inner loop is for comparison.

Selection Sort:

The name is selection because we select the largest (or smallest) element and move it to its final position. We repeatedly select the next largest (or smallest) element in the array and move it to its final position in the sorted array.

Example:
Let take an array of five elements as following and sort it in ascending order.

30	**70**	**10**	**20**	**50**
0	**1**	**2**	**3**	**4**

As there are five elements we need 4 passes.

Pass 1:

30	**70**	10	20	50

Select **0th** element and compare with others elements to find the smallest element. The maximum element between two elements should be at the right side otherwise we need to swap it.

30	**70**	10	20	50	No swap
30	70	**10**	20	50	Swap
10	70	30	**20**	50	No swap
10	70	30	20	**50**	No swap

We saw that at the end of Pass 1, the smallest element is found.

Pass 2:

Similarly select 1st element and compare with the element to its right which are the unsorted elements.

10	**70**	**30**	20	50	Swap

10	**30**	70	**20**	50	Swap
10	**20**	70	30	**50**	No Swap

At the end of Pass 2, we got the second smallest element. Similarly other passes are as follows.

Pass 3:

10	**20**	**70**	**30**	50	Swap
10	**20**	**30**	70	**50**	No Swap

Pass 4:

10	**20**	**30**	**70**	**50**	Swap
10	**20**	**30**	**50**	70	

As all the passes are over the array is sorted now as shown above. The c function for selection sort is as follows. Its best case, worst case and average case time complexity is **O (n^2).** Selection sort requires N (N – 1)/2 comparisons.

```
void selection ( int a [  ], int n )
{
        int i, j, t;
        for ( i = 0;i< n-1; i++ )
        {

                for ( j = i+1 ; j < n ; j++ )
                {

                        if ( a [ i ] < a [ j ] )
                        {
                                t= a [ i ] ;
                                a [ i ] = a [ j ] ;
                                a [ j ]  = t;
                        }
                }
        }
}
```

Bubble Sort:

The bubble sort is a sort that uses a different scheme for finding the minimum (or maximum) value. Each pass the algorithm puts the smallest (or largest) unsorted element into its correct place, but it also makes changes in the locations of the other elements in the array.

We compare successive pairs of elements, swapping whenever the bottom element of the pair is smaller than the one above it. In this way the smallest element “bubbles up” to the top of the array

Example:

Let take an array of five elements as following and sort it in ascending order.

30	**70**	**10**	**20**	**50**
0	**1**	**2**	**3**	**4**

As there are five elements we need 4 passes.

Pass 1:

We compare adjacent pairs of elements, swapping whenever the right element of the pair is smaller than the left one.

30	**70**	10	20	50	No Swap
30	**70**	**10**	20	50	Swap
30	**10**	**70**	**20**	50	Swap
30	**10**	20	**70**	**50**	Swap
30	**10**	20	50	**70**	

Now we can see the maximum element is at its final position. Similar other passes can be completed as follows.

Pass 2:

30	**10**	20	50	**70**

Swap

10	**30**	**20**	50	**70**

Swap

10	**30**	**20**	50	**70**

Swap

Pass 3:

10	**20**	30	**50**	**70**

No Swap

10	**20**	**30**	**50**	**70**

No Swap

Pass 4:

10	**20**	**30**	**50**	**70**

No Swap

As all the passes are over the array is sorted now as shown above. The c function for bubble sort is as follows. Its best case, worst case and average case time complexity is **O (n^2).** Selection sort requires N (N – 1)/2 comparisons.

```
void bubble ( int a[  ], int n )
{
        int i, j, t;
        for ( i= 0 ; I < n ; i++ )
        {

                for ( j=0; j < n-1 ; j++ )
                {

                        if ( a [ j ] < a [ j+1 ] )
                        {
                                t= a [ j ] ;
                                a [ j ] = a [ j+1 ] ;
                                a [ j+1 ]  = t;
                        }
                }
        }
}
```

Insertion Sort:

The principle of the insertion sort is quite simple: the element to be sorted is inserted into its proper place with respect to some already sorted elements of the array. Initially, the sorted portion contains only one element: the first element in the array. Now we take the second element in the array and put it into its correct place in the sorted part; that is, values[0] and values[1] are in order with respect to each other. Now the value in values[2] is put into its proper place, so values[0]..values[2] are in order with respect to each other. This process continues until all the elements have been sorted.
Our aim is to search for the insertion point from the beginning of the array and shift the elements from the insertion point down one slot to make room for the new element. We can combine the searching and shifting by beginning at the end of the sorted part of the array. We compare the current element to the one before it, if it is less, we shift the element. We then compare the current -1 element to the one before it, and shift if necessary. The process stops when the comparison shows that the values are in order or we have reached at the first place in the array.

Example:
Let take an array of five elements as following and sort it in ascending order.

30	**70**	**10**	**20**	**50**
0	**1**	**2**	**3**	**4**

As there are five elements we need 4 passes.

30	70	10	20	50

Initially, the sorted portion contains only one element: the first element in the array.

Pass 1:

30	**70**	10	20	50

Element to insert

temp=70

Select the 1st element to insert it at its proper element and store it in a temporary variable. To find the proper place we need to compare this element to the sorted element from the right side. As they are in order there is not required of any shift operation.

Pass 2:

30	70	10	20	50

Element to insert

temp= 10

Similarly select the second element and compare it with the elements with the sorted part of the array from right side to find the insertion point. In the comparison if the temporary element is maximum then the element is inserted at that place otherwise the element in the sorted part is shifted to the right place.

30	70	10	20	50

Shifting as array [1] > temp

30	70	70	20	50

Shifting as array [0] > temp

30	30	70	20	50

As we reaches at beginning of the array and all the elements are larger than the temporary element, so the 0th place is the right place to insert. The updated array after insertion of the element is as follows.

10	30	70	20	50

Pass 3:

10	30	70	20	50

Element to insert

temp = 20

10	30	70	20	50

10	30	70	70	50

.

10	30	30	70	50

Here we stop shifting as 10 is less than temp and insert 20 at te place of 30 as follows

10	20	30	70	50

Pass 4:

10	20	30	70	50

Element to insert

temp=50

10	20	30	70	50

10	20	30	70	70

As 30 is less than 50 we are not going to shift any more element and the temp value is to be inserted at te place of 70.

10	20	30	50	70

Pass 4 is over and the array is sorted now. The maximum number of comparisons is made only when the elements in the array are in reverse order. When the data are in ascending order only one comparison for each element is made each time and no shifts are necessary. Its best case is **O (n)** and worst case and average case time complexity is **O (n^2).**The c function for insertion sort is as follows.

```
void insert ( int b[10],int n )
{
        int i,j,t,count=0;

        for ( i=1; i < n ; i ++ )
        {
                t=b [ i ] ;
                for ( j= ( i-1);j>=0 && b [ j ] > t ; j-- )
                {
                        b [ j+1 ]=b[ j ];

                }

                b[ j+1 ] = t ;
        }
}
```

42. Sorting Techniques: Radix, Quick Sort

Radix Sort:

Radix sort method uses the radix value for sorting .If the radix value is n , then it creates n buckets and places the values in the bucket and arrange it. The no of pass in a radix sort is the no of digits in the largest number.

Example:

Sort the following values in the radix sort.

152, 762, 45, 80, 350, 414, 143

The maximum number is 762 and it has three digits. Hence three passes is required.

Pass 1:
Step 1:

Check the unit digit and place them as per the digit bucket.

buckets						
0	**80**	**350**				
1						
2	**152**	**762**				
3	**143**					
4	**414**					
5	**45**					
6						
7						
8						
9						

Step 2:

Collect the values as per the bucket sequence. After collection the sequence is as follows.

80, 350, 152, 762 143, 414, 45

Pass 2:
Step 1:

Check the tenth digit and place them as per the digit bucket.

buckets						
0						
1	**414**					
2						
3						
4	**143**	**45**				
5	**350**	**152**				
6	**762**					
7						
8	**80**					
9						

Step 2:

Collect the values as per the bucket sequence. After collection the sequence is as follows.

414, 143, 45, 350, 152, 762, 80

Pass 3:
Step 1:

Check the hundredth digit and place them as per the digit bucket.

buckets						
0	**45**	**80**				
1	**143**	**152**				
2						
3	**350**					
4	**414**					
5						
6						
7	**762**					
8						
9						

Step 2:

Collect the values as per the bucket sequence. After collection the sequence is as follows and it is the sorted sequence.

45, 80, 143, 152, 350, 414, 762

The algorithm is as follows.

```
void bucket_sort   (  int a [   ]  ,  int n )
{
int max, dig, rem;
int i, j, k, p;
int d=1;

max= large  (  a, n ) ;
        dig=digit  (  max ) ;
        for  (  i=0; i<dig; i++ )
        {
                int col [ 10 ] ={0};
                for  (  j=0; j<n; j++ )
                {
                        rem=  (  a [ j ] /d ) %10;
                        bucket [ rem ]  [ col [ rem ] ++ ] =a [ j ] ;
                }
                p=0;
                for  (  j=0; j<10; j++ )
                {
                        for  (  k=0; k<=col [ j ] ; k++ )
                                printf  (  "%d", bucket [ j ]  [ k ]  ) ;
                        printf  (  "\n" ) ;
                }
                for  (  j=0; j<10; j++ )
                {
                        for  (  k=0; k<=col [ j ] ; k++ )
                                a [ p++ ] =bucket [ j ]  [ k ] ;
                }
                d*=10;
        }
        for  (  p=0; p<n; p++ )
                printf  (  "%d", a [ p ]  ) ;
        return  (  0 ) ;
}
```

```
//CODE TO FIND THE LARGEST VALUE
int large  (  int a [  ] ,int n )
{
        int i,largest=a [ 0 ] ;
        for  (  i=1; i<n; i++ )
        {
                if  (  largest < a [ i ]  )
                        largest = a [ i ] ;
        }
        return  (  largest ) ;
}
```

```
//CODE FOR DIGIT FUNCTION
int digit  (  int max )
{
        int ctr=0;
        while  (  max > 0 )
        {
                ctr++;
                max/=10;
        }
        return  (  ctr ) ;
}
```

Quick Sort :

This algorithm works by partitioning the array to be sorted and each partition is sorted recursively.
In a partition one of the element is chosen as pivot element and the rest of the element are grouped into two parts where one group contains elements smaller than the pivot and the other group contains elements greater than the pivot elements.
After the partition of elements the pivot element occupies its actual location of the sorted array.

<	<	Pivot element	>	>

Example: int a [5] = { 50 ,20, 10, 70, 30 };

50	**20**	**10**	**70**	**30**
0	1	2	3	4
low				high
p	i			j

Let take the 1st element as pivot.
p = a [low];
i =low +1;
j=high;

if p > a [i] ,then do i++
Otherwise check if p < a [j] ,then do j - -
After this process, the array is as follows where i is 3 and j is 4

50	**20**	**10**	**70**	**30**
0	1	2	3	4
low				high
p			**i**	**j**

As I < j, swap a [i] and a [j] . After swapping the array is as follows.

50	**20**	**10**	**30**	**70**
0	1	2	3	4
low				high
p			**i**	**j**

Now repeat the same process.
if p > a [i] ,then do i++
Otherwise check if p < a [j] ,then do j - -
After this process, the array is as follows where i is 4 and j is 3

50	**20**	**10**	**30**	**70**
0	1	2	3	4
low				high
p			**j**	**i**

As I > j, swap a [p] and a [j] . After swapping the array is as follows.

30	**20**	**10**	**50**	**70**

0	1	2	3	4
low				high
p			**j**	**i**

Now the pivot element is as its actual place and the array will be portioned as per the pivo
t position. One array is from low to previous position of pivot and the other array is from the next position of pivot to high. The partition is shown as follows.

30	**20**	**10**

50

70

0	1	2	3	4
Low		j-1		j+1
				High

Similar partition technique is applied to array having size more than one. The algorithm is as follows.

```
void quick_sort  (  int a [  ] ,int l, int h )
{
        int p,i,j,temp;
        if  (  l<h )
        {
                p=a [ l ] ;
                i=l+1;
                j=h;
                while  (  j>=i )
                {
                        while  (  a [ i ] <p )
                                i++;
                        while  (  a [ j ] >p )
                                j--;
                        if  (  j>i )
                        {
                                temp=a [ i ] ;
                                a [ i ] =a [ j ] ;
                                a [ j ] =temp;
                        }
                }
```

```
            temp=a [ l ] ;
            a [ l ] =a [ j ] ;
            a [ j ] =temp;
            quick_sort  (  a,l,  (  j-1 )  ) ;
            quick_sort  (  a,  (  j+1 ) ,h ) ;
            }
}
```

Best case time complexity is O (n log n) which is the average case also and the worst case time complexity is O (n^2).

43. Sorting Techniques: Merge Sort

Merging:

In a merging method we merge two sorted array and store in another array.

A1 =

10	**30**	**40**

A2=

20	**50**

A3=

10	**20**	**30**	**40**	**50**

Here A1 and A2 are two sorted array.A3 is the merged array.

Merge sort divides the array until the array size is one then merge the array in the order they are divide to get back the whole array. This is called divide and conquer technique.

Example:

50	**20**	**10**	**70**	**30**
0	1	2	3	4
low				high

We find middle of the array and divide it two parts first.

mid = (low + high) / 2 = (0 + 5) / 2 = 2

Next 1st array is from **low** to **mid** and the second array from **mid+1** to **high**.

50	**20**	**10**
0	1	2
low		high

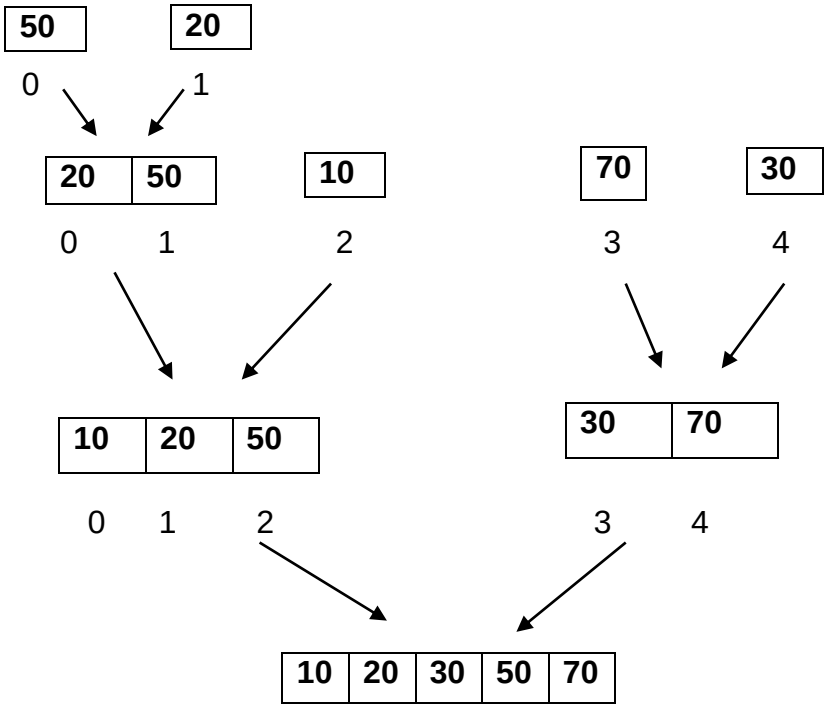
70	**30**
3	4
low	high

We continue partition of these arrays until the size of array is one.

50	**20**	**10**	**70**	**30**
0	1	2	3	4
low	high	low = high	low = high	low = high

50	20
0	1
low = high	low = high

As the size of all arrays is one, now we will merge them from the array they partitioned.



The algorithm of merging and merge sort is as follows.

```
void merge_sort  (  int a [  ]  , int low ,int high  )
{
 int mid;
if  ( low < high )
{
mid=   ( low + high  ) / 2 ;
merge_sort  ( a , low , mid  );
merge_sort  ( a , mid+1 ,high );
merge  ( a, low ,high , mid  )
}
}

void merge  (  int a  [  ]  , int l ,int h , int mid )
{
int temp  [h+1 ] ;
int l ,j ,k;
i = k = l;
j = mid + 1;
while  (  i ,<= mid && j <= h  )
```

```
{
if  (  a [i ]  < a [j ]   )
   temp  [ k ++  ]  = a  [ i++  ] ;
else
   temp  [ k++  ]  = a  [ j++ ] ;
}

while  (  i <= mid  )
    temp  [ k ++  ]  = a  [ i++  ] ;
while  ( j <= h )
   temp  [ k++  ]  = a  [ j++ ] ;
}
```

Time complexity of merge sort O (n log n)
Best case = Worst case = Average case = O (n log n)

44. Sorting Techniques: Heap Sort - Heapify

Heap Tree:

A heap tree is an almost complete binary Search tree or a complete binary tree which satisfies the following properties.
The property are based on the type of heap tree

a. Max heap
b. Min heap

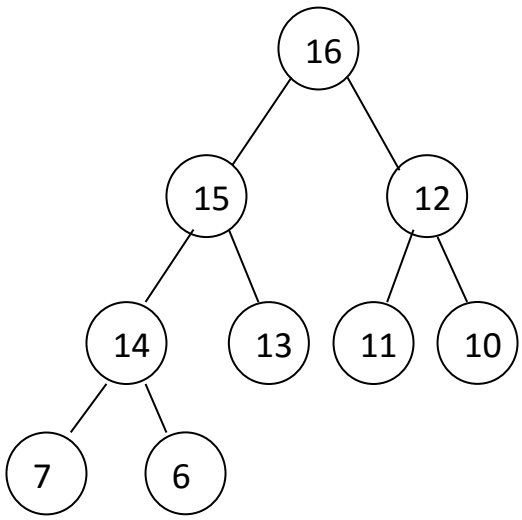
Max Heap tree property:

The parent value is greater than or equal to the value in the children node. Hence the root contains the maximum value in the tree.

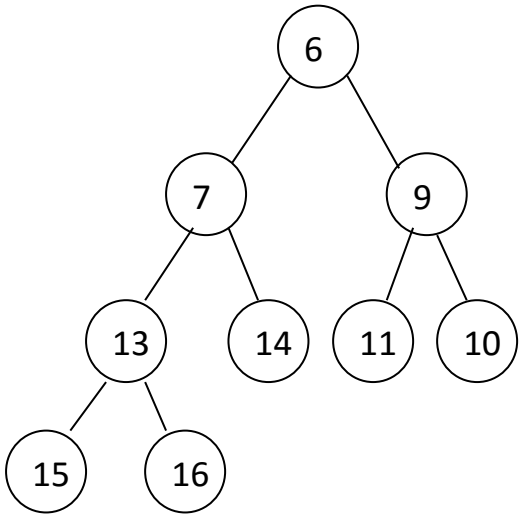
Min Heap tree property:

The parent value is smaller than the value in the children node. Hence the root contains the minimum value in the tree.

Example:



Max heap tree

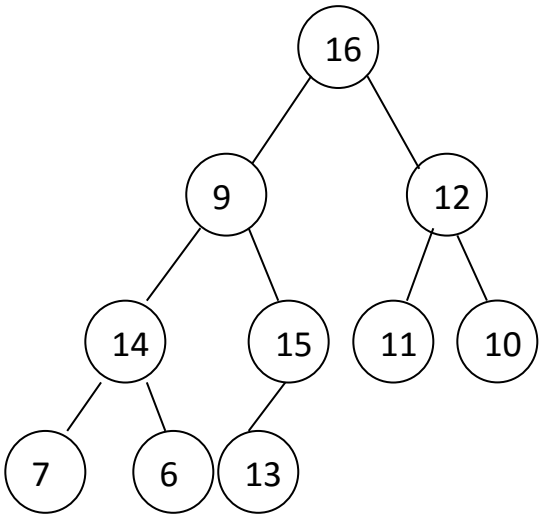


Min heap tree

Heapify Algorithm:

If the heap tree is not satisfying the heap property then we apply the heap satisfaction algorithm to the disorder node for maintain the property. This algorithm is called heapify.

Example:

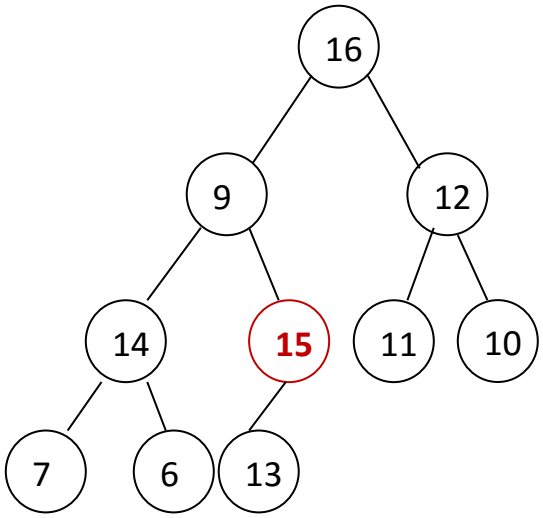


The array representation of the heap tree is as follows. If there are n levels then maximum array size is $2^{n+1} - 1 = 15$.

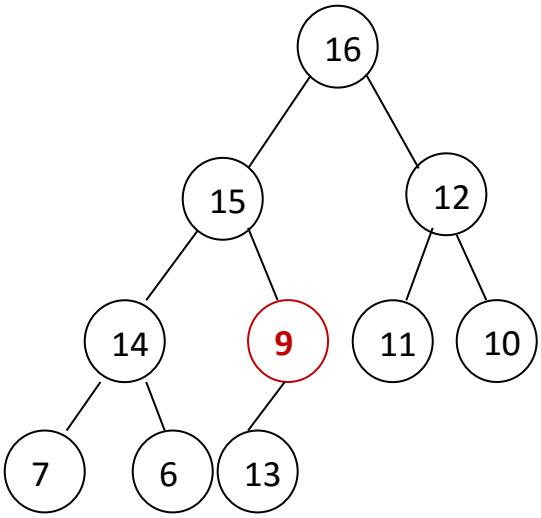
16	9	12	14	15	11	10	7	6	13					
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

The index of the array is used for inserting the heap element as following way. If parent is at index **i**, then left child (**lc**) is at **2i+1** and right child (**rc**) is at **2i+2**.

Here we will apply heapify algorithm on that node which violate the heap property i.e. 9.Max,lc,rc are taken in the algorithm for maintaining index of maximum ,left child and right child respectively. We find the maximum value among a node, its left child and its right child and store its index in max. Then the node value is swapped with the maximum value, if it's not the maximum value. . The algorithm is applied recursively until no more swapping occurs.

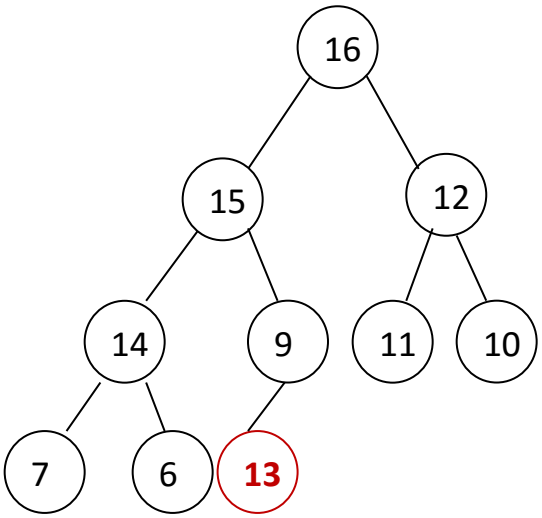


Before swapping

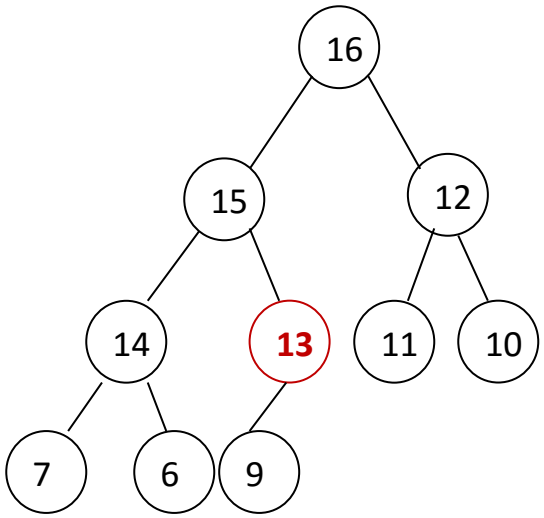


After swapping

Here we will apply heapify on 9 again by passing its index.



Before swapping



After swapping

The algorithm is described as follows. It takes log n times to reorder a node.

```
void heap  (  int a[ ], int n, int i )
{
        int max, lc, rc, t ;
        max = i ;
        lc =  (  2 * i ) + 1 ;
        rc =  (  2 * i ) + 2 ;
        if  (  lc <=  (  n – 1 )  &&  a [ lc ] > a [ i ] )
                max = lc ;
        if  (  rc <=  (  n – 1 ) &&  a [ rc ] > a [ max ] )
                max = rc ;
        if  (  a [ max ] != a [ i ] )
        {
                t = a [ max ] ;
                a [ max ] = a [ i ];
                a [ i ] = t ;
                heap  (  a , n , max ) ;
        }
}
```

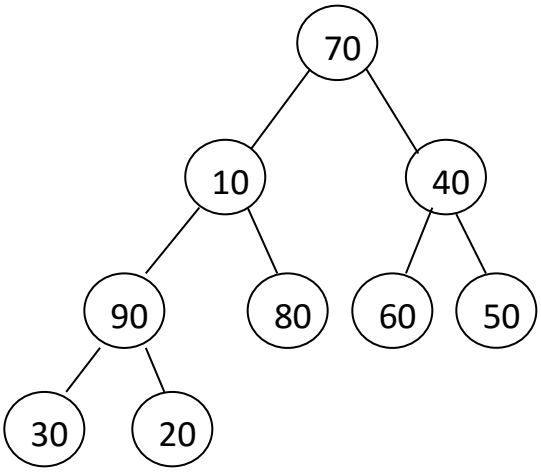
45. Sorting Techniques: Heap Sort – Build Heap

A heap sort technique creates a heap tree on the given set of values. Then the heap tree is used to sort the array. For building a heap tree first we need to find the last non leaf node from which we will start applying heapify down to the root index of the array. The last non leaf node index can be got from the size of the heap n.

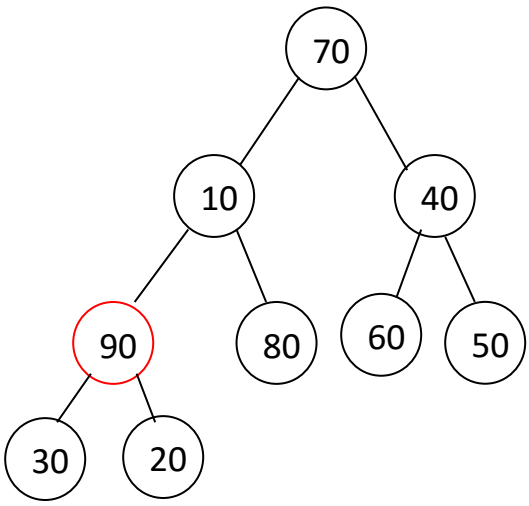
last non leaf node index = (n-2) / 2

Example:

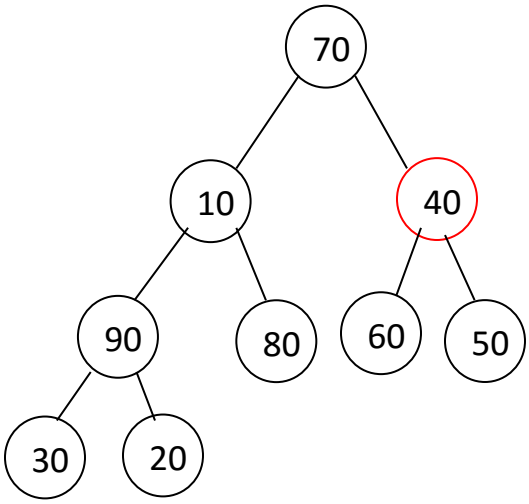
70	10	40	90	80	60	50	30	20
0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**



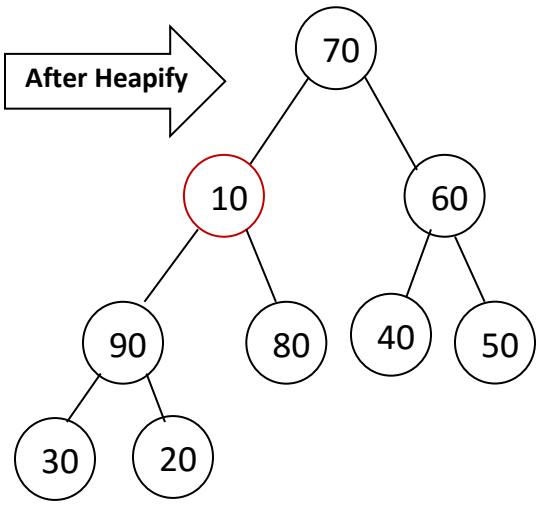
Binary tree of given array

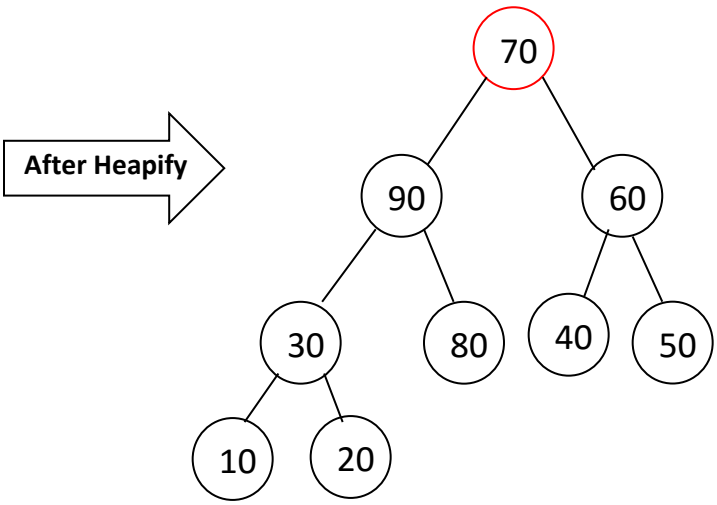


Applying haepify on node index 3



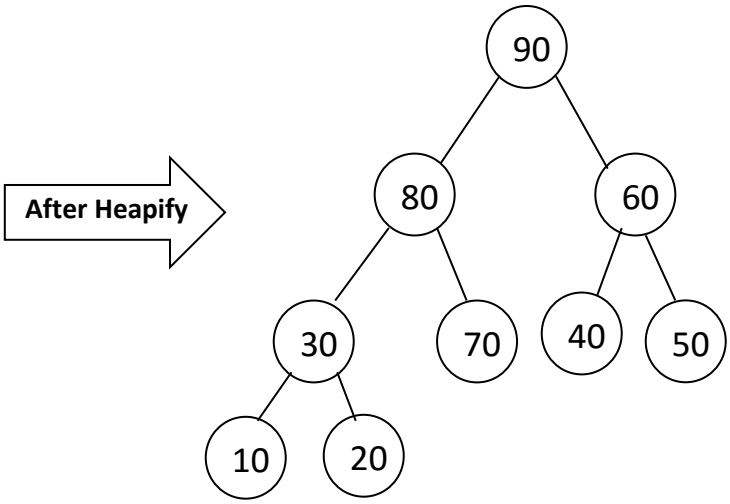
Applying haepify on node index 2





Applying haepify on node index 1

Applying haepify on node index 0



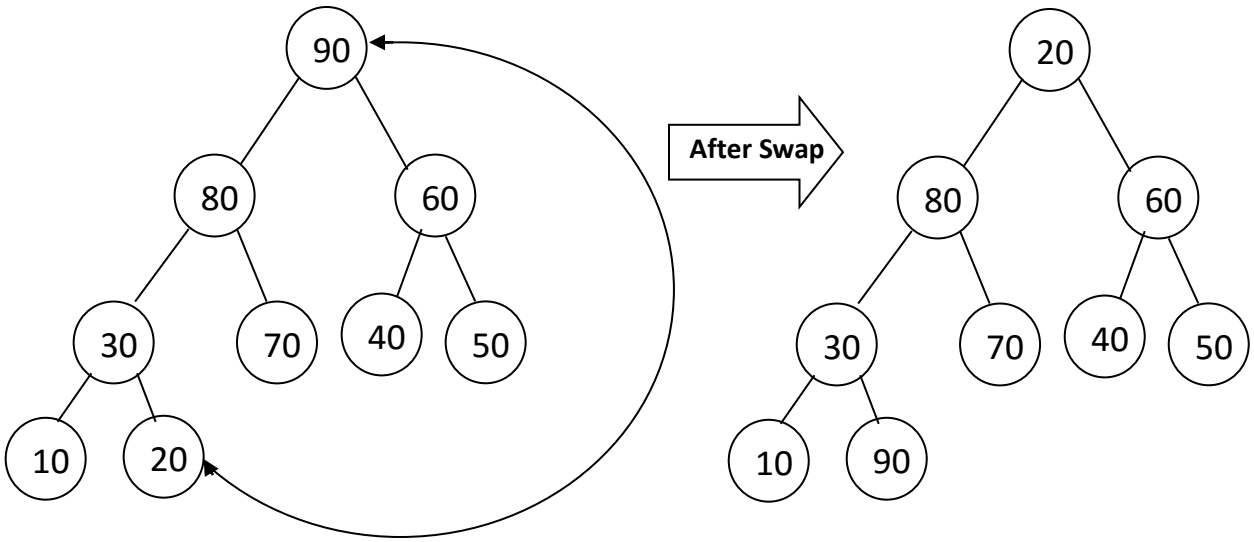
Constructed heap tree

The code for building a heap tree is as follows.

```
void buildheap ( int a[ ], int n )
{
        int i;
        for (  i =  (  n – 2 ) / 2 ; i >= 0 ; i -- )
                heap (  a, n, i );
}
```

Heap Sort using Heap tree:

We know that the maximum element is at the root node of a max heap tree and if we sort the values in ascending order the maximum element should be at the end of list. So the last and first element are swapped. Now the array found the largest element.



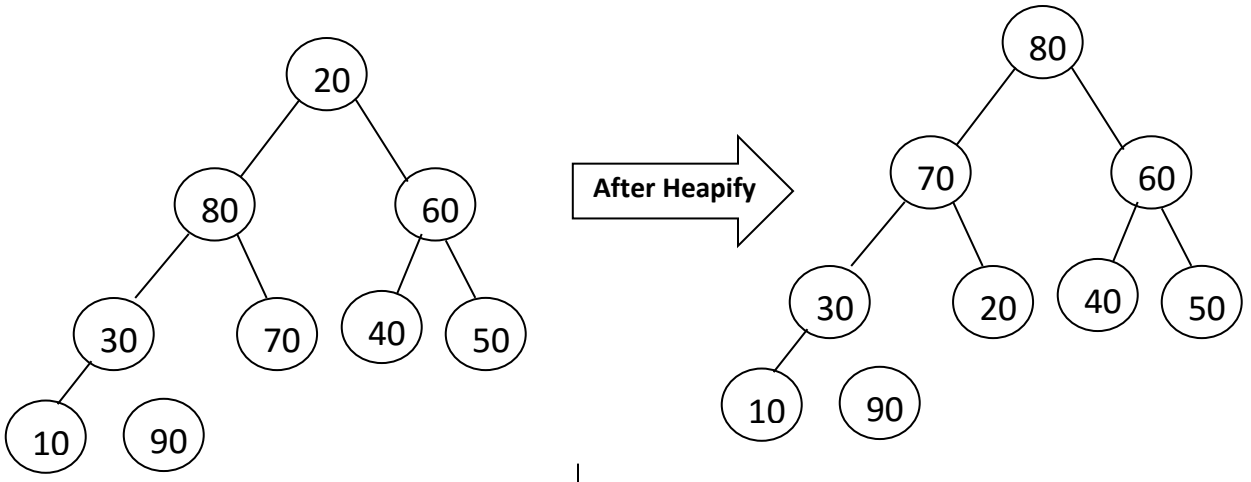
90	**80**	**60**	**30**	**70**	**40**	**50**	**10**	**20**
0	1	2	3	4	5	6	7	8

Before swapping

20	**80**	**60**	**30**	**70**	**40**	**50**	**10**	**90**
0	1	2	3	4	5	6	7	8

After swapping

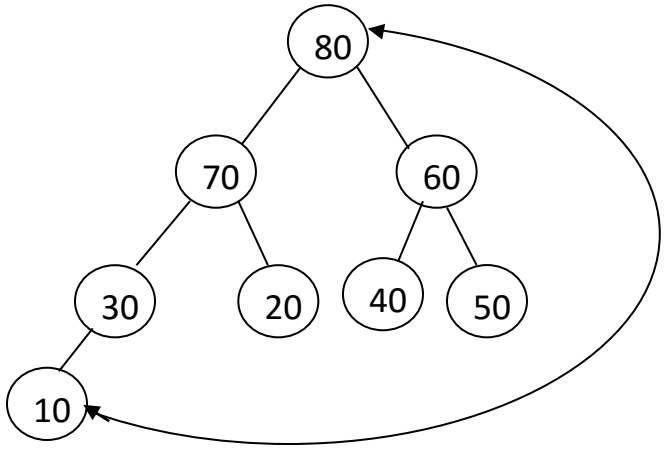
Now we reduce the array size by one. Then apply heapify on the root node to get the maximum value among these values.



20	**80**	**60**	**30**	**70**	**40**	**50**	**10**	**90**

80	**70**	**60**	**30**	**20**	**40**	**50**	**10**	**90**

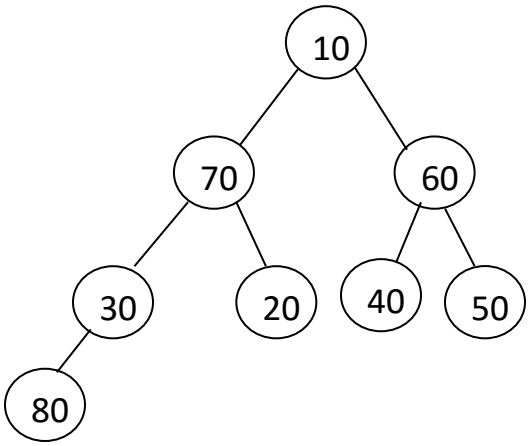
Tree after reducing the array size by one



80	**70**	**60**	**30**	**20**	**40**	**50**	**10**	**90**

Before swapping

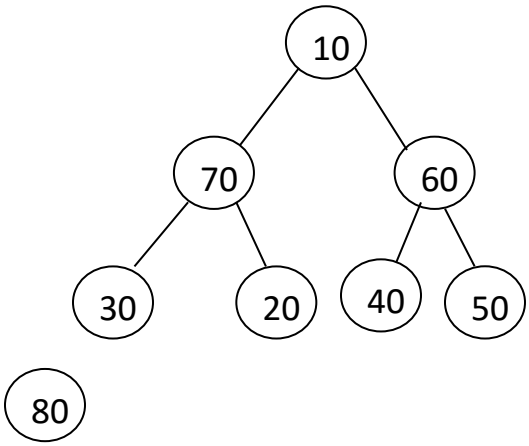
Tree after Heapify

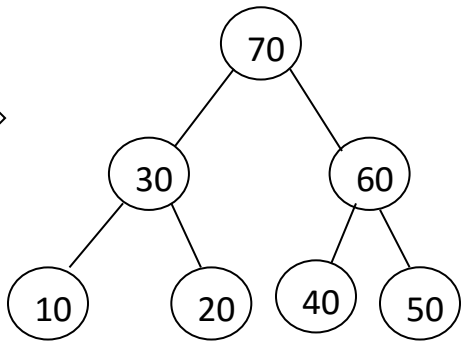


10	**70**	**60**	**30**	**20**	**40**	**50**	**80**	**90**

After swapping

Again we reduce the array size by one. Then apply heapify on the root node to get the maximum value among these values.

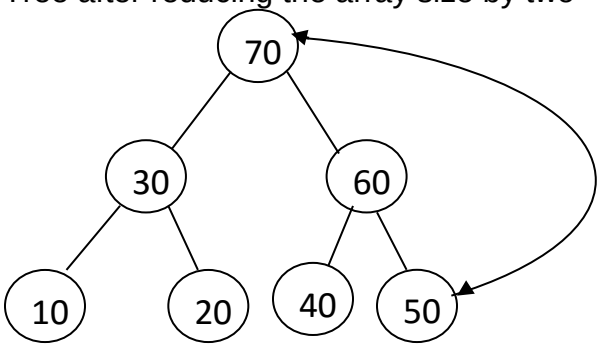




10	**70**	**60**	**30**	**20**	**40**	**50**	**80**	**90**

70	**30**	**60**	**10**	**20**	**40**	**50**	**80**	**90**

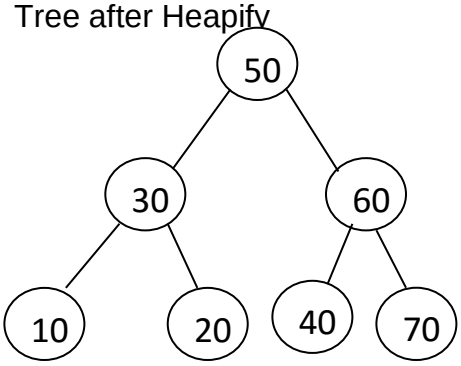
Tree after reducing the array size by two



70	**30**	**60**	**10**	**20**	**40**	**50**	**80**	**90**

Before swapping

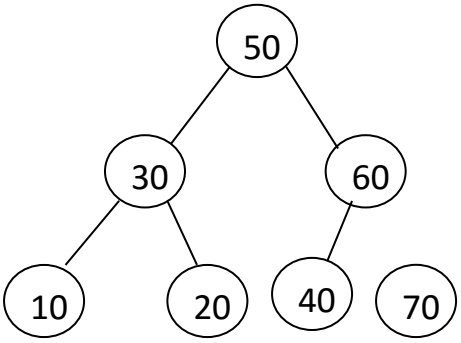
Tree after Heapify



50	**30**	**60**	**10**	**20**	**40**	**70**	**80**	**90**

After swapping

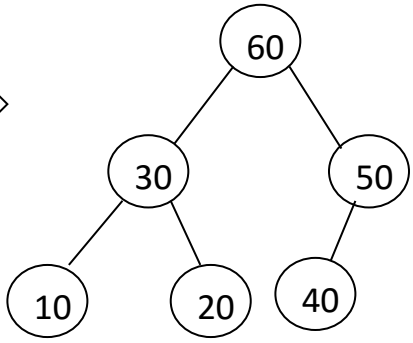
Similarly we reduce the array size by one. Then apply heapify on the root node to get the maximum value among these values.



50	**30**	**60**	**10**	**20**	**40**	**70**	**80**	**90**

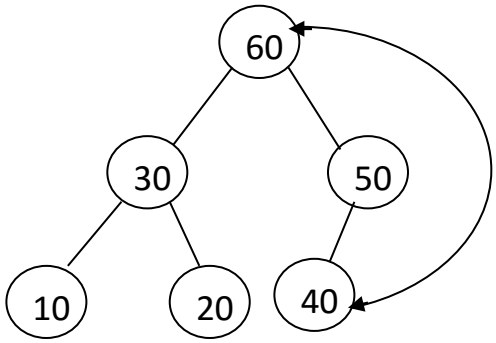
Tree after reducing the array size by three

After Heapify



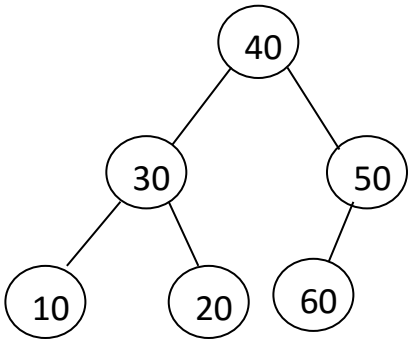
60	**30**	**50**	**10**	**20**	**40**	**70**	**80**	**90**

Tree after Heapify



60	**30**	**50**	**10**	**20**	**40**	**70**	**80**	**90**

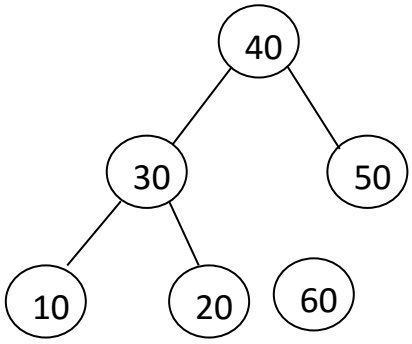
Before swapping



40	**30**	**50**	**10**	**20**	**60**	**70**	**80**	**90**

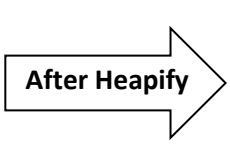
After swapping

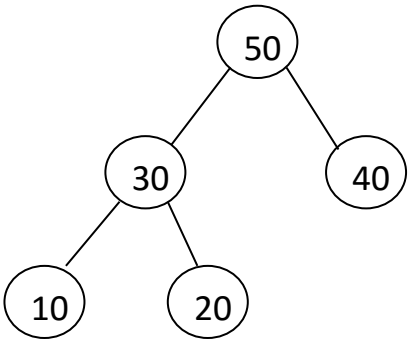
Here we reduce the array size by one. Then apply heapify on the root node to get the maximum value among these values.



40	**30**	**50**	**10**	**20**	**60**	**70**	**80**	**90**

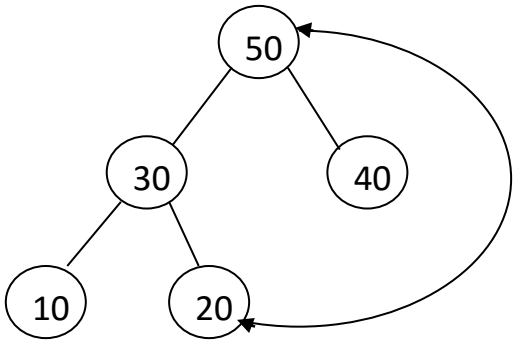
Tree after reducing the array size by four

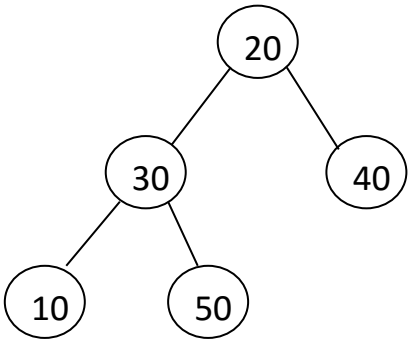




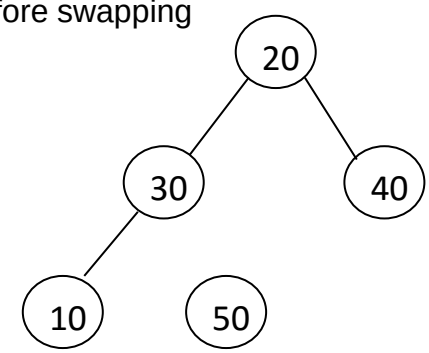
50	**30**	**40**	**10**	**20**	**60**	**70**	**80**	**90**

Tree after Heapify





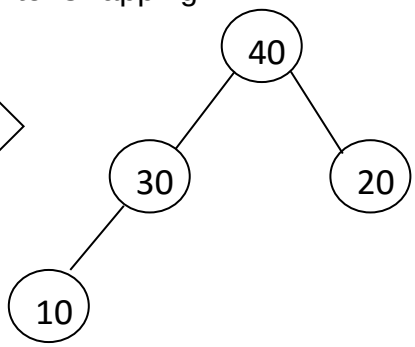
Before swapping



20	30	40	10	50	60	70	80	90

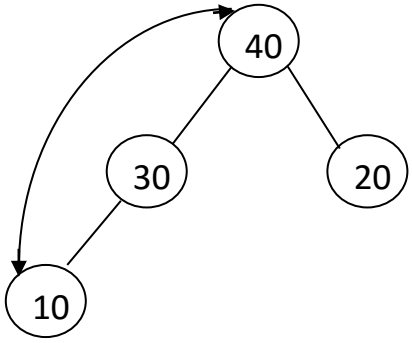
Tree after reducing the array size by five

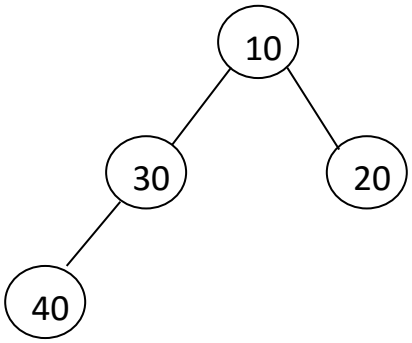
After swapping



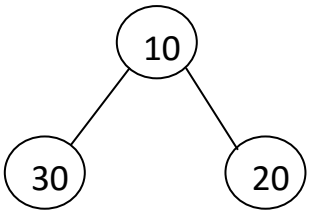
40	30	20	10	50	60	70	80	90

Tree after Heapify



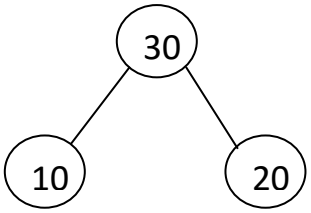


Before swapping



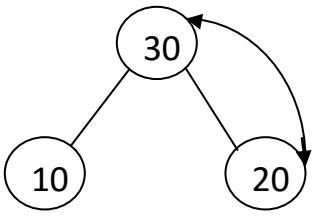
10	30	20	40	50	60	70	80	90

After swapping



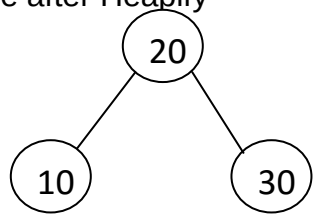
30	10	20	40	50	60	70	80	90

Tree after reducing the array size by six



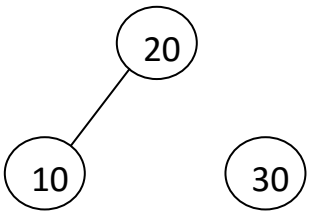
30	**10**	**20**	**40**	**50**	**60**	**70**	**80**	**90**

Tree after Heapify



20	**10**	**30**	**40**	**50**	**60**	**70**	**80**	**90**

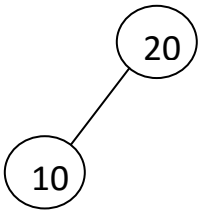
Before swapping



20	**10**	**30**	**40**	**50**	**60**	**70**	**80**	**90**

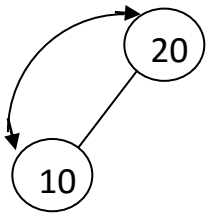
After Heapify

After swapping



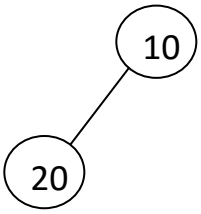
20	**10**	**30**	**40**	**50**	**60**	**70**	**80**	**90**

Tree after reducing the array size by seven



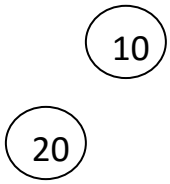
20	**10**	**30**	**40**	**50**	**60**	**70**	**80**	**90**

Tree after Heapify



20	**10**	**30**	**40**	**50**	**60**	**70**	**80**	**90**

Before swapping



After swapping

10	**20**	**30**	**40**	**50**	**60**	**70**	**80**	**90**

Tree after reducing the array size by eight

As the length of array is reduced to one, we stop the iteration. We got the sorted array. The algorithm for sorting an array using heap sort is as follows.

```
void heap_sort (  int a [ ], int n )
{
        int len, t;
        buildheap (  a , n );
        for (  len =  (  n-1 ); len>1; len -- )
        {
                t = a[ 0 ];
                a[ 0 ] = a[ len ];
                a [ len ] = t;
        }
}
```

The time complexity of heapify is $\log_2 n$ .So the time complexity of buildheap is $n \log_2 n$. Hence heap sort time complexity is $n \log_2 n$.

46 Searching Techniques: Linear Search, Binary Search
Searching:

It is a technique to find weather an element is present or nit in a list. If the element is present then it return the index where the element is present.
There are basically two different ways of searching

1. Linear Search
2. Binary Search

Best case, worst case and average case input of searching techniques:

If the value to search is found at the first place of the list, then it's the best case as one comparison was there. If the value is not found in the list, then all comparison were tried which is the worst case otherwise it's the average case.

Linear search:

In Linear Search the list is searched sequentially and the position is returned if the key element to be searched is available in the list, otherwise -1 is returned. The list need not be ordered.

Example:

30	**70**	**10**	**20**	**50**
0	**1**	**2**	**3**	**4**

Let take an array of five elements as above and the element to search is 20.

30	70	10	20	50	Compare and Not found
30	**70**	10	20	50	Compare and Not found
30	70	10	20	50	Compare and Not found
30	70	10	20	50	Found and return 3

The C function for linear search is as follows which best case is O (1) and worst case is O (n).

```
int linear_search ( int a[ ],int n,int k)
{
        int i ;
        for ( i = 0; i < n ; i ++ )
                if ( a[ i ] == k )
                        return k;
        return -1;
}
```

Binary search:

It works on sorted array only. In this method the middle element is compared first with the search value. If the element is not equal then the search value is consider to be available either in the left part or right of the middle value. If the searched value is less than the middle value then the value is available at its left part otherwise the value is available at the right part.

Example:

30	70	10	20	50
0	1	2	3	4

Let take an array of five elements as above and the element to search is 20.

Middle value index = (0 + 4) / 2 = 2. As a[2] < 20 the ,so we search the element in the right part of the middle element. Hence the new lower index value is middle index + 1 that is 3. So new middle index is (3 + 4) / 2 =3. As A[3] = = 20 , the index value 3 will be return. Similarly if we search the element in the left part of the middle value the higher index value is changed which is middle index – 1. The algorithm for binary search is as follows. The algorithm worst case is O (n log n) and best case is O (1).

```
int binary_search  ( int a [  ] ,int n,int k )
{
        int l=0, mid, r=n-1;
        while  ( l<= r )
        {
                mid=   ( l+r )/2;
                if  ( k== a [ mid ]  )
                {
                        return in;
                }
                else if  (  k> a [ mid ]  )
                        l= mid+1;
                else
                        r= mid-1;
        }
}
```

47. Hashing Techniques and Hash functions.

Hashing is the technique to search an item in the list. In case of linear search and binary search, the search operation depends on the no of elements but in case of hashing, it doesn't depends on the item present in the list.

Hashing technique uses Hash function to generate the address for a key value. The hash function can be any arithmetic equation. The popular one is the division method or modulo method.

Division Method:

This method divides the key value with the array size and the remainder is the address of the key. The size of the array needs to be a prime number for generating less no of repeated address.

$$H(k) = k \% m$$

Here m is table size. H (k) is the address of the key.

There are other methods for the Hash function. Two of them are discussed below.

Folding Method:

The key value is broken into pieces and then added up to find out the address.

$$H(k) = k_1 + k_2 + k_3 + - - - - - + k_n$$

For example H (234517) = 23 + 45 + 17 = 85

Mid-Square Method:

In a mid-square method, the address is the middle value of the square of the key value.

$$H(k) = \text{middle}(k^2)$$

For example if k is 125, H (k) = middle ($(125)^2$) = = middle (15625) = 6

48. Collision Resolution Techniques

Collision:

If the Hash function generates similar address for different key value then it is called a collision. To resolve this, there are three different ways of collision resolution technique.

1. Open Addressing
    i. Linear Probing
    ii. Quadratic Probing
    iii. Rehashing
2. Chaining

Linear Probing:

In this technique collision is resolved by the nest location of the generated hash function address. If the address generated is h and a collision is occurred, then it searches for h+1 location. If h+1 location is filled then h+2 location.

Example:

Place the following keys in the hash table of size 11 using division method and calculate the no of collision.

29, 18, 43, 10, 36, 25, 46, 26, 7

0	10
1	
2	46
3	36
4	25
5	
6	
7	29
8	18
9	
10	43

Linear Probing Table

The table is filled using division method where the function **h (k) = k mod m** is used. Here table size m is 11. So the address for a key is generated as follows.

K	**h (k)**	**Collision**
29	7	No
18	7	Yes
43	10	No
10	10	Yes
36	3	No
25	3	Yes
46	2	No
26	4	No

Quadratic Probing:

In a linear probing there is a possibilities of cluster of consecutive cells where the hash function generate the same address. The disadvantages of cluster is the increment of searching time as linear search. To overcome this problem the quadratic probing method is introduced.

Example:

Place the following keys in the hash table of size 11 using quadratic probing method and calculate the no of collision.

29, 18, 43, 10, 36, 25, 46, 26, 7

0	10
1	7
2	46
3	36
4	25
5	
6	
7	29
8	18
9	
10	43

Quadratic Probing table

Rehashing:

The hash function is again applied again and again on the data value if the collision occur.

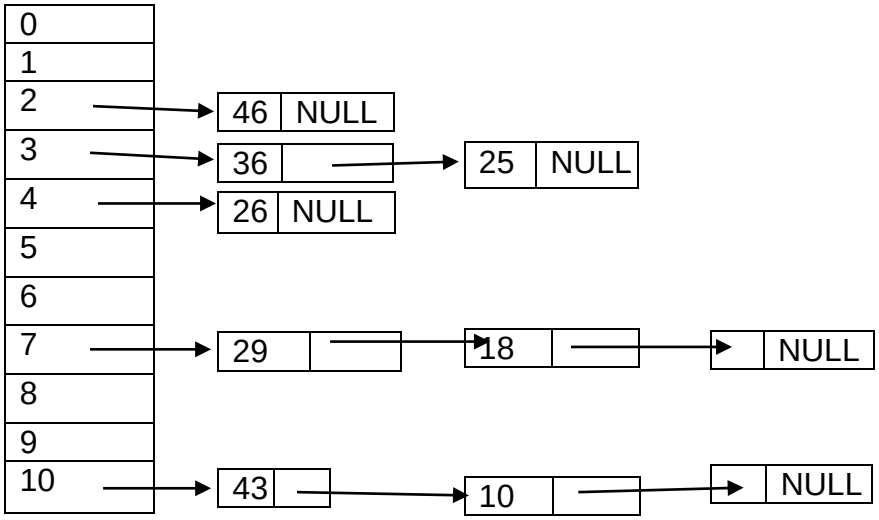
Chaining:

In this technique chain of elements are built for the values which are generating same address. An array of pointers is used to maintain the address of list values producing the same address.

Example:

Place the following keys in the hash table of size 11 using chaining method and calculate the no of collision.

29, 18, 43, 10, 36, 25, 46, 26, 7



Chaining Method List

DISCLAIMER:

The Data Structure Using C course material prepared includes some text, figures and diagrams taken from different books and web pages. The material is prepared for the use of students in the Silicon Institute of Technology as a preparatory material for BPUT examinations and not for any commercial purpose.